

MCS26_V26_01

Mission Control System Operations Simulator

User Manual

Document Version: 1.0

Software Baseline: MCS26_V26_01

Date: August 2026

Prepared by

Hoshyar Iranpoor

[Alireza Iranpoor Mobarakeh]

[linkedin.com/in/alireza-iranpoor-mobarakeh-53080a307](https://www.linkedin.com/in/alireza-iranpoor-mobarakeh-53080a307)

Educational Use Context

DLR_School_Lab TU Darmstadt

Educational and training use only. Not intended for flight operations or for the control of real spacecraft.

Table of Contents

0 Introduction	11
0.1 Spacecraft Operations and Training Context	11
0.2 MCS26_V26_01 System Overview	11
0.3 Scope and Intended Use	12
0.3.1 Training Scope	12
0.3.2 User Roles and Operational Interaction	12
0.3.3 Functional Boundaries	12
0.4 Simulation Fidelity, Data Provenance and Engineering Assumptions	13
0.4.1 Synthetic Telemetry and Commanding Model	13
0.4.2 Representation of Space-to-Ground Visibility and Telemetry Availability	13
0.4.3 Standards Context and Non-Qualification	13
0.5 Development Context, Ownership and Institutional Status	13
1 Motivation for Development	15
1.1 Value of the Existing Telespazio Training Concept	15
1.2 Telespazio Simulation and Training Software as an Engineering Reference	15
1.3 Limitations of the Local Legacy Installation	16
1.4 Rationale for Migration to a Web-Based Architecture	16
1.5 Preservation of Operational Training Value During the Migration	17
1.6 Engineering Trade-Off and Development Decision	18
2 Required Tools and Prerequisites	19
2.1 System Configuration Baseline	19
2.2 Host Workstation Prerequisites	19
2.2.1 Operating System and Command Environment	19
2.2.2 Node.js Runtime and npm	19
2.2.3 Git and Repository Access	20
2.2.4 Project Directory Integrity	20
2.2.5 Port Availability on the Host	20
2.2.6 Host Hardware Considerations	20
2.3 Operator Workstation Prerequisites	21
2.3.1 Browser Requirement	21
2.3.2 Client-Side Software and Role Selection	21
2.3.3 Display and Media Support	21
2.4 Network Prerequisites	21
2.4.1 Local Network Topology and Addressability	21
2.4.2 Service Interfaces and Connection Logic	22
2.4.3 Firewall and Local Security Policy	22
2.4.4 Trust Boundary and Exposure Limitations	22
2.4.5 Internet Connectivity	22

2.5 Software Dependency Baseline	23
2.6 Components Not Required by the Current Baseline	23
2.7 Pre-Installation Readiness Criteria	24
3 Installation, Configuration and Initial System Bring-Up.....	25
3.1 Installation and Deployment Sequence	25
3.2 Obtaining the Software from GitHub	25
3.2.1 First-Time Repository Clone	25
3.2.2 Updating an Existing Repository	26
3.2.3 Existing Local Modifications	27
3.3 Installing the Project Dependencies	27
3.3.1 Runtime Verification	27
3.3.2 Standard Dependency Installation.....	27
3.3.3 Clean Dependency Reinstallation	27
3.4 Pre-Operational Build Verification	28
3.5 Starting the Host / Simulation Server	28
3.5.1 Standard Startup Using the Supplied Script.....	28
3.5.2 Expected Host Processes	28
3.5.3 Port 5173 Verification	28
3.5.4 Port 3001 Verification	29
3.6 Determining the Host IPv4 Address	29
3.6.1 Using the Supplied Host-IP Script	29
3.6.2 Selecting the Correct Address	29
3.7 Firewall Configuration and Network Verification	29
3.7.1 Windows Firewall Permission	29
3.7.2 TCP Reachability Test from an Operator Workstation	30
3.8 Opening the Operator Clients.....	30
3.8.1 Direct Role URLs	30
3.8.2 Optional Role-Launcher Scripts.....	30
3.9 Initialising a Multi-Console Session.....	31
3.9.1 Connection Order	31
3.9.2 Scenario Selection on Every Client	31
3.9.3 Local Panel Navigation	31
3.9.4 Starting the Simulation	31
3.10 Initial Bring-Up Acceptance Test.....	32
3.11 Controlled Reset and Restart Between Exercises	32
3.11.1 Why a Host Restart Provides the Cleanest Baseline	32
3.11.2 Clean Restart Procedure	32
3.12 Installation Completion Criteria	33
4 Spacecraft System Engineering Fundamentals and MCS26 Simulation Model	34
4.1 System-Level View of a Spacecraft.....	34
4.1.1 Spacecraft as an Integrated System	34

4.1.2 Functional Chains and Interface Dependencies	36
4.2 Mission-Operations View of Spacecraft State	36
4.2.1 Engineering Telemetry, Mission Data and Reporting Data	36
4.2.2 Telecommands and Onboard Actions	36
4.2.3 Onboard State versus Ground Observability	36
4.3 Ground-Station Contact Geometry and RF Acquisition	37
4.3.1 Local Topocentric Reference Frame	37
4.3.2 Azimuth	37
4.3.3 Elevation	37
4.3.4 Elevation Mask	37
4.3.5 Acquisition of Signal and Loss of Signal	38
4.3.6 Slant Range	38
4.3.7 Doppler Shift	39
4.3.8 Link-Quality Quantities	39
4.3.9 Receiver Acquisition Chain	39
4.3.10 MCS26 Ground-Station Geometry Model	40
4.4 Electrical Power System (EPS)	41
4.4.1 EPS Functions and Architecture	41
4.4.2 Power, Current and Energy Balance	41
4.4.3 Solar Arrays and Power Conditioning	41
4.4.4 Batteries, State of Charge and State of Health	42
4.4.5 EPS Interfaces with Other Subsystems	42
4.4.6 EPS Representation in MCS26	42
4.5 Thermal Control System (TCS)	43
4.5.1 Spacecraft Thermal Environment	43
4.5.2 Passive and Active Thermal Control	43
4.5.3 Thermal Limits, Margins and Trends	43
4.5.4 TCS Representation in MCS26	44
4.6 Attitude and Orbit Control System / Guidance, Navigation and Control	44
4.6.1 Purpose of AOCS	44
4.6.2 Reference Frames	45
4.6.3 Attitude Representation: Euler Angles and Quaternions	45
4.6.4 Attitude and Navigation Sensors	45
4.6.5 Actuators and Momentum Management	45
4.6.6 Estimation, Guidance and Control Loop	46
4.6.7 Pointing Knowledge, Pointing Error and Settling	46
4.6.8 Orbit and Navigation State	46
4.6.9 AOCS Representation in MCS26	47
4.7 Command and Data Handling (C&DH)	47
4.7.1 Onboard Computer and Processing	47
4.7.2 Onboard Data Buses and Interfaces	47

4.7.3 Memory and Mass Data Storage	48
4.7.4 Command, Telemetry, Time and Fault Management	48
4.7.5 C&DH Representation in MCS26	48
4.8 Spacecraft Telecommunications and the Ground Link	49
4.8.1 Spacecraft RF Chain	49
4.8.2 Link Budget and Margin.....	49
4.8.3 MCS26 Communications Representation	49
4.9 Optical Imaging Payload.....	49
4.9.1 Payload Function and Spacecraft Interfaces	49
4.9.2 Optical Chain and Focal Plane	50
4.9.3 Calibration and Timing.....	50
4.9.4 Image Data Generation, Compression and Storage	50
4.9.5 Payload Readiness.....	50
4.9.6 Payload Representation in MCS26	50
4.10 Structure, Mechanisms and Propulsion in the System Context	51
4.10.1 Structure and Mechanisms.....	51
4.10.2 Propulsion.....	51
4.11 Cross-Subsystem Dependencies Used in Operations	51
4.11.1 EPS-TCS Coupling	51
4.11.2 EPS-Payload Coupling	51
4.11.3 AOCS-Payload Coupling	52
4.11.4 Payload-C&DH Coupling	52
4.11.5 Communications-C&DH and Communications-All-Subsystem Coupling	52
4.11.6 Time as a Cross-Cutting Interface	52
4.11.7 Contingency Coupling in Scenario 2	52
4.12 Interpretation of the MCS26 Engineering Model	53
4.12.1 Synthetic Telemetry.....	53
4.12.2 Deterministic Variation versus Physical Simulation	53
4.12.3 Status Colours and Limits.....	53
4.12.4 What the Model Can and Cannot Demonstrate.....	54
5 Mission Scenario Operations	55
5.1 General Rules Applicable to Both Scenarios	55
5.1.1 Role Authority and Console Discipline	55
5.1.2 Procedure-State Indications	55
5.1.3 SPACON SELECT-ARM-GO Execution.....	55
5.1.4 Link-Quality Command Gating	56
5.1.5 Mission Time, Fixed Windows and Event-Driven Steps	56
5.1.6 Interpretation of Approximate UI Durations and Role Text.....	57
5.2 Scenario 1 - Nominal Ground Pass and Frankfurt Airport Imaging.....	57
5.2.1 Mission Objective and Training Intent	57
5.2.2 Initial Conditions and Fixed Mission Events	57

5.2.3 Phase A - GS1 Acquisition and Downlink Validation (Steps 1-6)	58
5.2.4 Phase B - Payload Memory and Standby Verification (Steps 7-11)	59
5.2.5 Phase C - EPS Assessment and Thermal Mitigation (Steps 12-16)	60
5.2.6 Phase D - Payload Restoration, Imaging and Standby (Steps 17-21)	61
5.2.7 Scenario 1 Completion State	62
5.3 Scenario 2 - Communications and Thermal Contingency	62
5.3.1 Mission Objective and Scenario Logic.....	62
5.3.2 Initial Conditions and Time-Dependent Events	63
5.3.3 Phase A - Common Initial Pass (Steps 1-11)	64
5.3.4 Phase B - Battery Emergency Detection and Procedure Hold (Step 12)	64
5.3.5 Phase C - Emergency Communication with GNC	64
5.3.6 Phase D - Importing the Contingency Procedure	65
5.3.7 Phase E - Battery Recovery and Low-Power Configuration (Steps 13-17)	65
5.3.8 Phase F - Thermal Assessment and GS1 Loss (Steps 18-19).....	66
5.3.9 Phase G - GS2 Acquisition and Telemetry Recovery (Steps 20-22).....	66
5.3.10 Phase H - EPS Thermal Recovery after GS2 Reacquisition (Steps 23-26)	67
5.3.11 Phase I - Payload Restoration and Camera Readiness (Steps 27-30)	68
5.3.12 Phase J - Recovered Frankfurt Imaging and Standby (Steps 31-32).....	69
5.3.13 Scenario 2 Completion State	70
5.4 Time-Critical Decision Aids	70
5.5 Command Rejection and Immediate Operator Response	71
5.6 Operator Completion Checks	71
5.6.1 Scenario 1 Completion Check	71
5.6.2 Scenario 2 Completion Check	71
6 Troubleshooting, Fault Isolation and Recovery	72
6.1 Troubleshooting Philosophy	72
6.1.1 Separate Infrastructure Faults from Simulated Mission Conditions.....	72
6.1.2 Use the Least-Disruptive Diagnostic Action First	72
6.1.3 Preserve Diagnostic Evidence Before a Full Reset.....	72
6.2 Rapid Fault-Isolation Matrix.....	73
6.3 Host Startup and Runtime Failures	74
6.3.1 start-host-dev.bat Does Not Start Correctly.....	74
6.3.2 'node' or 'npm' Is Not Recognised	74
6.3.3 Missing Package or Module Errors.....	74
6.3.4 npm Cache Problems	74
6.3.5 Build Failure.....	74
6.3.6 Port 3001 Already in Use.....	75
6.3.7 Frontend Starts on 5174 or Another Port	75
6.3.8 Vite Dependency-Optimisation Cache Suspected.....	75
6.4 Host IP, LAN and Firewall Problems	75
6.4.1 Selecting the Correct IPv4 Address.....	75

6.4.2 Test TCP Services, Not Only Ping	76
6.4.3 Frontend Reachable Locally but Not from Other Computers	76
6.4.4 Windows Firewall.....	76
6.4.5 Wireless Client Isolation or Segmented Networks.....	76
6.5 WebSocket Synchronisation Faults.....	76
6.5.1 Architecture Relevant to Troubleshooting	76
6.5.2 Expected Browser Console Messages.....	77
6.5.3 Expected WebSocket Server Messages	77
6.5.4 Frontend Loads but Consoles Do Not Share Mission State	77
6.5.5 Shared State Persists When Browsers Are Closed.....	77
6.5.6 SOM Browser Refresh or Loss During an Active Run.....	77
6.5.7 Non-SOM Client Refresh During an Active Run.....	78
6.6 Browser Cache and Stale Frontend Recovery.....	78
6.6.1 Typical Cache Symptoms.....	78
6.6.2 Standard Hard Reload.....	78
6.6.3 Empty Cache and Hard Reload.....	78
6.6.4 Distinguish Browser Cache from Server State	78
6.6.5 Do Not Hard-Reload the SOM During an Active Exercise.....	78
6.7 Role URL and Local Scenario-State Problems	78
6.7.1 Wrong Operator Role	78
6.7.2 Scenario Selection Is Local	79
6.7.3 Panel Selection Is Local and Is Not a Synchronisation Test	79
6.8 Telemetry Availability: Expected States versus Faults.....	79
6.8.1 Before Simulation Start.....	79
6.8.2 RUNNING but Before GS1 AOS	79
6.8.3 After AOS but Below the Elevation Mask	79
6.8.4 Scenario 2 GS1 Loss.....	79
6.8.5 Scenario 2 GS2 Acquisition.....	79
6.8.6 Persistent NO TELEMETRY after a Valid Link.....	80
6.9 Procedure and Telecommand Failures	80
6.9.1 Always Read the Complete Failure String.....	80
6.9.2 Repeated Failure after the Correct Precondition Is Met	81
6.10 Scenario 1 Specific Troubleshooting.....	81
6.10.1 Memory Dump Appears Slow.....	81
6.10.2 EPS Mitigation Does Not Complete.....	81
6.10.3 Procedure Returns from Payload Preparation to EPS Mitigation	81
6.10.4 Frankfurt Image Is Wrong or Invalid	81
6.10.5 Imaging Window Missed.....	81
6.11 Scenario 2 Specific Troubleshooting.....	81
6.11.1 Battery Emergency Does Not Appear	81
6.11.2 Emergency Workflow Appears Stuck	81

6.11.3 GS1 Is Lost but GS2 Does Not Appear Immediately.....	82
6.11.4 GS2 Geometry Is Present but Spacecraft Panels Still Show NO TELEMETRY	82
6.11.5 GS2 Signal Remains MEDIUM.....	82
6.11.6 Payload Restoration Is Invalidated before T+30:00.....	82
6.11.7 Image Taken after T+30:30	82
6.12 Audio, Video and Image Asset Problems.....	82
6.12.1 Browser Autoplay Restrictions.....	82
6.12.2 Welcome Video Does Not Start or Finish	82
6.12.3 End Video Does Not Play	82
6.12.4 Captured Image Is Missing.....	83
6.13 Browser Developer Tools as a Diagnostic Instrument	83
6.13.1 Console	83
6.13.2 Network	83
6.13.3 Do Not Use DevTools to Modify State during Training.....	83
6.14 Source-Control and Version-Mismatch Problems	83
6.14.1 Confirm the Host Revision.....	83
6.14.2 Software Updated but Browser Still Shows Old Behaviour	83
6.14.3 Git Pull Refuses to Fast-Forward	83
6.15 Known Architectural Limitations Relevant to Troubleshooting	84
6.16 Controlled Recovery Procedures	84
6.16.1 Recovery of One Non-SOM Operator Console	84
6.16.2 Full Clean Restart.....	84
6.16.3 Rebuilding a Suspect Host Installation	85
6.17 Troubleshooting Acceptance Criteria	85
7Appendix A : Research Roadmap	86
7.1 Development Boundary and Research Objectives	86
7.1.1 Current Baseline.....	86
7.1.2 Research Objectives beyond the Training Baseline.....	86
7.2 Architecture Required before Increasing Model Fidelity	87
7.2.1 Move Authoritative Simulation Time to the Host.....	87
7.2.2 Introduce a Telemetry-Provider Interface	88
7.2.3 Separate Data Generation from Presentation	88
7.2.4 Server-Side Scenario Orchestration.....	88
7.3 Track A - Integration of Real ESA Spacecraft Telemetry.....	88
7.3.1 Identification of the Relevant ESA Dataset.....	88
7.3.2 Scale and Engineering Content of ESA-AD	89
7.3.3 Why ESA-AD Is Valuable for MCS26 Research.....	89
7.3.4 Anonymisation Constraints.....	89
7.3.5 Data Acquisition and Configuration Control.....	89
7.3.6 Ingestion and Normalisation Pipeline	90
7.3.7 Replay Engine	90

7.3.8 Telecommands in Recorded Data	90
7.3.9 Constructing Training or Research Scenarios from ESA-AD Events	90
7.3.10 Limitations of ESA-AD for a Commandable Spacecraft Simulator	90
7.4 OPS-SAT Telemetry as a Complementary Named-Mission Dataset.....	91
7.5 Track B - Physics-Based Spacecraft Models in Python	91
7.5.1 Model-Service Architecture	91
7.5.2 EPS and Solar-Array Model	91
7.5.3 Battery Model	92
7.5.4 Thermal Model.....	92
7.5.5 AOCS Model.....	92
7.5.6 Orbit, Environment and Ground-Station Geometry	92
7.5.7 Communications Model	92
7.5.8 Payload Model.....	93
7.5.9 C&DH and Data-System Model.....	93
7.5.10 Fault and FDIR Models.....	93
7.6 Simulation-Model Portability and Standards-Based Integration.....	94
7.6.1 ECSS Simulation Modelling Platform	94
7.6.2 Model Reuse across Development Stages	94
7.7 Track C - Hybrid Recorded-Data and Physics-Based Simulation	94
7.7.1 Purpose of a Hybrid Mode.....	94
7.7.2 State Estimation before Branching	94
7.7.3 Counterfactual Limitations	94
7.8 Verification, Validation and Model Credibility	94
7.8.1 Verification versus Validation	94
7.8.2 Verification Activities.....	95
7.8.3 Cross-Subsystem Invariants.....	95
7.8.4 Validation Data and the Role of ESA-AD	95
7.8.5 Uncertainty and Fidelity Statement.....	95
7.9 Data Engineering for Large-Scale Telemetry	95
7.9.1 Storage Strategy.....	95
7.9.2 Indexing and Query Model	96
7.9.3 Decimation for Visualisation	96
7.9.4 Provenance and Reproducibility.....	96
7.10 Anomaly Detection and AI-Assisted Operations Research.....	96
7.10.1 ESA-ADB as a Benchmark rather than only a Dataset	96
7.10.2 Advisory, Not Autonomous Commanding.....	96
7.10.3 Relevant Research Questions.....	96
7.10.4 Avoiding Data Leakage	96
7.11 From Simulator to Digital Twin: Terminology and Required Evidence.....	96
7.11.1 Why the Current System Is Not a Digital Twin	96
7.11.2 Conditions for a Future Digital-Twin Claim.....	97

7.11.3 Recommended Terminology during Intermediate Development	97
7.12 Proposed Staged Development Roadmap.....	97
7.12.1 Stage 0 - Preserve the Current Training Baseline	98
7.12.2 Stage 1 - Architecture Hardening	98
7.12.3 Stage 2 - Real-Telemetry Replay	98
7.12.4 Stage 3 - Independent Physics-Based Subsystem Models	98
7.12.5 Stage 4 - Integrated Closed-Loop Simulator	98
7.12.6 Stage 5 - Correlation, Calibration and Hybrid Experiments	98
7.12.7 Stage 6 - Asset-Linked Digital-Twin Research, If Data Access Exists	98
7.13 Recommended Research Priorities for MCS26	99
7.14 Final Development Perspective	99
8 References	100

0 Introduction

0.1 Spacecraft Operations and Training Context

Spacecraft operations are conducted through the coordinated use of the space segment, ground segment, mission-operations data, operational procedures and trained personnel. The operational task is not limited to observing spacecraft telemetry or transmitting telecommands in isolation; it requires continuous assessment of spacecraft status, interpretation of subsystem behaviour, management of ground-contact opportunities, execution of approved procedures, coordination between console functions and an effective response to off-nominal conditions. ECSS defines mission operations as the planning, execution and evaluation activities associated with the combined space and ground segments, and explicitly treats operations personnel, mission-operations data and the supporting ground systems as integral elements of the mission-operations domain [1].

Training environments for spacecraft operations therefore need to expose users to more than static subsystem descriptions. They should provide a controlled setting in which operators can follow procedures, observe evolving telemetry, recognise the consequences of system state changes, exchange information between console positions and practise decision-making under both nominal and contingency conditions. Within ECSS ground-segment engineering, simulations and rehearsals are recognised as part of the process used to establish the readiness of operations personnel and mission-operations data for an actual mission [1]. MCS26_V26_01 does not perform such operational validation; however, the same underlying training principle - learning through coordinated, time-dependent operational activity - is adopted at an educational level.

This approach is consistent with the way spacecraft-operations training is presented by ESA, where the relationship between spacecraft subsystem design and operational behaviour is treated as a central learning objective and realistic anomaly simulations are used to train participants in team-based operations [5]. The simulator described in this manual applies that principle to a simplified classroom mission-control environment in which the operational sequence, the information available to each role and the consequences of selected actions are made visible to the trainees.

0.2 MCS26_V26_01 System Overview

MCS26_V26_01, designated the Mission Control System Operations Simulator, is a browser-based, multi-operator training application for simplified spacecraft mission operations. The current software baseline provides two implemented mission scenarios: a nominal ground-pass and imaging sequence, and a communications-and-thermal contingency sequence. Both scenarios are executed against a shared simulation state so that actions performed from one operator position are reflected at the other connected positions.

The operational interface is divided among a Spacecraft Operations Manager (SOM), a Spacecraft Controller (SPACON), and two Subsystem Operator/Engineer positions (SOE1 and SOE2). The SOM coordinates the procedure and evaluates completion criteria; the SOE positions inspect the relevant telemetry and report subsystem status; and SPACON performs simulator telecommand execution when requested through the procedure flow. Command execution follows a deliberate SELECT-ARM-GO sequence, while telemetry-reporting steps require the responsible SOE to inspect the requested parameter before the SOM confirms the procedural action.

The simulated engineering domains comprise the ground-station interface and spacecraft subsystems for Electrical Power (EPS), Attitude and Orbit Control (AOCS), Thermal Control (TCS), Payload, and Command and Data Handling (C&DH), together with a dedicated captured-image view. The subsystem models are coupled to common mission state where operationally relevant. Examples include the relationship between payload power and thermal behaviour, the effect of communications-link state on ground visibility of spacecraft telemetry, the use of mass-memory state during data-dump activities, and changes in payload and AOCS behaviour during imaging windows. These couplings are intentionally simplified, but they preserve the operational idea that spacecraft subsystems cannot be treated as independent displays during mission execution.

The software is designed for local multi-computer operation. A host computer provides the browser application and a WebSocket-based synchronisation service, while operator computers connect through role-specific browser sessions on the same reachable network. The shared-state mechanism is used to maintain a common operational picture across consoles; local panel selection remains independent so that each operator can work within the assigned console role.

0.3 Scope and Intended Use

0.3.1 Training Scope

The simulator is intended to support introductory and intermediate training in the logic of spacecraft operations rather than to reproduce the complete ground segment of a specific mission. Its principal training scope includes procedural discipline, telemetry interpretation, role-based coordination, telecommand handling, subsystem cross-checking, ground-contact logic, imaging operations and contingency response. The scenarios are structured so that trainees must obtain information from the appropriate console, compare the observed condition with the procedure criteria, and perform the next authorised action in the required sequence.

The nominal scenario introduces the operational flow of a ground contact from acquisition of signal through telemetry verification, subsystem preparation, payload operations, image acquisition and transition to spacecraft standby. The contingency scenario introduces a controlled degradation in which thermal and communications conditions evolve, spacecraft telemetry becomes unavailable through the primary ground-station path, coordination with a simulated GNC support function is required, a recovery procedure is introduced, and telemetry is subsequently re-established through a secondary ground-station context. The detailed procedures, timing logic, success criteria and operator actions are defined later in this manual.

0.3.2 User Roles and Operational Interaction

MCS26_V26_01 is designed for team operation. It is not intended to be used as a single-user dashboard in which one person performs all console functions simultaneously. The separation of SOM, SOE and SPACON responsibilities is part of the training model because operational coordination is itself a learning objective. A telemetry observation, a procedural decision and a telecommand execution are represented as distinct activities, even when they are driven by the same underlying simulation state.

The simulator consequently places emphasis on sequence and authority. A procedure step assigned to an SOE requires observation and reporting before the SOM confirms the step. A step requiring a spacecraft command is initiated by the SOM and executed by SPACON using the command-control sequence implemented in the application. This separation reduces the tendency to treat a mission-control exercise as a collection of independent buttons and instead reinforces the relationship between information, decision and action.

0.3.3 Functional Boundaries

The current software provides an educational representation of mission-control functions; it does not reproduce a complete Mission Control System, flight dynamics system, spacecraft simulator, ground-station network, mission-planning system or mission database as used in an operational programme. The application does not implement end-to-end spacecraft protocol stacks, certified command verification, operational authentication, real orbital propagation, hardware-in-the-loop interfaces, mission-specific flight software, or a validated digital representation of a flight spacecraft.

Functions presented in the user interface therefore have to be interpreted according to their training purpose. For example, telemetry pages represent realistic categories of spacecraft and ground equipment, but the existence of a displayed parameter does not imply that a particular flight mission uses the same identifier, hardware configuration, numerical range or operational limit. Likewise, the simulated contingency logic demonstrates how interacting technical conditions can drive an operational response; it is not a mission-certified failure detection, isolation and recovery implementation.

0.4 Simulation Fidelity, Data Provenance and Engineering Assumptions

0.4.1 Synthetic Telemetry and Commanding Model

All spacecraft and ground-segment telemetry used by the present simulator is generated within the software. No telemetry stream, monitoring database, telecommand database or operational parameter set from a specific spacecraft is embedded in the baseline described by this manual. Parameter identifiers, subsystem inventories, numerical values, thresholds and status classifications are simulation-development constructs selected to support the training scenarios. Where values vary with mission time or operational state, the variation is produced by simplified deterministic or state-coupled models rather than by a flight-qualified spacecraft dynamics or subsystem model.

The same distinction applies to commands. The command catalogue provides a controlled interaction with simulator state and supports the procedural behaviour required by the scenarios. A successful simulated command indicates that the relevant software action has been accepted within the training logic; it does not represent generation, encoding, radiating, reception or onboard execution of a real telecommand. Telemetry and command history are therefore training records of simulator activity, not mission records.

0.4.2 Representation of Space-to-Ground Visibility and Telemetry Availability

The simulation distinguishes between onboard state and the ability of the ground segment to observe that state. Spacecraft subsystem telemetry is made available to the operator interfaces only when the simulated communications conditions permit a valid telemetry path. Before a valid contact is established, and during the loss-of-signal interval represented in the contingency scenario, spacecraft subsystem displays report the absence of telemetry rather than continuing to expose internal spacecraft values. The onboard simulation state itself may continue to evolve during that interval.

This behaviour is important for training because loss of telemetry is not equivalent to proof that an onboard subsystem has stopped functioning. From the ground operator's perspective, the available evidence has changed: the spacecraft state can no longer be directly confirmed until the telemetry path is recovered. The simulator uses this distinction to support the transition from nominal monitoring to contingency coordination and subsequent re-acquisition.

0.4.3 Standards Context and Non-Qualification

The organisation and terminology of this manual are informed by the European Cooperation for Space Standardization (ECSS) framework. ECSS-E-ST-10C Rev.1 includes a Product User Manual Document Requirements Definition, while ECSS-E-ST-70C addresses ground systems and mission operations and ECSS-E-ST-40C Rev.1 addresses software engineering for space-system software [1]-[3]. These standards are used as references for document discipline, terminology and engineering context; they are not a declaration that MCS26_V26_01 has been developed, verified, validated or accepted under an ECSS contractual lifecycle.

MCS26_V26_01 is not flight-qualified, safety-qualified or operationally validated. It shall not be used to command a spacecraft, to make safety-critical decisions, to establish operational readiness of a flight ground segment, to validate flight procedures, or to derive mission limits. It is not a flight twin or certified digital twin of any spacecraft. Any resemblance between simulator parameters and real engineering quantities reflects the use of common spacecraft-engineering concepts and does not establish traceability to a flight article or mission operations database.

0.5 Development Context, Ownership and Institutional Status

MCS26_V26_01 was independently conceived and developed by the author for educational use in the context of the DLR_School_Lab TU Darmstadt satellite-operations environment. The DLR_School_Lab provides a control-room learning setting in which students can take on mission-controller roles and work through satellite-related tasks; the facility forms part of a broader educational programme in which aerospace and research topics are presented through practical, team-based experiments [4].

The project is a personal software development and remains the intellectual property of the author. It is not an official software product of DLR, TU Darmstadt, ESA, Telespazio or any other institution, and the

use of institutional or operational terminology in this manual does not imply sponsorship, certification, endorsement or transfer of ownership. External organisations and standards are referenced only where they provide relevant technical, educational or mission-operations context.

The simulator and this manual are provided for education, training, demonstration and engineering study. They are intended to support understanding of spacecraft-operations principles and to provide a maintainable platform for classroom exercises. Responsibility for any use outside that scope remains with the user, and no part of the software or documentation shall be interpreted as authority for real spacecraft operations.

1 Motivation for Development

1.1 Value of the Existing Telespazio Training Concept

The development of MCS26_V26_01 was initiated in an environment that already possessed a strong spacecraft-operations training concept. The DLR_School_Lab TU Darmstadt had been equipped with the Spacecraft Operations Training Centre (STC) supplied by Telespazio VEGA Deutschland. Telespazio reported that the STC had been developed in 2009 in collaboration with ESA to make spacecraft operations accessible to students and other newcomers, and that the Darmstadt installation was provided for the opening of the DLR_School_Lab in 2017 [6]. The system was not conceived as a conventional classroom presentation. Its value lay in allowing participants to work from control-room consoles, interact with a simulated spacecraft and experience the relationship between spacecraft behaviour, mission events, operator responsibilities and team coordination.

That training philosophy remains technically sound and is one of the principal reasons the legacy system was valuable. Telespazio described the STC as a simplified but realistic spacecraft-operations simulation in which trajectory calculations, critical manoeuvres and teamwork could be experienced in a control-room setting [6]. The same source notes that the STC has been used by universities and to introduce newcomers and lateral entrants to spacecraft operations before they begin work on operational consoles. This is a significant pedagogical strength: the trainee is not only shown what a subsystem does, but is placed in an operational sequence in which technical information has to be interpreted and communicated to other members of the team.

The broader Telespazio simulator heritage is also important when assessing the legacy system. Telespazio Germany develops operational spacecraft simulators for European missions and describes such simulators as high-fidelity software representations used for flight-control-team training, procedure validation and anomaly investigation [8]. The company has supplied simulation solutions for missions operated from ESOC, and its BepiColombo work provides a representative example: the operational simulator was used by the Flight Control Team to rehearse mission phases, manoeuvres and contingency situations before and during operations [9]. ESA describes the spacecraft simulator in the same operational context as a central tool for ground-segment validation and for training a Flight Control Team in both nominal and contingency operations [10]. The educational STC belongs to a simplified training context, but its underlying approach is therefore connected to a mature and operationally relevant simulation discipline.

For this reason, the motivation for MCS26_V26_01 was never based on the assumption that the Telespazio training concept was inadequate. On the contrary, the existing system demonstrated which aspects of spacecraft-operations training were worth preserving: role separation, procedure-driven activity, time-dependent spacecraft behaviour, exposure to nominal and off-nominal situations, and the requirement for participants to coordinate rather than operate as isolated users. The redevelopment decision concerned the maintainability of the specific legacy installation available in the laboratory, not the technical competence of Telespazio or the validity of simulator-based operations training.

1.2 Telespazio Simulation and Training Software as an Engineering Reference

The strengths of the Telespazio approach are particularly visible in the way the company structures modern training software. Current Telespazio Germany material describes its VEGA Emulation Technology in a fourth-generation implementation, VET4, as a development and runtime infrastructure for training applications. The published characteristics include modular software development, consistent module construction, continuous testing and integration, customisation for different training needs and operation on standard laptop and desktop computers [7]. This is relevant to the present project because it shows that modularity, maintainability, reuse and deployment on standard computing platforms are not alternatives to professional simulation practice; they are themselves recognised requirements of a mature training-software infrastructure.

Telespazio also emphasises the combination of system knowledge with hands-on simulation rather than relying exclusively on static computer-based instruction [7]. This principle is directly applicable to

spacecraft-operations education. A trainee who only reads subsystem descriptions may understand individual functions, but does not necessarily experience the operational effects of changing telemetry, contact constraints, procedure dependencies or the consequences of an incorrectly sequenced action. Simulation provides the dynamic context in which those relationships become observable.

The MCS26_V26_01 project therefore takes the Telespazio training approach as a positive engineering reference while adopting a much smaller scope. It does not attempt to reproduce the high-fidelity operational spacecraft simulators developed for real missions, nor does it claim equivalence with the current VET4 infrastructure. Instead, it preserves selected principles that are appropriate to the School Lab: modular software, operator interaction, scenario-based training, nominal and contingency logic, and execution on ordinary workstation hardware. The difference is primarily one of scale, fidelity, ownership and lifecycle control.

1.3 Limitations of the Local Legacy Installation

The practical problem encountered in the laboratory was that the locally available STC/VEGA installation represented an older software baseline whose execution environment had become difficult to sustain. The observed installation identified itself as STC VEGA 4.0.0 and relied on a legacy browser and Java-based execution environment. During routine use, Java runtime and access-control exceptions occurred repeatedly rather than as isolated events. In addition, the manual function present in the interface did not provide an accessible and operationally useful set of instructions for installation recovery, fault diagnosis or maintenance of the local baseline. The consequence was that a training session could be delayed by software-platform issues before the actual spacecraft-operations exercise had begun.

This distinction between the vendor technology and the installed baseline is essential. Telespazio continues to maintain and evolve its own training infrastructure, as demonstrated by the current VET4 product generation [7]. The laboratory, however, did not have the same engineering position as a customer operating a current, supported and actively maintained deployment. The School Lab had an ageing local installation, limited practical update possibilities and no locally modifiable source baseline from which the recurring compatibility problems could be corrected. The fact that a current generation of vendor technology exists does not remove the maintenance burden of an older installation when migration, support or replacement cannot be performed within the available project constraints.

The network environment further increased this burden. The training room is operated under security constraints and cannot be treated as an unrestricted Internet-connected development environment. In practice, this means that software updates, runtime components, online repair resources or external dependencies cannot be assumed to be available at the moment a fault occurs. For a legacy system with browser- and Java-specific dependencies, such restrictions make recovery more difficult because the required component versions may no longer be part of a standard workstation installation and may not be obtainable through an ad hoc update process during a training session.

The resulting issue was therefore a lifecycle problem rather than a criticism of the original training design. The STC had delivered educational value for many years, but the cost of preserving the particular local baseline was increasing while the ability to change it was decreasing. From a software-engineering perspective, this is precisely the point at which maintainability becomes a design decision. ECSS software engineering treats operations and maintenance as lifecycle activities that must be planned and controlled [3]. For a small educational laboratory, continuing to depend on a difficult-to-modify legacy runtime was no longer the most efficient way to preserve the training capability.

1.4 Rationale for Migration to a Web-Based Architecture

The decision to move to a web-based implementation was made after considering the requirements of the local training environment rather than on the premise that browser technology is inherently superior to dedicated simulator software. A professional spacecraft simulator may require hardware interfaces, deterministic execution, mission-specific protocol stacks, validated spacecraft models and specialised simulation infrastructure. MCS26_V26_01 has a different objective. It is an educational control-room application whose main technical requirements are multi-user access, shared scenario state,

straightforward deployment, maintainable source code, local-network operation and the ability to modify training content without dependence on proprietary runtime components.

For these requirements, a browser-based client is economically and technically appropriate. The operator computers do not require a separate mission-specific desktop installation; they only require a compatible browser and network access to the simulation host. The application logic can be updated centrally on the host project, after which the same revised frontend is served to all operator positions. This reduces configuration drift between consoles and avoids the need to maintain a separate application installation on every SOM, SOE and SPACON computer. It also makes replacement of a failed operator workstation substantially simpler because the client does not contain the authoritative simulation state.

The present implementation deliberately remains local. The host computer runs the frontend service and the WebSocket synchronisation service, and the operator clients communicate with that host over the School Lab network. No cloud service or external Internet connection is required during the exercise. This is a decisive advantage under the laboratory security constraints: Internet access can remain disabled while the complete multi-console simulation continues to operate. External connectivity is required only when the development environment itself is intentionally being installed or updated, not as a dependency of normal exercise execution.

The web architecture also provides a more transparent maintenance path. The source code, dependencies, startup commands, network ports and role-specific access method are all part of the controlled project baseline. A developer can inspect the same code that generates the telemetry, manages the scenario and renders the operator interface. Problems can therefore be traced and corrected within the project rather than being treated as opaque failures in an external runtime. The project can be version-controlled, formatted, built, tested and redeployed using standard development tools, which substantially lowers the maintenance burden for a small laboratory team.

1.5 Preservation of Operational Training Value During the Migration

A change in software architecture was acceptable only if the operational training value of the existing concept was preserved. The replacement was therefore not designed as a set of independent subsystem pages. The simulation remains procedure-driven and multi-role. The Spacecraft Operations Manager coordinates the sequence and evaluates procedural criteria; the Subsystem Operator/Engineer positions inspect and report the required telemetry; and the Spacecraft Controller performs simulator telecommands after the corresponding request is issued. The separation between observation, decision and command execution is intentional because the communication between roles is part of the exercise.

The shared simulation state is central to this design. A telecommand executed from the SPACON console must produce consequences that are subsequently visible in the telemetry observed by the SOE positions and in the procedure status coordinated by the SOM. Likewise, a loss of the space-to-ground telemetry path must be reflected consistently across the spacecraft subsystem panels rather than independently at each workstation. The WebSocket-based synchronisation mechanism was selected to preserve that common operational picture while allowing each operator to navigate locally within the assigned console.

The current scenarios also retain the nominal-versus-contingency training distinction that is fundamental to professional operations simulation. ESA describes simulation campaigns in which flight-control teams rehearse both planned operations and injected failures in a realistic environment [10]. Telespazio similarly describes simulation officers preparing nominal and contingency scenarios adapted to a specific mission [6][9]. MCS26_V26_01 applies the same training logic at reduced fidelity. Scenario 1 establishes the nominal ground-pass and imaging workflow, whereas Scenario 2 requires the team to recognise degraded conditions, manage loss of telemetry, coordinate a recovery action and continue the mission through a secondary ground-station context.

The migration therefore retains the most valuable element of the legacy STC: the exercise is still centred on coordinated spacecraft operations rather than on the software interface itself. What changes is the technical mechanism used to deliver the exercise. The legacy installation provided a proven training

concept but had become difficult to maintain locally; the web-based implementation provides a maintainable execution platform while preserving the role-based and scenario-based training philosophy.

1.6 Engineering Trade-Off and Development Decision

Continuing with the existing STC installation would have had the advantage of retaining a mature training system produced by an organisation with extensive spacecraft-simulation experience. It would also have avoided the engineering effort required to recreate selected training functions. If the laboratory had possessed a current supported baseline, a practical upgrade path, maintained documentation and the ability to adapt the installation to current workstations, continued use or migration within the Telespazio ecosystem would have been a technically credible option.

Under the actual local constraints, however, that option was less favourable. The project had to operate with the hardware and network already available in the School Lab, without assuming continuous Internet access or access to the vendor source code. The recurring Java and browser compatibility issues had to be considered together with the effort required to diagnose an older system for which usable local maintenance guidance was unavailable. In this situation, each attempt to preserve the existing installation addressed symptoms while leaving the underlying lifecycle dependency unchanged.

Developing a new web-based simulator required greater initial effort, but shifted that effort from repeated recovery of an opaque legacy environment to a source-controlled platform that could be understood and extended locally. This trade-off was considered appropriate for the School Lab because the required fidelity is educational rather than flight-operational. The project does not need to emulate the complete spacecraft computer, reproduce mission-specific communication protocols or qualify flight procedures. It needs to present technically credible subsystem behaviour, enforce a meaningful operational sequence and remain available for repeated training sessions.

The resulting development decision should therefore be interpreted as a context-specific engineering trade rather than a comparison in which MCS26_V26_01 is claimed to be superior to Telespazio software. Telespazio provides mature, modular and high-fidelity simulation solutions for professional and training applications [7]-[9]. MCS26_V26_01 addresses a narrower problem: a maintainable, locally owned and locally deployable spacecraft-operations training simulator for a restricted-network educational laboratory. Within that scope, the web architecture offers a lower long-term maintenance burden, faster recovery from workstation changes, clearer control of the software baseline and a practical path for incremental development.

This rationale also establishes a requirement for future development. Improvements to MCS26_V26_01 should increase the technical quality of the simulation without sacrificing the maintainability that motivated the redevelopment. More rigorous subsystem models, externally validated engineering data, additional contingency cases and stronger software verification can be introduced progressively, but the system should remain understandable, version-controlled, deployable on standard laboratory hardware and operable without dependence on external online services during a training session.

2 Required Tools and Prerequisites

2.1 System Configuration Baseline

MCS26_V26_01 is operated as a small distributed training system rather than as a standalone desktop application. The baseline configuration consists of one host workstation running the simulation services and a set of browser-based operator clients connected to the host over a local network. The host provides both the frontend development server and the shared-state synchronisation service. The operator workstations do not execute an independent spacecraft simulation; they render role-specific views of the common state and send authorised user actions back to the host-side synchronisation service.

The recommended classroom topology uses five computers: one dedicated Host / Simulation Server, one SOM workstation, one SPACON workstation, and two SOE workstations. This arrangement most closely preserves the role separation on which the training concept is based. For software development or diagnostic testing, several role sessions can technically be opened on one computer, but such co-location is not considered the preferred training configuration because it reduces the physical and procedural separation between console functions.

The two SOE positions are operational assignments rather than different software role types. Both SOE1 and SOE2 use the same application role identifier, SOE, and therefore receive the same class of interface. Their distinction during an exercise is created by the procedure and by the allocation of subsystem responsibilities. SOM and SPACON, by contrast, are distinct application roles selected through their dedicated role identifiers.

2.2 Host Workstation Prerequisites

2.2.1 Operating System and Command Environment

The supplied operational helper scripts are Windows batch files and define Windows as the reference host environment for this manual. The scripts rely on the Windows command processor, standard commands such as `ipconfig`, and the ability to launch additional command windows. A current 64-bit Windows 10 or Windows 11 installation is therefore recommended for the Host / Simulation Server. The application itself is based on portable web technologies and Node.js, but use of another host operating system requires manual replacement of the supplied `.bat` workflow and is outside the baseline installation procedure documented here.

The user account on the host shall have permission to execute locally installed command-line applications, create and modify files inside the project directory, and install the software dependencies required by the Node.js project. Administrative rights are not required for normal execution once the environment has been prepared, but local administrator or equivalent IT support may be required when installing Node.js or Git, or when creating inbound firewall rules for the network services used by the simulator.

2.2.2 Node.js Runtime and npm

Node.js is a mandatory host-side prerequisite. It is used directly to execute the synchronisation server and indirectly to execute the Vite frontend server and the project toolchain. The current project declares Vite 8.0.10. Vite 8 requires Node.js 20.19 or later within the supported 20.x line, or Node.js 22.12 or later in subsequent supported release lines [12]. Because software support status changes over time, the installation should use a currently maintained Long-Term Support release rather than an end-of-life Node.js branch. At the time of preparation of this manual, Node.js 24 is the current LTS line published by the Node.js project [11]. Node.js 24 LTS is therefore the recommended host baseline for a new installation.

The npm package manager is also required on the host. A standard Node.js installation provides npm, which is used to resolve and install the project dependencies defined in `package.json`. The repository also contains a `package-lock.json` file. npm uses a compatible lock file to reproduce the dependency tree selected for the project, which reduces variation between installations and is important for a training

system that should behave consistently across workstations and re-installations [15]. The exact installation procedure is given in Section 3; at the prerequisite stage it is sufficient to ensure that both node and npm commands are available from the host command line.

2.2.3 Git and Repository Access

Git is required when the project is obtained from, or subsequently updated through, the authoritative GitHub repository. A maintained Git for Windows installation is recommended for the reference host environment [14]. Git is not a runtime dependency of the simulation after the complete project has been copied to the host; however, retaining Git on the host is strongly recommended because it provides an unambiguous method for retrieving updates, identifying the installed revision, comparing local modifications, and returning to a known software baseline.

Repository access and simulator execution shall be treated as separate activities. Internet access is required when the host is expected to clone or pull the repository directly from GitHub and is normally required when npm downloads dependencies from the public package registry. Continuous Internet access is not required during a training session. If the target laboratory host is intentionally isolated, the repository and dependency installation shall be prepared in advance on an approved connected system or through an approved internal package source before the host is placed in the restricted environment.

2.2.4 Project Directory Integrity

The complete repository shall be preserved as a project, not reduced to the visible frontend files. The runtime depends on the relationship between the project root, package.json, server.js, the Vite configuration, the src directory, public assets, TypeScript configuration files and the scripts directory. In particular, the supplied start-host-dev.bat script changes its working directory to the parent of the scripts directory before starting the services. Moving that script to an arbitrary location or copying only individual source files breaks this relative-path assumption and can prevent the standard start procedure from working.

The public media assets are also part of the operational baseline. The current application loads local audio and video resources for the introductory and end-of-simulation sequences. These resources are served by the frontend and are not retrieved from external streaming services. A complete repository copy is therefore required even when the primary training interest is limited to telemetry and command execution. Opening an index file directly from the filesystem is not an equivalent substitute for running the Vite service because the application is designed to be delivered as a module-based web application and to connect to the synchronisation service associated with the host from which the page was loaded.

2.2.5 Port Availability on the Host

Two host-side network ports are part of the current configuration baseline: TCP port 5173 for the Vite frontend service and TCP port 3001 for the Socket.IO/WebSocket synchronisation service. Port 3001 is explicitly fixed in both the server and the client connection logic. Port 5173 is the Vite default development-server port and is also hard-coded into the supplied operator URL scripts and operating instructions.

The availability of port 5173 shall be checked before the training session. Vite normally attempts the next available port if its default port is already occupied [16]. Such automatic fallback is convenient during general web development but is undesirable for the current simulator baseline because the operator helper scripts continue to direct clients to port 5173. A host on which Vite has silently moved to port 5174 or another value can therefore appear operational locally while remote role URLs fail. Port 3001 must likewise be free; the synchronisation server listens explicitly on that port and cannot start normally if another process already owns it.

2.2.6 Host Hardware Considerations

No formally verified minimum CPU, memory or storage specification has been established for MCS26_V26_01, and this manual therefore does not assign unsupported numerical hardware requirements. The host shall be a contemporary desktop or laptop capable of running a supported Node.js release, maintaining the two local server processes, and providing a stable wired or wireless network interface. Sufficient storage shall be available for the repository, the installed node_modules

dependency tree and the local media assets. The training software does not require a dedicated GPU, spacecraft interface hardware, database server or specialised real-time computing platform.

2.3 Operator Workstation Prerequisites

2.3.1 Browser Requirement

Each operator workstation requires a modern web browser with native support for current JavaScript modules, CSS features and WebSocket communication. The normal MCS26_V26_01 launch procedure uses the Vite development server. Vite assumes a modern browser during development and does not attempt to transform the application for legacy browser engines [13]. Current stable releases of Microsoft Edge, Google Chrome or Mozilla Firefox are therefore recommended for the laboratory consoles. Internet Explorer and other legacy browser environments shall not be used.

For a production build, the Vite 8 default compatibility target corresponds to Chrome 111, Edge 111, Firefox 114 and Safari 16.4 or later [13]. These values describe the build tool's baseline rather than a dedicated qualification campaign for MCS26_V26_01. For classroom operation, the practical rule is to keep the selected browser on a maintained release and to use the same browser family across consoles where possible. This reduces differences in media autoplay policy, font rendering, caching behaviour and developer-tool diagnostics.

2.3.2 Client-Side Software and Role Selection

Node.js, npm, Git and the project source tree are not required on a normal operator workstation. The operator client receives the application from the host through the browser. This is a deliberate architectural property of the web-based implementation and substantially reduces configuration effort on the SOM, SPACON and SOE stations. A client workstation needs only a compatible browser, network access to the host, and the correct role URL.

Role selection is encoded in the URL query parameter. The recognised values are SOM, SPACON and SOE. If the role parameter is absent or does not match one of these values, the current application defaults to the SOE role. For that reason, operator URLs should not be entered approximately or reconstructed from memory during an exercise. The supplied role scripts or the exact URL formats documented in Section 3 should be used. SOE1 and SOE2 both connect with role=SOE; their console responsibilities are separated operationally by the procedure, not by different client software.

2.3.3 Display and Media Support

The user interface is designed for desktop-class operation and contains dense telemetry tables, procedure tables and multi-column console layouts. A desktop or laptop display is therefore recommended; mobile-phone operation is not part of the baseline training configuration. No formal minimum display resolution has been qualified, so a specific numerical resolution is not stated as a mandatory requirement. The browser should be used at a zoom level that keeps the procedure and telemetry tables readable without altering the intended role workflow.

The application also contains local MP3 audio and MP4 video sequences. Standard browser audio and H.264/MP4 playback capability is sufficient; no external media player or browser plug-in is required. Modern browsers can restrict unrequested audio autoplay, and the application therefore unlocks audio following user interaction with the scenario interface. Speakers or headphones are recommended when audible cues are to be included in the exercise, but media playback is not a substitute for the procedure status and telemetry indications presented on the console.

2.4 Network Prerequisites

2.4.1 Local Network Topology and Addressability

All participating computers shall be connected to the same reachable local network or to routed local networks that permit direct communication with the host. The standard laboratory configuration uses the host's IPv4 address and does not depend on external DNS. Wired Ethernet is preferred where a fixed control-room installation is available because it reduces variation in link quality; Wi-Fi is also

supported and is sufficient when the local wireless network provides stable connectivity between all consoles.

The host IPv4 address shall remain valid for the duration of an exercise. A dynamically assigned address is acceptable provided that it is checked before the session and communicated to all operators. For a permanent laboratory installation, a DHCP reservation or another controlled addressing method is recommended so that the operator URLs remain predictable. Changing the host address during an active session requires the clients to reconnect using the new address and should therefore be avoided.

2.4.2 Service Interfaces and Connection Logic

The frontend is started with the Vite `--host 0.0.0.0` option, which causes the development server to listen on the host network interfaces rather than only on localhost. Vite documents 0.0.0.0 as the configuration used to make the development server reachable from LAN addresses [16]. The synchronisation server is independently bound to 0.0.0.0 on port 3001. These two listeners are the only network services required by the baseline simulator.

When an operator opens the frontend using the host address, the application takes the hostname from the browser location and constructs the synchronisation endpoint on the same hostname at port 3001. Consequently, there is no separate WebSocket host field for the operator to configure. A client that successfully loads the frontend from the correct host but cannot reach port 3001 can display the application while remaining unsynchronised with the shared mission state. Both services shall therefore be considered mandatory parts of a valid connection.

2.4.3 Firewall and Local Security Policy

The host firewall shall permit inbound communication to the services required by the simulator from the intended training network. On Windows, inbound traffic is blocked unless it is solicited or matches an applicable allow rule, and Microsoft documents inbound port or program rules as the normal mechanism for permitting a listening service to receive network traffic [17]. In practice, the host must permit the frontend on TCP 5173 and the synchronisation service on TCP 3001 for the network profile used by the laboratory.

Firewall access should be limited to the required local network scope wherever the local IT policy allows this. Disabling the firewall globally is neither required nor recommended. The operator workstations normally initiate outbound connections to the host and therefore do not require inbound listening rules for MCS26_V26_01.

2.4.4 Trust Boundary and Exposure Limitations

The current synchronisation service is intended for a trusted classroom LAN. It accepts WebSocket connections without application-layer user authentication, allows cross-origin access in its present server configuration, and uses unencrypted HTTP/WebSocket transport within the local environment. These choices simplify a closed training installation but do not provide the controls expected for an Internet-exposed mission system. Ports 5173 and 3001 shall therefore not be forwarded through an Internet router or deliberately exposed to an untrusted external network.

This restriction is an operational prerequisite, not merely a cybersecurity recommendation. The shared-state server accepts state updates from connected simulator clients and broadcasts the resulting state to all participants. The integrity of a training run therefore assumes that only authorised exercise workstations can reach the service. If the simulator is later deployed beyond a controlled LAN, authentication, transport encryption, origin restrictions and a defined access-control architecture should be introduced before such deployment is considered acceptable.

2.4.5 Internet Connectivity

Internet connectivity is not part of the runtime path between the host and operator consoles. After the repository and its dependencies have been installed, the frontend, synchronisation service, telemetry generation, scenario logic, audio and video assets are served locally. This allows the training exercise to continue on an isolated LAN. Internet access is needed only for maintenance activities that depend

on external repositories or package registries, such as cloning or updating the GitHub repository and downloading npm packages.

For a restricted laboratory, the required software should therefore be staged before the exercise day. The host should not be placed into an isolated network before the project has been cloned, the dependencies have been installed successfully, and the complete local startup sequence has been verified. This separation between preparation and execution prevents an otherwise functional training session from depending on last-minute access to an external package registry or source-control service.

2.5 Software Dependency Baseline

The project dependency manifest defines the software components used by the current baseline. The version expressions in Table 2-1 are taken from package.json. They should be interpreted as the declared dependency ranges of the project; the package-lock.json file is the controlling record for the concrete dependency tree installed by npm when the lock file is compatible with the manifest [15]. Operators do not normally interact with these packages directly, but the table is relevant for reproducibility, troubleshooting and future maintenance.

Table 2-1 - Declared software dependency baseline

Component	Declared version	Role in the simulator
Vue	^3.5.32	Reactive browser user interface and application state binding
Express	^5.2.1	Host-side HTTP server framework used by the synchronisation service
Socket.IO	^4.8.3	Host-side real-time state synchronisation over WebSocket transport
Socket.IO Client	^4.8.3	Browser-side connection to the shared-state service
Vite	^8.0.10	Frontend development server and production build tool
@vitejs/plugin-vue	^6.0.6	Vue single-file-component integration for Vite
TypeScript	~6.0.2	Static typing and source-language toolchain
vue-tsc	^3.2.7	Vue/TypeScript type checking used by the build command
@types/node	^24.12.2	Node.js type definitions used during development/build
@vue/tsconfig	^0.9.1	Base TypeScript configuration for Vue projects
Prettier	^3.8.3	Source formatting; maintenance tool, not a runtime dependency

2.6 Components Not Required by the Current Baseline

Several technologies commonly associated with simulation or mission-control environments are not prerequisites for the present software. No Java Runtime Environment, Java browser plug-in or Internet Explorer component is required. No relational database, message broker, container runtime, cloud account or external authentication service is used by the current baseline. No Python interpreter is required for execution; Python-based subsystem models are discussed only as a possible future extension of the simulator.

Likewise, no physical ground-station equipment, RF modem, antenna controller, spacecraft interface unit or hardware-in-the-loop device is needed. Ground-station and spacecraft engineering quantities are generated by the simulation software. This distinction is important when preparing the laboratory: the prerequisite set is intentionally limited to conventional computing and networking infrastructure, even though the user interface presents data that represent a much wider spacecraft and ground-segment equipment inventory.

2.7 Pre-Installation Readiness Criteria

Before the installation procedure in Section 3 is started, the configuration should satisfy the readiness criteria in Table 2-2. These checks are intended to prevent installation problems from being confused with simulator defects during the first operational run.

Table 2-2 - Pre-installation readiness criteria

Item	Acceptance criterion
Host operating environment	Windows host available; user can run Command Prompt or PowerShell and modify the project directory.
Node.js	A supported Node.js release is installed; Node.js 24 LTS is recommended for a new installation.
npm	npm is available from the same command environment as Node.js.
Git	Git is installed when the repository will be cloned or updated directly from GitHub.
Repository access	GitHub access is available during preparation, or an approved complete repository copy is available offline.
Dependency access	npm packages can be obtained during preparation, or an approved offline/internal package source has been prepared.
Host ports	TCP 5173 and TCP 3001 are not occupied by unrelated processes.
Host network	The host has a usable IPv4 address on the training LAN/Wi-Fi and that address will remain stable during the session.
Firewall authority	The host user or local IT administrator can permit inbound access to TCP 5173 and 3001 on the intended network profile.
Operator clients	Each client has a current supported browser and can reach the host over the local network.
Security boundary	The simulation network is trusted/local; the host services are not intentionally exposed to the public Internet.
Project completeness	The complete repository structure, including scripts, src, public assets and root configuration files, is available.

Once these criteria are met, the system is ready for software acquisition, dependency installation, host startup and client connection. Those activities are intentionally separated from the present prerequisite definition and are described step by step in Section 3.

3 Installation, Configuration and Initial System Bring-Up

3.1 Installation and Deployment Sequence

This section defines the reference procedure for obtaining MCS26_V26_01 from its GitHub repository, preparing the Node.js project on the Host / Simulation Server, starting the two required host services, connecting the operator workstations, and verifying that the distributed simulation is ready for an exercise. The procedure assumes that the prerequisites and network conditions defined in Section 2 have already been satisfied. The sequence should be followed in the order given because several later checks depend on the successful completion of earlier steps.

Installation is required only on the host computer for the baseline classroom configuration. Normal SOM, SPACON, SOE1 and SOE2 workstations receive the application from the host through a web browser and therefore do not require a local Node.js project, npm dependency tree or Git working copy. The host and operator functions are deliberately separated: the host provides the software services and common state, whereas the operator workstations provide the human-machine interfaces used during the exercise.

Table 3-1 - Reference installation and bring-up sequence

Step	Activity	Acceptance condition
1	Obtain or update the repository on the host	A clean MCS26_V26_01 working tree is available.
2	Verify Node.js, npm and Git	Required commands execute and report supported versions.
3	Install project dependencies	Dependency installation completes without an npm error.
4	Perform a build check	The project build completes successfully.
5	Start the WebSocket and frontend services	Port 3001 and port 5173 are listening on the host.
6	Determine the correct host IPv4 address	The address belongs to the network used by the operator consoles.
7	Verify firewall and TCP reachability	Operator workstations can reach TCP 5173 and TCP 3001.
8	Open the four operator roles	SOM, SPACON, SOE1 and SOE2 interfaces load from the same host.
9	Select the same scenario on all clients	All participating clients are in the same scenario context.
10	Start the simulation from SOM	Mission status and time advance consistently across connected clients.

3.2 Obtaining the Software from GitHub

3.2.1 First-Time Repository Clone

For a new host installation, the recommended method is to clone the repository with Git rather than downloading individual source files. A Git clone creates a complete local working tree and preserves the repository metadata required for later updates and revision identification [18]. The authoritative project repository for this manual is:

```
https://github.com/alirezairanpoor/MCS26_V26_01
```

Open Windows PowerShell, Windows Terminal, or Git Bash on the host. Select a parent directory in which the project may be retained permanently. The path should be writable by the user account that will maintain the simulator and should not be a temporary download directory.

Step 1: Verify that Git is available before attempting the clone.

```
git --version
```

A valid response contains the installed Git version. If the command is not recognised, complete the Git installation described in Section 2 before proceeding.

Step 2: Change to the parent directory in which the project is to be stored. The following example is illustrative; use the actual laboratory path.

```
cd "D:\MCS"
```

Step 3: Clone the repository.

```
git clone https://github.com/alirezairanpoor/MCS26_V26_01.git
```

Unless an alternative directory name is supplied to Git, the command creates a new directory named MCS26_V26_01 and checks out the repository's default branch [18]. Do not copy the contents out of this directory after cloning; the relative relationship between the project root and the scripts directory is part of the supplied startup workflow.

Step 4: Enter the newly created project directory.

```
cd MCS26_V26_01
```

Step 5: Confirm that the working tree is recognised as a Git repository and identify the checked-out branch and revision.

```
git status
git branch
git log -1 --oneline
```

For the baseline described here, the operational working branch is expected to be main. The last command records the short commit identifier and commit subject of the installed revision. That identifier should be retained when a formally controlled training baseline is prepared, because it provides an unambiguous link between the manual, the software used for the exercise and any later defect report.

3.2.2 Updating an Existing Repository

A host that already contains a Git clone should normally be updated rather than cloned again. Before any update, stop the simulator services and inspect the local working tree. Updating an active installation while the frontend and synchronisation server are running is not a controlled operational procedure.

Step 1: Open a terminal in the existing MCS26_V26_01 project root.

Step 2: Check the current branch and local modification state.

```
git status
git branch --show-current
```

If the branch is main and the working tree is clean, retrieve and integrate the current remote main branch with a fast-forward-only update:

```
git pull --ff-only origin main
```

The --ff-only form is recommended for an operational training installation because it updates the local branch only when the remote history can be applied without creating a new merge commit. Git documents this mode as refusing the pull when local and remote histories have diverged [19]. A refusal is therefore a useful protection: it indicates that the local working copy is no longer a straightforward replica of the repository and should be reviewed before it is used as the exercise baseline.

Step 3: After the update, record the resulting revision and check that the working tree is clean.

```
git log -1 --oneline
git status
```

Step 4: Re-run dependency installation after a repository update, particularly when package.json or package-lock.json has changed.

```
npm install
```

A source-code update should not be considered complete until the dependency installation and build verification in Sections 3.3 and 3.4 have succeeded.

3.2.3 Existing Local Modifications

Do not execute a routine pull over an installation that contains unexplained local modifications. The command `git status` shall first be used to identify the changed files. Local development work that is intended to be retained should be committed to an appropriate development branch or otherwise preserved under the project's source-control policy before the operational baseline is updated. Uncommitted changes shall not be discarded merely to force an update unless their loss has been explicitly accepted.

For classroom operation, the preferred condition is a clean working tree on a known commit. This makes it possible to distinguish a software defect in the repository baseline from a defect introduced by an unrecorded local change. Where experimental changes are being developed, the operational training copy and the development copy should be kept as separate working trees or branches.

3.3 Installing the Project Dependencies

3.3.1 Runtime Verification

Dependency installation shall be performed from the project root, i.e. the directory containing `package.json`, `package-lock.json` and `server.js`. Before invoking `npm`, verify the runtime environment:

```
node --version
npm --version
```

The Node.js version shall satisfy the Vite 8 requirement stated in Section 2. A command-line session opened before Node.js was installed may not yet contain the updated `PATH` environment; if `node` or `npm` is not recognised immediately after installation, close the terminal and open a new one before diagnosing the condition as a software fault.

3.3.2 Standard Dependency Installation

For the normal first-time installation and for routine repository updates, install the declared dependencies with `npm` from the project root:

```
npm install
```

`npm` reads `package.json` and the existing `package-lock.json` to construct the local `node_modules` tree. The repository declares the runtime packages used by the current implementation, including `Vue`, `Express`, `Socket.IO` and `Socket.IO Client`, together with the `TypeScript` and `Vite` development toolchain. Dependency installation requires access to the configured `npm` package registry unless the required packages are already available through an approved cache or internal registry [15].

A successful installation returns control to the shell without an `npm` error and leaves the project with a populated `node_modules` directory. Informational notices concerning package funding or newer package-manager versions do not by themselves indicate installation failure. Warnings concerning unsupported engines, unresolved peer dependencies, failed lifecycle scripts, inaccessible registries or permission errors shall be investigated before the host is accepted for training.

3.3.3 Clean Dependency Reinstallation

When dependency corruption is suspected, after a major environment change, or when a repeatable clean installation is required, the repository's lock file permits the use of `npm ci`:

```
npm ci
```

`npm ci` is designed for a clean installation from an existing lock file. It removes an existing `node_modules` directory before installation, refuses to proceed if `package.json` and the lock file are inconsistent, and does not rewrite the package manifest or lock file [20]. It is therefore useful for restoring a controlled host dependency tree. It should not be used as a method for adding or updating individual packages.

3.4 Pre-Operational Build Verification

Before the simulator is introduced into the control room for a new installation or a materially changed software revision, perform a TypeScript and production-build check. This step is separate from starting the training services. It verifies that the checked-out source and installed dependencies can be processed by the toolchain defined in the project.

```
npm run build
```

The project build command runs the Vue/TypeScript type-checking build and then the Vite production build. The command shall complete without compilation or bundling errors. A failed build is an installation or software-baseline problem and should be resolved before a training session is started, even if the development server appears able to serve part of the application.

The production output generated by this check is not required for the baseline classroom start procedure, which uses the Vite frontend service defined by `npm run start:frontend`. The build check is retained because it provides a stronger pre-operational integrity test than merely opening the initial browser page.

3.5 Starting the Host / Simulation Server

3.5.1 Standard Startup Using the Supplied Script

The reference startup method is the supplied Windows batch script. It shall be executed on the Host / Simulation Server from the repository's scripts directory. The script may be started by double-clicking it in File Explorer or from a terminal:

```
.\scripts\start-host-dev.bat
```

The script first changes the working directory from scripts to the project root. It then starts the synchronisation server in a new command window using `npm run start:server` and starts the Vite frontend service in the original command window using `npm run start:frontend`. Both command windows are part of the running simulator and shall remain open for the complete exercise.

3.5.2 Expected Host Processes

A correct startup produces two logically independent services. The synchronisation service is implemented by `server.js` and listens on TCP port 3001 on all host interfaces. It maintains the shared simulator state in memory, sends the current state to a newly connected client, receives state updates from clients and broadcasts the resulting shared state to all connected clients. The frontend service is provided by Vite and serves the browser application on the host network.

Table 3-2 - Host services required for an operational session

Service	Start command	Required port	Operational function
WebSocket synchronisation server	<code>npm run start:server</code>	TCP 3001	Common mission-state exchange between clients
Vite frontend server	<code>npm run start:frontend</code>	TCP 5173	Delivery of the browser application to operator consoles

The WebSocket server console should report that the server is running on port 3001. The Vite console should report the local and network addresses of the frontend. The standard role URLs and helper scripts assume that the frontend is available on port 5173.

3.5.3 Port 5173 Verification

Vite's development server normally uses port 5173. If that port is already occupied, Vite can select the next available port unless strict port behaviour is configured [16]. The current MCS26_V26_01 role URLs and helper scripts are fixed to 5173; therefore an automatic Vite fallback to 5174 or another port is not an acceptable baseline condition.

Required check: Read the Vite startup output. If it reports any frontend port other than 5173, stop the frontend, identify the process occupying port 5173, free the port, and restart the MCS26 host services. Do not distribute modified role URLs as a normal workaround, because doing so creates a configuration different from the documented baseline.

On Windows PowerShell, the following diagnostic command can be used to identify a listener on the expected frontend port:

```
Get-NetTCPConnection -LocalPort 5173 -State Listen
```

If a listener exists before MCS26 is started, its owning process should be identified and assessed before termination. Do not terminate an unknown process on a shared institutional computer without confirming that it is safe to do so.

3.5.4 Port 3001 Verification

Port 3001 is not selected dynamically. The synchronisation server is explicitly configured to listen on port 3001. If another application already owns the port, the Node.js server cannot establish the required listener and the distributed simulation will not operate correctly. A frontend page may still open in this condition, which can make the installation appear partially functional even though no common mission state is available.

```
Get-NetTCPConnection -LocalPort 3001 -State Listen
```

After MCS26 has started, one listening entry should be associated with the synchronisation service. If no listener is present, inspect the WebSocket server command window for a Node.js startup error before proceeding to the operator workstations.

3.6 Determining the Host IPv4 Address

3.6.1 Using the Supplied Host-IP Script

Operator clients address the host by IPv4 address. The supplied helper script displays the IPv4 addresses currently assigned to the host:

```
.\scripts\show-host-ip.bat
```

The script executes ipconfig and filters the output for IPv4 entries. It can therefore display more than one address when the host has multiple active or virtual network interfaces. The first address shown shall not be accepted automatically.

3.6.2 Selecting the Correct Address

Select the IPv4 address assigned to the physical or approved wireless adapter that connects the host to the same network used by the operator computers. Typical laboratory addresses are private IPv4 values, for example 192.168.x.x or 10.x.x.x, but the address range alone is not sufficient to identify the correct interface. VPN adapters, Hyper-V or other virtual switches, WSL adapters and disconnected network interfaces can also have IPv4 addresses.

When more than one address is displayed, run the full ipconfig command and identify the adapter whose connection name and network correspond to the control-room LAN:

```
ipconfig
```

Record the selected address before opening the operator clients. The same host address shall be used for all four role sessions during a given exercise.

3.7 Firewall Configuration and Network Verification

3.7.1 Windows Firewall Permission

On first execution, Windows may request permission for Node.js or the frontend service to accept incoming network connections. Access should be permitted only for the network profile authorised for the laboratory. The required communication is inbound to the host on TCP 5173 and TCP 3001. A

global firewall disable is not required and should not be used. Microsoft recommends scoped allow rules rather than disabling the firewall as a troubleshooting shortcut [17].

Where firewall configuration is centrally managed, the required rules should be created by the responsible administrator before the exercise. The simulator should remain confined to the trusted training LAN as specified in Section 2.4.4.

3.7.2 TCP Reachability Test from an Operator Workstation

Before opening all role consoles, verify from at least one operator workstation that both host services are reachable. Windows PowerShell provides Test-NetConnection for TCP diagnostics [21]. Replace HOST-IP with the IPv4 address selected in Section 3.6.

```
Test-NetConnection HOST-IP -Port 5173
Test-NetConnection HOST-IP -Port 3001
```

For each command, TcpTestSucceeded should be True. The two results test different parts of the system: success on 5173 confirms that the browser frontend is reachable; success on 3001 confirms that the synchronisation endpoint is reachable. Both shall succeed before the system is accepted for a multi-console run.

A failed test shall be resolved as a host-service, routing or firewall problem before browser troubleshooting begins. This separation is important because a browser can provide ambiguous symptoms when a page loads from port 5173 but the WebSocket endpoint on port 3001 is blocked.

3.8 Opening the Operator Clients

3.8.1 Direct Role URLs

The most direct deployment method is to enter the role URL into a modern browser on each operator workstation. Replace HOST-IP with the host IPv4 address. All role sessions shall point to the same host.

Table 3-3 - Operator role URLs

Operator station	Application role	URL
SOM	SOM	http://HOST-IP:5173/?role=SOM
SPACON	SPACON	http://HOST-IP:5173/?role=SPACON
SOE1	SOE	http://HOST-IP:5173/?role=SOE
SOE2	SOE	http://HOST-IP:5173/?role=SOE

For example, if the host IPv4 address is 192.168.178.129, the SOM URL is:

```
http://192.168.178.129:5173/?role=SOM
```

SOE1 and SOE2 intentionally use the same technical URL role, SOE. Their different duties are defined by the operational procedure and console assignment, not by separate application role codes. The application recognises only SOM, SPACON and SOE as role query values.

Important: If the role query parameter is absent or contains an unrecognised value, the current application defaults to SOE. A bare URL such as http://HOST-IP:5173/ shall therefore not be used for the SOM or SPACON station.

3.8.2 Optional Role-Launcher Scripts

The repository also provides Windows helper scripts for the operator roles:

```
scripts\open-som.bat
scripts\open-spacon.bat
scripts\open-soe1.bat
scripts\open-soe2.bat
```

Each role launcher prompts for the Host IP address and opens the corresponding URL in the default browser. These scripts do not run a local simulation service. They are convenience launchers only.

Accordingly, they may be copied to an operator workstation if desired, while the complete Node.js project remains installed only on the host.

When a launcher requests the Host IP address, enter only the IPv4 address, for example 192.168.178.129. Do not include http://, the port number, or the role query string; the batch script constructs those elements itself.

3.9 Initialising a Multi-Console Session

3.9.1 Connection Order

For a clean exercise, start both host services before opening the operator clients. Once ports 5173 and 3001 have been verified, open the SOM, SPACON, SOE1 and SOE2 browser sessions. The host computer does not need to act as an operator console unless the exercise configuration explicitly requires it.

When a client establishes its WebSocket connection, the synchronisation server immediately sends the shared state currently held in server memory. Subsequent state updates received from a client are merged into that server-side state and broadcast to all connected clients. This is the mechanism that keeps mission time, procedure state, telecommand history, contingency status and other shared operational variables coherent across the classroom.

3.9.2 Scenario Selection on Every Client

Scenario selection is a local client choice in the present software baseline. The selected scenario identifier is not part of the state object broadcast by the synchronisation server. Accordingly, each participating browser shall select the same scenario before the run begins. It is not sufficient for only the SOM workstation to choose the scenario.

For Scenario 1, select Nominal Ground Pass and Imaging on SOM, SPACON, SOE1 and SOE2. For Scenario 2, select Communications & Thermal Contingency on all four workstations. The introductory media sequence is also local to each browser and may complete at slightly different times. Operational start shall wait until every participating console has entered the selected scenario.

Configuration rule: Do not mix Scenario 1 and Scenario 2 clients on the same shared-state server. The shared mission variables are broadcast between clients, while scenario-dependent interpretation is local. Mixed scenario selection can therefore produce a technically connected but logically inconsistent training session.

3.9.3 Local Panel Navigation

Panel navigation is local to each browser. Operators may therefore open different subsystem panels without forcing the other workstations to follow the same view. This behaviour is intentional and supports console specialisation. A SOE may inspect EPS or TCS telemetry while the SOM remains on the procedure panel and SPACON remains on the command interface.

3.9.4 Starting the Simulation

Only the SOM role is authorised by the user interface to start the mission timer. In the procedure table, Step 1 contains the Start Simulation action. Once all clients are connected and have selected the same scenario, the SOM presses Start Simulation.

The start action changes the shared simulation status to RUNNING, starts the one-second mission timer on the SOM client and immediately broadcasts the resulting mission state. During the run, the active timer updates mission time and the coupled simulation transitions and publishes the updated shared state once per second. Connected clients therefore receive the evolving mission state from the common session.

Operational constraint: SPACON and SOE clients shall not attempt to initiate the exercise. Their function begins after the SOM has started the simulation and the first procedure conditions become active.

3.10 Initial Bring-Up Acceptance Test

The following checks shall be completed before the exercise proceeds beyond the initial procedure phase. They form a practical acceptance test of the installed software, network path and shared session.

Table 3-4 - Initial multi-console acceptance checks

Check	SOM	SPACON / SOE	Acceptance criterion
Frontend	Application loaded	Application loaded	All consoles are served from the same HOST-IP on port 5173.
Role	SOM controls enabled	Correct SPACON or SOE role loaded	No console is operating under an unintended default SOE role.
Scenario	Selected scenario visible	Same scenario selected locally	All participating clients use the identical scenario.
Simulation status	RUNNING after Start Simulation	RUNNING	Status becomes consistent across connected clients.
Mission time	T+00:xx advances	T+00:xx advances	Displayed mission time converges to the shared session and continues updating.
Procedure state	Step 1 active/completing according to AOS logic	Shared effects visible where applicable	No console remains on stale pre-start mission state.
Network	Host services remain open	No loss of frontend or state updates	Both host command windows remain active and TCP connections remain available.

If the frontend is visible on every workstation but simulation status or mission time fails to propagate, treat the condition as a synchronisation-path failure rather than as a successful bring-up. The most likely boundary to test first is TCP 3001. Detailed fault-isolation procedures are provided in Section 6.

3.11 Controlled Reset and Restart Between Exercises

3.11.1 Why a Host Restart Provides the Cleanest Baseline

The WebSocket synchronisation server stores its shared state in process memory. That state persists for as long as the server process remains running and is supplied to clients that connect later. For this reason, closing only the browser windows does not guarantee that the next class or test will begin from an empty shared-state baseline.

The most reliable reset between independent exercises is therefore a clean restart of both host services followed by a browser hard refresh on the operator consoles. Restarting the synchronisation server clears its in-memory shared-state object, while restarting the frontend ensures that the current source is being served.

3.11.2 Clean Restart Procedure

Step 1: End the current exercise and ensure that no operator is still issuing commands.

Step 2: In the WebSocket server command window, press Ctrl+C and confirm termination if the shell requests it.

Step 3: In the frontend/Vite command window, press Ctrl+C and confirm termination if required.

Step 4: Start the host again.

```
.\scripts\start-host-dev.bat
```

Step 5: On every operator workstation, perform a hard browser refresh using Ctrl+F5, or close and reopen the role URL.

Step 6: Select the required scenario again on SOM, SPACON, SOE1 and SOE2 before the SOM starts the new run.

This procedure is preferred when the objective is a reproducible fresh training session. It clears the server's retained state and avoids carrying a previous procedure step, telecommand history, emergency condition or mission timer into the next exercise.

3.12 Installation Completion Criteria

Installation is considered complete only when the host has a known repository revision, project dependencies are installed, the build check succeeds, both runtime services start on their documented ports, the selected host IPv4 address is reachable from the operator network, all four operator roles can load the application, all clients can establish the shared-state path, the same scenario has been selected on each client, and the SOM can start a session whose mission status and time are observed consistently across the connected consoles.

After these criteria have been met, routine training operation may proceed without external Internet connectivity. The subsequent sections describe the spacecraft-system concepts represented by the simulator and the detailed operational procedures for the implemented scenarios.

4 Spacecraft System Engineering Fundamentals and MCS26 Simulation Model

The simulator can be operated correctly only when the displayed telemetry is interpreted in the context of the physical spacecraft functions that it represents. This section therefore establishes the engineering model used throughout the remainder of the manual. Each topic is treated in two layers. First, the underlying spacecraft or ground-segment function is described according to standard spacecraft-engineering practice. Second, the corresponding representation in MCS26_V26_01 is identified. This distinction is essential because the simulator deliberately preserves realistic functional dependencies while using simplified numerical models and scenario-driven timing.

4.1 System-Level View of a Spacecraft

4.1.1 Spacecraft as an Integrated System

A spacecraft is not a collection of independent subsystem displays. It is an integrated flight system in which energy, information, thermal loads, mechanical forces and control authority are exchanged continuously across subsystem interfaces. Common spacecraft classifications include structure and mechanisms, electrical power, thermal control, attitude and orbit control or guidance-navigation-control, command and data handling, telecommunications, propulsion and mission payloads [22, 29]. The exact decomposition is mission dependent: a subsystem may be implemented as a dedicated unit on one spacecraft and as a distributed function on another. What matters operationally is the function and its interfaces, not the label used on an organisation chart.

The spacecraft bus provides the supporting functions required to keep the vehicle alive, controlled and communicative. The payload is the mission-specific equipment that produces the principal mission product, for example optical imagery, scientific measurements or telecommunications service. This bus-payload distinction is useful but not absolute. A payload may contain its own processor, power converter, thermal control hardware, memory and timing electronics; those elements remain dependent on bus resources such as electrical power, attitude knowledge, spacecraft time and the communications path.

From an operations perspective, the important question is therefore not merely whether a subsystem is 'nominal', but whether it can support the current mission activity. An imaging payload can be electrically healthy while being unusable because the spacecraft is not pointing at the target; an AOCS can be controlling attitude correctly while the payload is thermally outside its acquisition range; and an onboard computer can continue executing normally while the ground has no telemetry visibility. Mission operations require these conditions to be interpreted together.

Functional interaction of spacecraft subsystems represented in MCS26

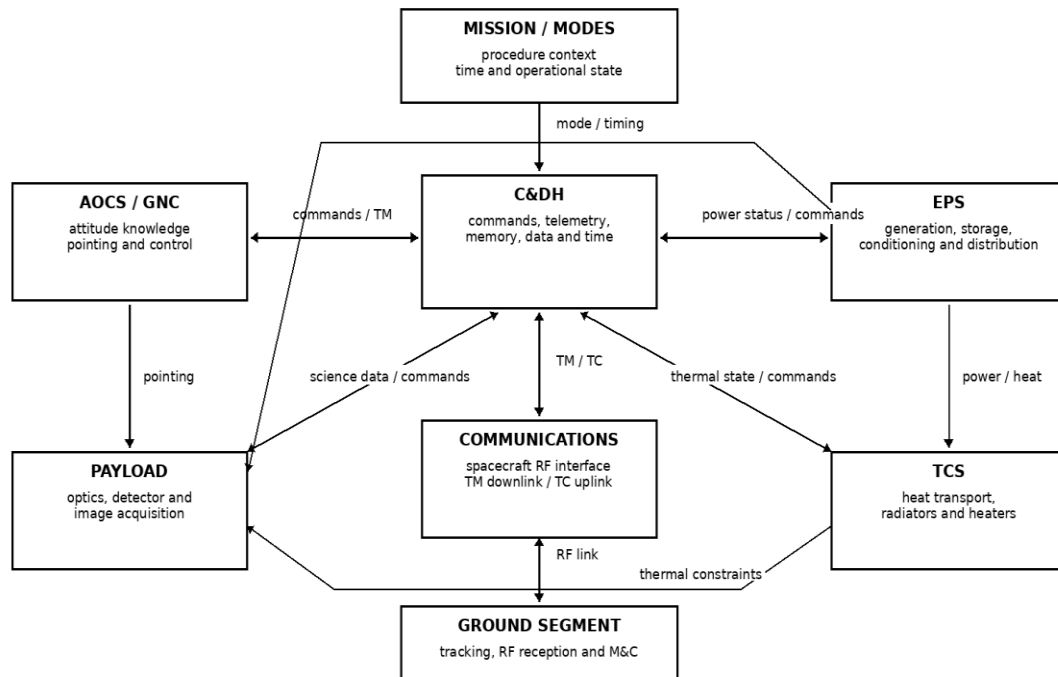


Figure 4-1 - Functional interaction of the spacecraft subsystems represented in MCS26_V26_01.

Table 4-1 - Typical spacecraft functions and their representation in MCS26_V26_01

Engineering domain	Primary physical function	MCS26 representation
Electrical Power System (EPS)	Generate, condition, store, distribute and protect electrical power.	Dedicated EPS telemetry; solar-array, bus, battery, PDU, LCL and power-margin parameters.
Thermal Control System (TCS)	Maintain equipment and structure within allowable temperature ranges by controlling heat flow.	Dedicated TCS telemetry; equipment temperatures, radiators, heat pipes, heaters, thermal margins and trends.
AOCS / GNC	Estimate and control attitude; support navigation, pointing and selected orbit-control functions.	Dedicated AOCS telemetry; attitude, rates, sensors, reaction wheels, magnetorquers, navigation and guidance/control states.
Command & Data Handling (C&DH)	Execute onboard software, manage commands, telemetry, data buses, time and memory.	Dedicated C&DH telemetry; OBC, CPU/FPGA, memory, buses, command handling, telemetry services, file system and dump state.
Telecommunications	Provide RF uplink/downlink between spacecraft and ground and support tracking observables.	Represented mainly through Ground Station telemetry and link-state logic rather than a separate spacecraft COMMS panel.
Payload	Perform the mission-specific observation or service and create mission data.	Optical-imaging payload model with camera, detector, optics, calibration, compression, storage-interface and image-acquisition telemetry.
Ground segment	Track the spacecraft, establish RF links, receive telemetry, transmit commands and support operations.	GS1 and GS2 contexts with antenna geometry, receiver/baseband telemetry, signal quality, Doppler and link acquisition.
Structure / mechanisms / propulsion	Provide load-bearing structure, deployable or movable mechanisms, and translational/attitude impulse where required.	Referenced by selected telemetry fields but not implemented as independent operational panels in the current baseline.

4.1.2 Functional Chains and Interface Dependencies

Five functional chains are particularly important for understanding the simulator. The energy chain runs from power generation and storage through conditioning and distribution to every electrical load. The thermal chain converts part of that electrical energy into heat and transports or rejects the heat through the spacecraft. The information chain moves commands, telemetry, time and payload data through the onboard computer and communications system. The attitude chain converts sensor measurements and guidance targets into actuator commands and vehicle pointing. Finally, the mission-product chain begins with payload configuration and observation, passes through onboard processing and storage, and ends with delivery of the data to the ground.

These chains are coupled. Increasing payload electrical power changes EPS loading and dissipated heat; the resulting temperatures affect thermal margins and may force a reduction in payload power. Payload imaging requires AOCS pointing and rate stability, adequate electrical power, acceptable thermal conditions, sufficient mass-memory capacity and valid timing. Downlinking stored data requires the C&DH data path and a usable RF contact. The MCS26 scenario logic is intentionally built around several of these dependencies so that a trainee cannot correctly complete a procedure by considering only one panel.

4.2 Mission-Operations View of Spacecraft State

4.2.1 Engineering Telemetry, Mission Data and Reporting Data

Telemetry is the information transferred from the spacecraft to the ground for monitoring and mission use. A useful operational distinction is between engineering data and mission or science data. Engineering telemetry describes the health, status and configuration of the spacecraft, for example temperatures, voltages, currents, mode identifiers, computer states and mechanism positions. Mission data is the product for which the spacecraft is operated, such as an image acquired by an optical instrument [38]. ECSS monitoring-and-control terminology uses the broader concept of reporting data for data used to assess the functioning of the space system [33].

A telemetry parameter normally has more meaning than a numeric value alone. Operations databases associate a parameter with an identifier, engineering unit, conversion rule, validity conditions, limits, status interpretation and often the subsystem or equipment to which it belongs [33]. MCS26 adopts the same conceptual structure at training level. Each displayed row contains a parameter code, subsystem description, measurement, unit and status class. The codes are simulator identifiers and are not copied from an operational mission database.

4.2.2 Telecommands and Onboard Actions

Telecommands are requests sent from the ground to cause a spacecraft function to execute or to change state. In flight systems, the end-to-end path can include command generation, validation, packetisation, security or authentication controls, uplink encoding, RF transmission, onboard reception, decoding, acceptance checks, execution and subsequent verification through telemetry. ECSS-E-ST-70-41C defines standardised packet-utilisation services for remote monitoring and control, including the transport of telecommand requests and telemetry reports [32].

MCS26 represents only the operational decision and simulator-state portion of this chain. The SPACON SELECT-ARM-GO sequence is a training control barrier; it is not a physical representation of CCSDS coding, an RF uplink or flight-software command acceptance. When a simulator command succeeds, the relevant application state changes and the resulting telemetry model responds. The training value lies in enforcing authority, sequencing and post-command verification rather than reproducing the complete space-link protocol stack.

4.2.3 Onboard State versus Ground Observability

A fundamental mission-operations concept is the difference between the true onboard state and the state currently observable from the ground. Loss of telemetry does not imply that the spacecraft has stopped executing. Onboard control loops, thermal processes, timers, software and stored-data handling can continue while the spacecraft is outside ground-station coverage or while an RF link is

unavailable. The ground team must then reason from the last valid telemetry, predicted evolution, autonomous spacecraft behaviour and the next opportunity for contact.

MCS26 models this distinction explicitly. Spacecraft-subsystem telemetry is replaced by NO TELEMETRY when the simulated spacecraft-to-ground telemetry path is unavailable, while the underlying mission state continues to evolve. Ground-station housekeeping is conceptually different: local station equipment can remain observable even when the spacecraft RF link is lost. This distinction is central to Scenario 2 and should be maintained when interpreting all later procedures.

4.3 Ground-Station Contact Geometry and RF Acquisition

4.3.1 Local Topocentric Reference Frame

A ground antenna tracks a spacecraft in a local topocentric frame referenced to the station. The local vertical points approximately away from the Earth through the station, while the local horizontal plane is tangent to the reference Earth at that location. A line of sight from the station to the spacecraft can then be described by two look angles: azimuth in the horizontal plane and elevation above the local horizon. Coordinate-system definitions and transformations are a controlled engineering subject because navigation, pointing and operations can be invalidated by mixing reference frames [26].

4.3.2 Azimuth

Azimuth specifies the horizontal direction in which the ground antenna points. In the convention used throughout this manual, azimuth is measured clockwise from local north: 0 degrees is north, 90 degrees east, 180 degrees south and 270 degrees west. A pass therefore has an azimuth history as the apparent spacecraft direction moves across the local sky. The exact AOS and LOS azimuths depend on the orbit, ground-station location and pass geometry.

4.3.3 Elevation

Elevation is the angle between the station-to-spacecraft line of sight and the local horizontal plane. A spacecraft on the ideal geometric horizon has an elevation of 0 degrees; a spacecraft directly overhead at the zenith has an elevation of 90 degrees. Elevation is an instantaneous geometric quantity. It should not be confused with orbital altitude, which is the spacecraft's height above a reference surface, or with antenna boresight error, which is the angular difference between commanded and actual antenna pointing.

Elevation strongly influences operations. At low elevation, the slant range is longer, the signal traverses a longer atmospheric path, local terrain or structures are more likely to obstruct the line of sight, and multipath or radio-frequency interference can be more significant. As elevation increases, slant range normally decreases until closest approach and the RF link generally becomes more favourable, subject to antenna patterns, spacecraft attitude and other link-budget terms.

4.3.4 Elevation Mask

An elevation mask is an operational minimum elevation used to decide whether a spacecraft should be considered usable by a station or processing function. It is not the same quantity as elevation. Elevation is measured from geometry; the mask is a configured acceptance boundary. ESA Flight Dynamics software represents a station mask as a minimum elevation that can vary with azimuth, allowing terrain or local obstructions to be represented explicitly [28]. Real ground stations may therefore have a constant minimum elevation or an azimuth-dependent horizon mask.

A mask above the geometric horizon creates a deliberate guard band. The station may be able to detect energy below the mask while operations still reject the pass as unsuitable for commanding, telemetry reception or precision tracking. A NASA ground-station example for the HESSI mission required clear sky visibility to approximately 5 degrees elevation and used a station mask to account for nearby structures and terrain [41]. The numerical mask is mission- and site-specific; 5 degrees is not a universal flight rule.

Engineering note. In MCS26_V26_01, 5.0 degrees is the procedural operational elevation mask used for both GS1 elevation confirmation and GS2 telemetry recovery. It is a simulator acceptance threshold, not a generic ECSS requirement and not a value that should be transferred to a real mission.

4.3.5 Acquisition of Signal and Loss of Signal

Acquisition of Signal (AOS) and Loss of Signal (LOS) are operational contact events. At the most basic geometric level, AOS occurs when a spacecraft rises into station visibility and LOS occurs when it leaves that visibility. In an operational system, however, the useful contact boundary is normally determined by more than geometric horizon crossing. Station masks, antenna slew limits, prediction uncertainty, RF acquisition thresholds, frequency uncertainty, required link margin and mission rules can shift the operational start and end of a pass.

It is therefore useful to distinguish geometric AOS from operational acquisition. Geometric visibility means that an unobstructed line of sight can exist. Operational acquisition means that the antenna is tracking sufficiently well and the receiver has progressed through the required acquisition chain to provide the service needed by operations. Similarly, a station can remain geometrically visible while the usable link has already been lost because received power, carrier lock or frame synchronisation is no longer adequate.

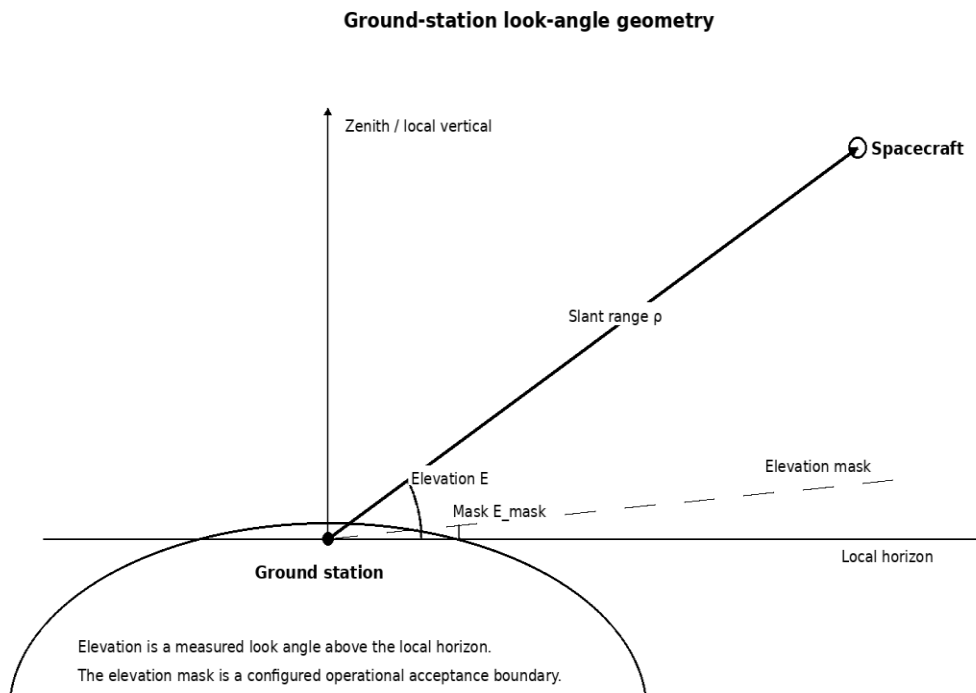


Figure 4-2 - Ground-station look-angle geometry showing elevation, elevation mask and slant range.

4.3.6 Slant Range

Slant range is the straight-line distance between the ground station and spacecraft. For a spherical Earth approximation with Earth radius R_E , spacecraft altitude h and ground-station elevation E , the range can be written as [30]:

$$\rho(E) = -R_E \sin(E) + \sqrt{[(R_E + h)^2 - (R_E \cos(E))^2]} \quad (4-1)$$

At zenith, the slant range approaches the spacecraft altitude h . Near the horizon, the range is much larger because the line of sight is nearly tangent to the Earth. Slant range enters the RF link budget through free-space path loss and also determines one-way propagation delay. For a fixed wavelength λ , ideal free-space loss increases with the square of range:

$$L_{\text{FS}} = (4\pi\rho / \lambda)^2 \quad (4-2)$$

$$L_{FS,dB} = 20 \log_{10}(4\pi r / \lambda) \quad (4-3)$$

The free-space expression is not a complete ground-link model. Atmospheric attenuation, polarisation loss, pointing loss, feeder losses, antenna gain, receiver noise temperature, implementation loss and coding performance must also be considered in a real link budget [27, 29].

4.3.7 Doppler Shift

Relative radial motion between the spacecraft and ground station changes the observed carrier frequency. For radial velocity v_r much smaller than the speed of light c , the first-order Doppler relation is:

$$\Delta f / f_0 \approx -v_r / c \quad (4-4)$$

With the usual convention for received-frequency shift, an approaching low-Earth-orbit spacecraft is received above the nominal carrier frequency and a receding spacecraft below it; the Doppler shift passes through zero and changes sign as the radial component of relative velocity reverses near closest approach. Receiver frequency acquisition and tracking must therefore accommodate a predictable Doppler envelope. Doppler is also a navigation observable in many mission architectures [27].

4.3.8 Link-Quality Quantities

Several related quantities are used to describe whether an RF link can support telemetry. Received signal strength is a direct or derived indication of carrier power at the receiver input. Carrier-to-noise-density ratio C/N_0 expresses carrier power relative to noise spectral density and is measured in dB-Hz. Energy-per-bit to noise-density ratio E_b/N_0 normalises the available signal energy to the bit rate. For an uncoded conceptual relation with bit rate R_b :

$$C/N_0 = E_b/N_0 + 10 \log_{10}(R_b) \quad (4-5)$$

Bit Error Rate (BER) is the probability or measured fraction of erroneous bits; Frame Error Rate (FER) expresses errors at the framed-data level. Their relationship to E_b/N_0 depends on modulation, coding, receiver implementation and channel conditions. Link margin is the difference between the available performance and the minimum performance required for the selected service. A positive margin does not by itself guarantee error-free data, but it indicates that the link budget exceeds the design threshold.

Table 4-2 - Ground-link quantities used in the simulator

Quantity	Physical interpretation	Representative MCS26 parameter
Elevation	Line-of-sight angle above local horizon.	GEL005
Azimuth	Horizontal antenna look direction.	GAZ230
Slant range	Station-to-spacecraft line-of-sight distance.	GRN420
Doppler offset	Received carrier-frequency displacement caused by radial motion.	GDS740
Received signal strength	Receiver input signal level.	GRS121
C/N_0	Carrier-to-noise-density ratio.	GCN124
E_b/N_0	Energy-per-bit to noise-density ratio.	GSN612
BER / FER	Bit-level and frame-level error indicators.	GBE125 / GFE126
Link margin	Available margin relative to a simulator receiver threshold.	GLM127
Carrier lock	Receiver carrier-tracking state.	GCL001
Frame synchronisation	Recognition of the telemetry framing structure.	GFR128
Signal quality	Training-level aggregate signal assessment.	GSE001 / GBL092

4.3.9 Receiver Acquisition Chain

The transition from 'spacecraft above the horizon' to 'valid telemetry available' is a chain of engineering functions. The antenna must first point with sufficient accuracy. The RF front end and low-noise amplifier must provide usable signal power without excessive noise or distortion. The receiver must acquire the

carrier and track frequency and phase as required. The demodulator and bit synchroniser must recover the symbol or bit stream. Channel decoding may correct errors. The frame synchroniser then identifies telemetry transfer frames or equivalent framing structures before higher-level packets and parameters can be extracted. NASA's spacecraft-telecommunications overview describes the distinction between carrier lock and data framing, while ECSS/CCSDS standards define the formal telemetry and telecommand data-link structures used in flight systems [27, 32].

MCS26 exposes several of these stages as separate training parameters rather than collapsing them into a single ON/OFF link flag. This allows a trainee to distinguish, for example, antenna geometry from carrier lock and frame validity. The displayed BER, FER, C/N0, Eb/N0 and signal-quality values are synthetic and are intended to be internally coherent with the simulator link state, not to reproduce a flight link budget.

4.3.10 MCS26 Ground-Station Geometry Model

The MCS26 ground-pass geometry is a scenario reference model, not an orbit propagator. A spherical Earth radius of 6378 km and a simulated orbital altitude of 500 km are used for the slant-range calculation. GS1 uses a nominal maximum displayed elevation of 38 degrees and a pass-direction interpolation from approximately 230 degrees azimuth toward 70 degrees. GS2 uses a higher maximum elevation of 85 degrees and a nominal azimuth progression from approximately 188 degrees toward 42 degrees. The azimuth histories are smoothed functions of a normalised pass phase.

For GS1, the scenario defines initial AOS at T+01:00. Before that time, no GS1 look-angle data is presented and spacecraft telemetry is not available. After AOS, a sine-shaped reference elevation profile drives the displayed geometry. The procedure does not accept the contact for progression until the reported elevation is at least 5 degrees. The ground-station model classifies elevation below 3 degrees as bad, 3 to below 5 degrees as warning, and elevations at or above 5 degrees as acceptable for the operational sequence.

The simulator deliberately separates several visibility concepts. The presence of a pass establishes the scenario RF context, while receiver carrier-lock logic uses the 5-degree threshold and the procedure requires explicit SOM confirmation of that same mask. Telemetry-history generation is likewise gated by a valid GS1 or GS2 lock. These behaviours should be understood as training abstractions that teach the progression from geometric opportunity to operationally accepted link.

In Scenario 2, GS1 geometry can be driven into a controlled degradation profile when the contingency sequence reaches the link-degradation phase. The link is then declared lost and a GS2 acquisition context is initiated. GS2 spacecraft-telemetry lock is explicitly established only when GS1 is no longer connected and GS2 elevation reaches at least 5 degrees. Until that condition is met, spacecraft subsystem panels remain without telemetry. The GS2 elevation profile includes a deliberately slowed descending segment below 30 degrees to provide sufficient training time; this is a scenario-design feature and shall not be interpreted as orbital dynamics.

Table 4-3 - MCS26 ground-geometry constants and operational thresholds

Item	GS1	GS2	Interpretation
Simulated orbit altitude	500 km	500 km	Used with spherical Earth for slant-range reference.
Earth radius	6378 km	6378 km	Spherical reference used by simulator geometry.
Nominal maximum elevation	38 deg	85 deg	Scenario display profile, not propagated ephemeris.
AOS azimuth reference	230 deg	188 deg	Start direction of the synthetic pass.
LOS azimuth reference	70 deg	42 deg	End direction of the synthetic pass.
Doppler envelope	45 kHz	48 kHz	Bounded S-band-style simulator envelope.
Operational elevation mask	5 deg	5 deg	Procedure acceptance / lock threshold.
Signal classification	<3 bad; 3-<5 warning; >=5 usable after filtering	Same logic	Training-level quality logic.

4.4 Electrical Power System (EPS)

4.4.1 EPS Functions and Architecture

The Electrical Power System supplies electrical energy to every active spacecraft function. Its main engineering tasks are generation, conversion and regulation, energy storage, distribution, switching and protection. Solar-electric spacecraft typically generate power in photovoltaic arrays, regulate or condition that power onto one or more buses, charge batteries when generation exceeds load, and draw from batteries when generation is insufficient. Power distribution units and protective devices then connect individual loads to the bus [22][23][40].

A spacecraft power bus is characterised by voltage, allowable transients and ripple, source capability and protection behaviour. ECSS-E-ST-20-20C contains reference requirements for regulated and unregulated spacecraft power buses and for load protection using current-limiting devices [23]. The 28 V class used in MCS26 is therefore recognisable as a common spacecraft engineering concept; the specific simulator values remain training data rather than an adopted flight power specification.

4.4.2 Power, Current and Energy Balance

Instantaneous electrical power is the product of bus voltage and current:

$$P = V I \quad (4-6)$$

A power system must satisfy both instantaneous power demand and energy demand over time. A simple system-level balance can be expressed as generated power minus load power, with the difference charging or discharging storage after conversion losses:

$$P_{\text{storage}} \text{ approximately } P_{\text{generation}} - P_{\text{load}} - P_{\text{loss}} \quad (4-7)$$

A positive power margin indicates that generation exceeds the active load at that instant; a negative margin indicates that storage or load shedding is required. In real spacecraft analysis, power balance must be evaluated over sunlight and eclipse periods, mission modes, degradation, pointing constraints and worst-case combinations of loads. A single 'net power' value is therefore an operational indicator, not a complete energy budget.

4.4.3 Solar Arrays and Power Conditioning

A photovoltaic assembly consists of interconnected solar cells organised into strings and panels, with associated wiring, bypass/blocking diodes, sensors and interfaces [40]. The electrical output depends on solar irradiance, cell temperature, radiation degradation, incidence angle and electrical operating point. Maximum Power Point Tracking can be used to operate an array near the voltage-current condition that maximises available power. Power-conditioning electronics then convert or regulate the generated power for the spacecraft bus and battery-charging functions.

Distribution hardware isolates faults and controls load connection. Latching Current Limiters (LCLs) are one example used in European spacecraft power-distribution architectures; they limit overcurrent and trip a load when the defined condition persists [23]. MCS26 includes LCL, switch, relay, fuse and circuit-breaker style telemetry to expose this engineering hierarchy, but no detailed electrical network or protection transient is solved.

4.4.4 Batteries, State of Charge and State of Health

Rechargeable batteries provide energy when solar generation is unavailable or temporarily insufficient and can also support transient peak loads. State of Charge (SOC) estimates the currently available stored charge relative to a defined full-charge state. State of Health (SOH) expresses longer-term degradation relative to a reference battery capability. SOC can change significantly during a mission phase; SOH normally changes on a much longer timescale and is influenced by age, cycling, depth of discharge, temperature and cell history [39].

Battery operation is constrained by cell voltage, current, temperature and balancing requirements. Series-connected cells can diverge in voltage and SOC, so battery management systems monitor individual or grouped cells and may perform balancing. A real spacecraft does not redistribute arbitrary state-of-charge percentages between independent batteries without an implemented electrical path; any such operation must be supported by dedicated hardware, control logic and verified limits.

Engineering note. Scenario 2 uses a deliberately simplified battery equalisation model in which Battery A transfers simulated charge toward Batteries B and C until all three converge near the same SOC. This is a training state-transition model, not a circuit-level representation of a flight battery-transfer architecture.

4.4.5 EPS Interfaces with Other Subsystems

EPS is coupled to nearly every subsystem. Payload activation increases load. AOCS reaction wheels, actuator electronics and sensors draw power according to mode. Transmitters can be significant peak loads. Heaters create a direct EPS-to-TCS coupling because electrical energy is intentionally converted to heat. Conversely, battery and power-electronics temperatures influence allowable charge/discharge current and conversion efficiency. C&DH controls switches and records EPS telemetry, while survival or safe modes typically shed non-essential loads to preserve the energy required for command, thermal survival and attitude control.

4.4.6 EPS Representation in MCS26

The MCS26 EPS model exposes power generation, storage, conditioning, distribution and system-level telemetry. Solar-array voltage and current are related to a generated-power estimate. The main bus is represented near 28 V, while payload bus current is derived from payload power and bus voltage. Battery SOC values are coupled to Scenario 1 mission time and to the explicit contingency battery states in Scenario 2. Power-saving mode reduces non-essential loading, and payload-power transitions affect the EPS and thermal models.

The parameter NET118 is an operational power-margin indicator used by the scenario logic. It shall not be interpreted as a complete physical difference between a high-fidelity solar-array model and all spacecraft loads. Its values are intentionally shaped so that the procedures have clear nominal and non-nominal conditions. Likewise, the simulator's battery voltage-from-SOC relation, fixed SOH values, current distribution and temperature relationships are engineering approximations selected for internal consistency.

Table 4-4 - EPS functions and representative MCS26 telemetry

EPS function	Representative parameters	Operational interpretation
Power generation	SAW030, EPS717, SAW032, MPP101/102	Array electrical state and generation capability.
Energy storage	BCH096/097/098, BAT201-239, BAT221-223	Battery SOC, voltage/current, cell spread and SOH.
Power conditioning	PCU447-449, DCC208/209, REG512, BDR513	Conversion/regulation state and thermal loading.
Distribution	BUS281/282, EPS718, PWR740-742, PDU331	Main bus and payload electrical demand.
Protection / switching	LCL902/903, EPS901, PDU912, RPC913, SSW914	Load connection, current limiting and load shedding.
System assessment	EPT014, NET118, EPS700, EPS750-753	Thermal condition, operational margin and EPS mode.

4.5 Thermal Control System (TCS)

4.5.1 Spacecraft Thermal Environment

Spacecraft thermal control maintains hardware within allowable temperature limits and, where necessary, within tighter stability ranges required for performance. In vacuum, external heat exchange is dominated by radiation, while heat is transported within the spacecraft mainly by conduction and by engineered thermal paths such as heat pipes. There is no atmospheric convection around an orbiting spacecraft. Solar radiation, planetary albedo, planetary infrared emission, eclipse and internally dissipated electrical power all contribute to the thermal balance [24, 29].

At component level, a simplified lumped thermal balance can be written as:

$$C \, dT/dt = Q_{\text{abs}} + Q_{\text{diss}} - Q_{\text{rejected}} \quad (4-8)$$

where C is effective thermal capacitance, Q_{abs} is absorbed environmental heat, Q_{diss} is internal dissipation and Q_{rejected} includes conduction to other nodes and radiation to space. Radiative heat rejection from a surface is strongly temperature dependent and is commonly represented by the Stefan-Boltzmann relationship:

$$Q_{\text{rad}} = \epsilon \sigma A (T^4 - T_{\text{env}}^4) \quad (4-9)$$

A real thermal model contains many coupled nodes, contact conductances, surface properties, view factors and time-dependent boundary conditions. MCS26 does not solve such a network; it preserves selected cause-and-effect relationships needed for operations training.

4.5.2 Passive and Active Thermal Control

Passive thermal control uses material properties and geometry without continuous commanded actuation. Typical elements include multilayer insulation, surface coatings, optical solar reflectors, conductive straps, heat pipes and radiators. Active control uses electrical or mechanical devices such as heaters, thermostats, louvers, pumps or cryocoolers when the mission requires active temperature regulation [24]. NASA spacecraft examples similarly distinguish passive insulation and radiative design from commandable or thermostatically controlled heaters [22].

Radiators reject internally generated heat by providing a surface with suitable infrared emissivity and a favourable view to cold space. Heaters perform the opposite function: they deliberately add electrical dissipation to prevent equipment from becoming too cold. Heat pipes and thermal straps redistribute heat from local hot equipment to larger radiating areas. Thermal control is therefore tightly linked to EPS: cooling hardware may consume power, heaters always consume power, and almost every powered unit becomes a thermal load.

4.5.3 Thermal Limits, Margins and Trends

Operations normally distinguish survival limits, operational limits and performance-sensitive ranges. A component can remain electrically functional while exceeding a preferred temperature range, yet

continued operation can reduce reliability or violate instrument calibration. Thermal margin is the difference between the predicted or measured temperature and the applicable limit. Because temperature changes have time constants, trend is often as important as the instantaneous value: a value still inside limits but rising rapidly can require action before a violation occurs.

4.5.4 TCS Representation in MCS26

The MCS26 TCS model is coupled to EPS temperature, payload power, battery-transfer state, power-saving mode, payload configuration and spacecraft standby. It derives temperatures for EPS electronics, batteries, onboard computer, communications electronics, AOCS equipment, payload electronics, detector, optics and focal plane. It also represents structural-panel temperatures, radiators, heat pipes, heaters, thermostats, sensor-network status, thermal balance, thermal margin and a short-horizon thermal estimate.

The model intentionally changes thermal telemetry when operational state changes. Raising payload power increases payload and power-electronics thermal loading. An active cooling/mitigation state reduces the scenario EPS temperature and lowers selected payload temperatures. Power-saving mode changes dissipated power and heater behaviour. Battery equalisation increases battery/BMS thermal loading modestly. These relationships make the TCS panel consistent with the mission sequence without claiming thermal-analysis fidelity.

Engineering note. The telemetry inventory contains optional fluid-loop, cryocooler, propulsion-temperature and passive-hardware entries so that trainees can recognise common thermal equipment. An entry marked not configured or otherwise inactive does not mean that such hardware is part of the baseline simulated spacecraft.

Table 4-5 - TCS functions and representative MCS26 telemetry

Thermal function	Representative parameters	Purpose
Controller / health	TCS001-005	TCS mode, controller state, load and overall health.
Equipment temperatures	TMP101-144	Thermal state of EPS, batteries, OBC, COMMS, AOCS and payload.
Structure	STR201-208	Temperature gradients across panels and avionics structure.
Radiators / heat transport	RAD301-304, HP310-323	Heat-rejection and passive transport behaviour.
Heaters / thermostats	HTR401-452, THM460-462	Active thermal-control demand and command state.
Thermal balance	THB601-609	Dissipation, external input, rejection, gradients and trends.
Prediction	EST701-705	Training-level short-horizon temperature and margin estimates.

4.6 Attitude and Orbit Control System / Guidance, Navigation and Control

4.6.1 Purpose of AOCS

The Attitude and Orbit Control System is the functional chain that determines and controls spacecraft orientation and, where included in the mission architecture, supports orbit estimation and orbit-control functions. ECSS identifies attitude estimation, guidance, attitude control, orbit control and optional navigation as classical AOCS functions [25]. The terminology GNC is often used when guidance and navigation are emphasised. For this manual, AOCS and GNC refer to the same broad control domain unless a specific function is being distinguished.

Attitude control is required because spacecraft functions have directional constraints. Solar arrays may need Sun illumination; antennas may need Earth pointing; radiators may require particular environmental views; and an imaging payload must place its line of sight on the target while meeting rate and stability requirements. NASA's onboard-systems guidance explicitly notes that spacecraft

attitude supports antenna pointing, experiment pointing, thermal management and manoeuvre execution [22].

4.6.2 Reference Frames

Attitude has meaning only relative to a defined reference frame. Common frames include an inertial frame for orbit mechanics, an Earth-fixed frame for ground-referenced position, a spacecraft body frame fixed to the vehicle, and a local orbital frame such as Local Vertical/Local Horizontal (LVLH). ECSS-E-ST-10-09C standardises the treatment and documentation of coordinate systems because transformations between frames are fundamental to design and operations [26].

In an LVLH-type frame for an Earth-orbiting spacecraft, one axis is related to the local vertical and the remaining axes are formed from the orbital position and velocity directions according to the chosen convention. A nominal nadir-pointing spacecraft maintains a body orientation tied to this moving orbital frame. Target-tracking imaging may require a controlled offset from nadir so that the payload boresight intersects a ground target.

4.6.3 Attitude Representation: Euler Angles and Quaternions

Small attitude errors are often intuitive when expressed as roll, pitch and yaw angles, but Euler-angle sequences have singularities and are sequence dependent. Quaternions provide a compact, non-singular representation for numerical attitude propagation and control. A unit attitude quaternion q contains four elements and satisfies:

$$q_0^2 + q_1^2 + q_2^2 + q_3^2 = 1 \quad (4-10)$$

The sign of a quaternion is not unique: q and $-q$ represent the same physical rotation. Operational interpretation should therefore focus on the represented attitude and derived errors rather than on a single quaternion element in isolation. MCS26 displays both quaternion components and LVLH-relative roll, pitch and yaw to expose the relationship between machine-oriented and operator-oriented attitude data.

4.6.4 Attitude and Navigation Sensors

No single attitude sensor is ideal for every mission mode. Star trackers provide high-accuracy inertial attitude references by imaging star fields. Sun sensors provide a robust Sun direction and are important for acquisition and safe modes. Magnetometers measure the local geomagnetic field and support attitude estimation and magnetic control. Gyroscopes measure angular rate and provide high-bandwidth short-term propagation between absolute measurements. Earth or horizon sensors provide a planet-referenced direction. GNSS receivers can provide orbit position, velocity and time in Earth orbit [31].

Sensor fusion combines complementary measurements. A star tracker can provide accurate absolute attitude but may be temporarily unavailable; a gyro can propagate attitude through the outage but accumulates bias error over time. Estimators such as extended Kalman filters combine the measurements and models to obtain a best estimate of attitude, rate, sensor bias or other states. The estimator also produces residuals or innovations that are useful for monitoring consistency.

4.6.5 Actuators and Momentum Management

Reaction wheels generate spacecraft torque by changing the angular momentum of internal rotating masses. They provide fine, propellant-free three-axis control but have finite momentum storage. Environmental disturbance torque can drive wheel speeds toward saturation. Magnetorquers interact with the geomagnetic field to generate torque and are commonly used for detumbling or momentum unloading in Earth orbit. Thrusters can provide larger torques and translational impulse but consume propellant. NASA's current small-spacecraft GNC survey identifies reaction wheels, magnetic torquers and thrusters as common spacecraft actuators [31].

For a rigid spacecraft, the rotational dynamics can be represented in body coordinates by Euler's rigid-body equation:

$$\tau = I d\omega/dt + \omega \times (I\omega) \quad (4-11)$$

where τ is applied torque, I is the inertia tensor and ω is body angular rate. MCS26 does not integrate this full rigid-body equation; it generates attitude, rate, wheel and control telemetry that is consistent with the selected operational mode.

4.6.6 Estimation, Guidance and Control Loop

A closed-loop AOCS can be understood as a sequence of functions. Guidance defines the desired attitude or pointing target. Sensors measure the spacecraft and environment. An estimator combines measurements and models to determine the current state. The controller compares the estimated state with the commanded state and calculates actuator requests. Actuators generate torque, the spacecraft responds dynamically, and sensors observe the new state. This loop repeats at the control-cycle frequency.

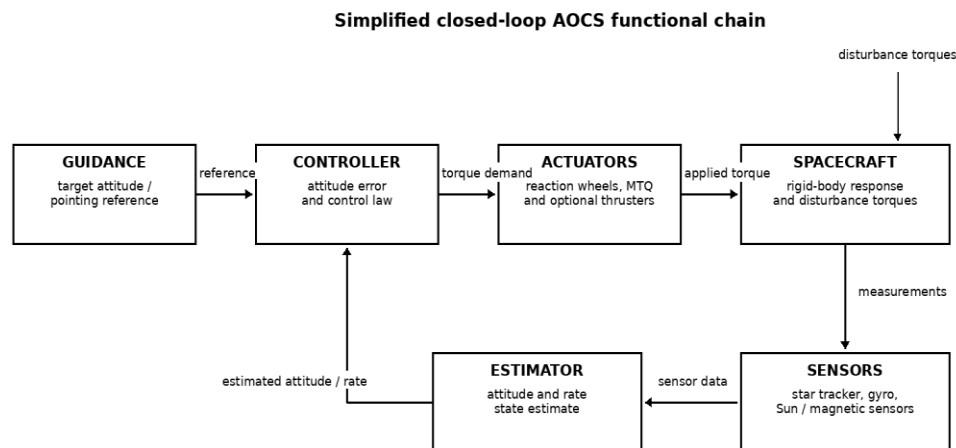


Figure 4-3 - Simplified closed-loop AOCS functional chain.

4.6.7 Pointing Knowledge, Pointing Error and Settling

Pointing performance has several distinct contributors. Attitude knowledge error is uncertainty in the estimated orientation. Attitude control error is the difference between the commanded and achieved orientation. Payload pointing error additionally includes alignment between the spacecraft reference frame and payload boresight, structural thermoelastic effects and instrument-specific geometry. Pointing stability or jitter describes short-timescale variation during an exposure. An imaging command should normally be issued only after the vehicle has slewed, settled and met the required knowledge/control performance.

This distinction is relevant in MCS26 because the AOCS panel includes attitude knowledge error, attitude control error, payload pointing error and a settling-status parameter. During the Frankfurt target-tracking windows the AOCS guidance target changes from nominal nadir to a target-tracking state and the simulated attitude errors are reduced to represent a controlled imaging condition.

4.6.8 Orbit and Navigation State

Navigation describes the spacecraft's translational state and its uncertainty. For an Earth-orbiting spacecraft, position and velocity may be represented in an Earth-centred inertial frame, Earth-fixed frame or as geodetic latitude, longitude and altitude. Operations also require knowledge of solution age and covariance or sigma values. A numerical position without its reference frame, epoch and uncertainty is incomplete.

MCS26 displays ECI position and velocity components, geodetic latitude/longitude, orbital altitude, position and velocity sigma values and covariance-style telemetry. These quantities are generated by a generic simulator model. They are not the output of an orbit determination process and are not used to propagate the ground-pass geometry described in Section 4.3.

4.6.9 AOCS Representation in MCS26

The AOCS telemetry inventory is intentionally broad. It includes an AOCS computer and processing state, quaternions, LVLH-relative angles, body rates, attitude errors, star trackers, Sun sensors, Earth/horizon sensors, magnetometer, rate gyros, fibre-optic gyro entries, IMU/accelerometers, GNSS/navigation data, reaction wheels, momentum state, magnetorquers and estimator/guidance/control status. Thruster and other actuator-interface entries are included as contextual telemetry even though propulsion is not an active standalone subsystem in the current scenarios.

Mission activity drives the AOCS operating context. The baseline mode is nominal nadir pointing. During Frankfurt imaging in Scenario 1, and during the recovered Frankfurt imaging window in Scenario 2, the model transitions to target tracking. The pre-recovery Scenario 2 test image remains a calibration/nadir activity. These mode changes influence computer load, attitude-error scale, guidance target and selected payload-readiness relationships.

Table 4-6 - AOCS functional groups and representative MCS26 telemetry

Function	Representative parameters	Engineering meaning
Attitude state	ATT101-104, ATT111-113, ATT121-123	Quaternion, LVLH angles and body rates.
Pointing performance	ATT130-136	Knowledge validity, knowledge/control/pointing errors and settling.
Star sensing	STR201-215	Tracker state, quality, stars tracked, temperature and residual.
Sun / Earth sensing	SUN301-315, EHS320-322, IRS330-332	External attitude-reference measurements.
Magnetic / inertial sensing	MAG401-407, GYR501-513, FOG521-523, IMU530-534	Field, angular-rate and inertial measurements.
Navigation	NAV601-606, ORB611-643, COV650-655	Position, velocity, geodetic state and uncertainty.
Actuation	RW701-735, MTQ801-806, ACT950-964	Reaction-wheel, magnetic and optional propulsion interfaces.
Estimation / guidance / control	CTL901-912	Estimator, Kalman, guidance target, control law and manoeuvre state.

4.7 Command and Data Handling (C&DH)

4.7.1 Onboard Computer and Processing

C&DH provides the computing and information-management infrastructure through which spacecraft subsystems are commanded, monitored and coordinated. Typical functions include onboard processing, command decoding and dispatch, telemetry collection, time management, mass-memory management, file services, data-bus routing, event handling and fault-management support. NASA mission descriptions and small-spacecraft surveys consistently treat C&DH as a central spacecraft function linking payload, subsystems and communications [36].

Flight architectures vary from a single central onboard computer to multiple redundant processors and distributed subsystem controllers. Redundancy can include cold, warm or hot standby computers, redundant memory banks and cross-strapped interfaces. A watchdog monitors software execution and can initiate recovery actions if the processor fails to service it within the required interval.

4.7.2 Onboard Data Buses and Interfaces

Subsystems exchange commands and telemetry over onboard data interfaces. Common spacecraft interfaces include SpaceWire, CAN, MIL-STD-1553, RS-422 and mission-specific serial or discrete links. SpaceWire networks use point-to-point links and routing switches to transport packets between nodes [37]. The choice of bus affects bandwidth, determinism, fault containment, redundancy, mass and software architecture.

MCS26 presents several interface families in the C&DH panel to familiarise trainees with the kinds of interconnects found on spacecraft. Their displayed states should not be interpreted as a declaration that the simulated spacecraft has a physically realised network containing every listed technology.

4.7.3 Memory and Mass Data Storage

Onboard memory supports several different purposes: boot software, non-volatile configuration, working memory, event logs, payload data and stored telemetry. Mass memory or a solid-state recorder decouples data production from contact availability. A payload can acquire data outside a ground pass, store it onboard, and downlink it later when a suitable link becomes available. This is especially important when payload data rate exceeds the instantaneous downlink capacity.

Memory utilisation is therefore an operational resource. If the recorder approaches capacity, further acquisition can overwrite data, be inhibited, or force a change in mission plan depending on spacecraft design. A data dump reduces stored data only if the downlink and ground data path can sustain the transfer and the onboard file or packet-management functions release the corresponding storage.

4.7.4 Command, Telemetry, Time and Fault Management

The command chain validates and dispatches requests to onboard functions. The telemetry chain acquires engineering data, formats it into packets or frames, applies prioritisation and routes it to the communications subsystem. Spacecraft time provides a common epoch for telemetry, event correlation, scheduled commands and payload products. Time synchronisation and 1-PPS validity are therefore not cosmetic parameters; inconsistent timing can prevent accurate reconstruction of events or observations.

Fault Detection, Isolation and Recovery (FDIR) is often distributed between subsystem electronics and central software. It can detect out-of-range conditions, software failures or inconsistent states and apply autonomous protection. MCS26 displays FDIR-related states for educational context, but the scenario logic is not a verified flight FDIR implementation.

4.7.5 C&DH Representation in MCS26

MCS26 treats mass memory as part of C&DH rather than as a separate spacecraft subsystem. The memory-used parameter MEM221 is directly coupled to the procedure. A memory dump begins only after the required SOM authorisation and SPACON command sequence, after which the simulator reduces used memory toward a low post-dump value. C&DH telemetry derives dump progress, dump pointer, queue usage and processing load from this shared state.

Command statistics are coupled to the actual simulator telecommand history. Command acceptance and rejection counts, last command and last result are therefore derived from operator actions rather than from unrelated random values. This is an important example of a high-value training coupling: the C&DH panel provides technical evidence of what SPACON has done.

Table 4-7 - C&DH functional groups and representative MCS26 telemetry

Function	Representative parameters	Purpose
Processing / redundancy	OBC101-103, CPU110-114, FPGA120-122	Computer state, load, temperature and active unit.
Watchdog / reset	WDG140/141, RST150-152	Execution supervision and reset history.
Memory	MEM001-902	Volatile/non-volatile memory, mass memory, ECC, queues and integrity.
Onboard network	NET201, SPW210-213, CAN220-222, MIL230, RS422251	Data-bus state, utilisation and errors.
Command handling	CMD301-308	Queue, accepted/rejected commands, decoder/authentication state.
Telemetry handling	TM401-408	Packet routing, generation rate, buffers and services.
Software services	SW501-532	Boot, RTOS, scheduler, mode, FDIR, files and onboard procedure services.
Time	TIM601-605	Onboard time, synchronisation, PPS and drift.
Data recorder / dump	DMP801-804	Dump state, progress, rate and integrity.

4.8 Spacecraft Telecommunications and the Ground Link

4.8.1 Spacecraft RF Chain

A spacecraft telecommunications subsystem provides the RF interface between onboard data handling and the ground. Typical elements include receivers, transmitters or transponders, modulators/demodulators, frequency references, filters, power amplifiers, RF switches, duplexers and one or more antennas. Antenna gain and beamwidth are selected together with transmitter power, frequency band, data rate, spacecraft pointing capability and ground-station performance [27].

The uplink carries commands, ranging tones or other ground-to-space information. The downlink carries telemetry, payload data and tracking observables. Uplink and downlink are related operationally but can have different frequencies, margins and failure modes. A spacecraft can, for example, transmit a detectable carrier while telemetry framing is invalid, or receive an uplink while the ground is unable to recover the downlink.

4.8.2 Link Budget and Margin

A link budget accounts for transmitter power, transmit-antenna gain, propagation loss, additional path losses, receive-antenna gain, receiver noise and required demodulation/coding performance. In logarithmic units the terms are added and subtracted. The purpose of the budget is not merely to predict received power, but to demonstrate adequate margin across the required geometries, environments, equipment tolerances and operational modes.

Because slant range and atmospheric path length change throughout a pass, a low-Earth-orbit ground link is dynamic. Antenna pointing, Doppler and link margin evolve rapidly around AOS and LOS. The ground station therefore performs prediction, antenna tracking, frequency acquisition, carrier tracking, data synchronisation and monitoring as coordinated functions.

4.8.3 MCS26 Communications Representation

The current MCS26 baseline does not provide a separate spacecraft COMMS telemetry panel. Instead, the space-to-ground communications state is represented primarily through GS1/GS2 ground-station telemetry and through the global spacecraftTelemetryAvailable state consumed by the spacecraft subsystem models. This design is appropriate for the training scenarios because the operational question is whether the ground has a usable telemetry path and how the ground receiver chain behaves during acquisition, degradation, loss and recovery.

The ground model includes an S-band-style configured downlink frequency near 2250 MHz, bounded Doppler, receiver signal strength, carrier/noise quantities, BER/FER, carrier lock, bit synchronisation, frame synchronisation, modulation/coding state and received telemetry data rate. The numerical relationships are synthetic. They are intended to maintain a coherent training story - for example, filtered signal quality improves Eb/N0 and error behaviour - rather than to represent a qualified radio design.

4.9 Optical Imaging Payload

4.9.1 Payload Function and Spacecraft Interfaces

The payload is the mission equipment that produces the primary observation or service. For an optical Earth-observation mission, the payload typically contains an optical system, detector or focal-plane assembly, instrument electronics, timing and control, calibration functions, thermal interfaces and a data path to onboard storage. Real instruments are integrated with the spacecraft through electrical, thermal, mechanical, data, timing and pointing interfaces [34][35].

An imaging payload cannot be considered ready solely because its power switch is on. It requires correct instrument mode, stable detector and optics temperatures, valid timing, adequate memory, a configured acquisition chain and spacecraft pointing compatible with the observation. These preconditions are why payload operations normally require cross-subsystem coordination.

4.9.2 Optical Chain and Focal Plane

An imaging instrument uses lenses or mirrors to form an image on a detector, where incident photons are converted to electrical signal and subsequently to digital numbers [34]. Optical performance depends on focus, alignment, throughput, contamination and stray light. Detector performance depends on temperature, bias, gain, read noise, dark current, saturation and defective pixels. The focal-plane electronics then clock, amplify and digitise the detector signal.

Modern Earth-observation instruments can contain large focal-plane assemblies and multiple detector bands. ESA's Sentinel-2 multispectral instrument, for example, uses multiple detector assemblies and requires high opto-mechanical stability and calibration hardware to meet geometric and radiometric performance [35]. MCS26 is not a Sentinel-2 model; the example demonstrates why detector, optics, thermal and calibration telemetry are operationally interconnected in a realistic optical payload.

4.9.3 Calibration and Timing

Calibration determines how detector output relates to a known physical input or reference. Ground calibration establishes pre-launch response, while many instruments include in-flight calibration using diffusers, lamps, blackbodies, dark views, celestial targets or other references. Calibration validity can drift with ageing, contamination or thermal conditions. Payload timestamps must also be tied to spacecraft time so that observations can be geolocated and correlated with attitude and orbit data.

4.9.4 Image Data Generation, Compression and Storage

Raw detector data can be far larger than the data volume that can be downlinked immediately. Onboard processing can perform correction, formatting and compression before storing a file or packet stream in mass memory. Compression reduces the required storage and downlink time but can consume processor resources and, for lossy schemes, alter image information. The resulting data product must retain enough metadata to identify acquisition time, configuration and validity.

This creates a direct payload-C&DH coupling. Image acquisition increases recorder utilisation, the compression processor and data queues become active, and the file must remain intact until it is transferred or deliberately deleted. A full memory condition can therefore prevent a new observation even when the payload itself is electrically and thermally healthy.

4.9.5 Payload Readiness

A useful operational readiness concept is the logical AND of independent preconditions: power available, thermal state acceptable, pointing/rate interface valid, configuration complete, storage available and time valid. A single failed precondition can make the observation invalid. In flight software these conditions may be enforced by command verification, onboard autonomy or procedure rules; in MCS26 they are represented through scenario state and telemetry.

4.9.6 Payload Representation in MCS26

The MCS26 payload model represents a generic optical imaging instrument. It includes payload overall health, electronics and computer states, electrical power conversion, controller/FPGA/CPU loading, camera configuration, optical baffle and lens state, optical-bench temperature, detector/focal-plane telemetry, GNSS/timing validity, calibration state, imaging-window status, exposure/gain, image dimensions and quality estimates, compression, storage-interface telemetry and thermal margins.

The model is coupled to mission state. Payload active/standby behaviour depends on payload power and spacecraft standby. Acquisition readiness requires active payload power, camera configuration, acceptable thermal condition, sufficient memory and absence of power-saving/standby states. Scenario 1 uses a Frankfurt Airport target window; Scenario 2 first uses a calibration-test window and, after recovery and procedure import, a later Frankfurt target window. AOCS guidance state changes consistently with these imaging contexts.

When an image is taken, the payload and C&DH models respond to the shared image state: file information, data recorder state, compression queues and memory-related telemetry change accordingly. The dedicated Captured Image panel then provides the operator-facing image product, name and validity state.

Table 4-8 - Payload functional groups and representative MCS26 telemetry

Payload function	Representative parameters	Engineering interpretation
Health / mode	PLD600-624	Instrument health, mode and acquisition readiness.
Electrical interface	PWR701-706, PLD740-744	Payload bus, converters and detector rails.
Processing	PLD331, PLD550-556	Controller, FPGA, CPU and software state.
Optics / camera	CAM000-204, OBF044/045, LNS054-056, OPT101-105	Camera configuration, optics temperatures, focus and alignment.
Detector / focal plane	SEN331-338, FPA341-345	Detector thermal/electrical/readout condition.
Timing	GPS112-117	PPS/GNSS/timestamp validity.
Calibration	CAL401-406	Calibration state, age and residual.
Image acquisition	IMG901-916	Window, target, exposure, image metadata and quality.
Compression / storage	CMP551-558, DAT601-608	Compression load, file size, buffers and recorder interface.
Thermal / interfaces	RAD087/088, THM701-705, IFC801-806	Thermal margin and AOCS/EPS/TCS/memory/time preconditions.

4.10 Structure, Mechanisms and Propulsion in the System Context

4.10.1 Structure and Mechanisms

The spacecraft structure provides the load path between equipment, payload, launch-vehicle interface and mechanisms. It must survive launch loads while maintaining alignment and dimensional stability in orbit. Mechanisms can include solar-array drives, antenna deployment systems, covers, shutters, filter wheels, latches and gimbals. These devices create interfaces with EPS, TCS, AOCS and C&DH because they require power, generate heat, have command/telemetry interfaces and can change spacecraft configuration.

MCS26 includes selected mechanism-related telemetry - for example solar-array drive state, optional payload covers/doors and focus mechanisms - but does not model structural dynamics, deployment mechanics or mechanism failure propagation. These parameters are contextual engineering telemetry.

4.10.2 Propulsion

A propulsion subsystem stores and conditions propellant and generates thrust for orbit changes, station keeping, momentum management or attitude manoeuvres depending on spacecraft design. Propulsion interfaces with AOCS for manoeuvre direction and timing, TCS for tank/thruster temperatures, EPS for valves/heaters and C&DH for command and telemetry. Propulsive manoeuvres also affect navigation state and subsequent ground-contact predictions.

The current MCS26 scenarios do not execute a physical orbit manoeuvre and contain no standalone propulsion panel. Thruster and propulsion-interface entries in AOCS/TCS are therefore informational placeholders. They should not be interpreted as an implemented propulsion simulation.

4.11 Cross-Subsystem Dependencies Used in Operations

4.11.1 EPS-TCS Coupling

Electrical consumption produces heat. Higher payload or transmitter power therefore increases both EPS load and thermal dissipation. TCS may respond through radiators, heaters or active cooling, while EPS must supply any active thermal-control power. Battery and power-electronics temperatures can in turn restrict allowable electrical operation. In MCS26, EPS temperature and payload power are directly coupled to TCS telemetry and to the mitigation sequence.

4.11.2 EPS-Payload Coupling

Payload operation is usually one of the major discretionary spacecraft loads. Mission procedures commonly sequence payload activation only after confirming bus margin, battery condition and thermal

readiness. MCS26 follows this logic: payload power is increased only after the required EPS mitigation/confirmation steps, and power-saving or standby modes suppress payload activity.

4.11.3 AOCS-Payload Coupling

Optical imaging requires the payload line of sight to intersect the desired target with adequate pointing knowledge and stability. The instrument can be configured correctly yet produce unusable data if attitude error or angular rate is excessive. MCS26 therefore exposes AOCS pointing interfaces in the payload model and changes the AOCS guidance target during the defined imaging windows.

4.11.4 Payload-C&DH Coupling

The payload creates data; C&DH must time-tag, process, store and later route that data. Memory capacity and file-system health are therefore payload-operability constraints. MCS26 begins with a high payload-memory utilisation and requires a memory dump before later payload activity, explicitly teaching that storage is a mission resource.

4.11.5 Communications-C&DH and Communications-All-Subsystem Coupling

C&DH produces the telemetry stream and accepts decoded commands, while communications provides the transport path. Every subsystem becomes operationally invisible to the ground when this path is lost, even though onboard functions may continue. This is why communications is not merely another subsystem in operations: it is the observation and control path through which the ground accesses all other subsystems.

4.11.6 Time as a Cross-Cutting Interface

Spacecraft time connects telemetry, commands, attitude, orbit and payload products. An image timestamp is meaningful only when it can be correlated with spacecraft position and pointing at the same epoch. Event reconstruction likewise depends on a common time base. MCS26 uses mission seconds as the primary scenario time and exposes onboard-time and timing-validity telemetry in C&DH and Payload.

4.11.7 Contingency Coupling in Scenario 2

Scenario 2 combines multiple subsystem effects to demonstrate why anomaly response is interdisciplinary. An abnormal battery distribution drives a recovery/equalisation activity. Power-saving changes the load state. Payload power and EPS temperature influence thermal conditions. A deliberate RF degradation then removes GS1 telemetry visibility. The team must continue from the last known state, coordinate a recovery procedure, acquire GS2, recover telemetry, re-establish acceptable EPS/thermal conditions, restore payload power and configuration, and finally perform the delayed imaging task.

This sequence should not be interpreted as a physically validated causal model in which battery equalisation or power-saving necessarily causes a ground-link failure on a real satellite. Its purpose is to force the operators to manage interacting state, degraded observability and procedural recovery. The engineering concepts within each subsystem are real; the exact contingency chain is scenario logic.

Table 4-9 - Cross-subsystem dependency matrix used for training

Source / condition	Affected domain	Operational consequence in MCS26
Payload power increase	EPS + TCS	Higher payload load; EPS temperature rises; thermal confirmation required.
Thermal mitigation / cooling	EPS + TCS + Payload	Temperature decreases; payload power remains constrained until acceptable state is confirmed.
Power-saving mode	EPS + Payload	Reduced payload load and non-essential activity.
Battery equalisation	EPS + TCS	Battery SOC redistribution and modest thermal/BMS response.
Image acquisition	Payload + C&DH	Image state becomes valid; storage/data-recorder and processing telemetry respond.
Mass-memory dump	C&DH + operations	MEM221 decreases; later payload activities gain storage margin.
Target-tracking window	AOCS + Payload	Guidance target and pointing context change for imaging.
GS1 link loss	Ground + all spacecraft panels	Spacecraft telemetry becomes unavailable from ground.
GS2 elevation ≥ 5 deg after GS1 loss	Ground + all spacecraft panels	Telemetry path is re-established for contingency recovery.
Spacecraft standby	Payload + EPS/TCS context	Payload is deactivated and low-power end state is established.

4.12 Interpretation of the MCS26 Engineering Model

4.12.1 Synthetic Telemetry

The telemetry values in MCS26 are synthetic. In ECSS monitoring-and-control terminology, a synthetic parameter is reporting data generated within the monitoring and control system from other reporting data and constants [33]. MCS26 uses a broader simulator version of that idea: many values are calculated from mission time, shared scenario states and bounded mathematical functions. They are not received from spacecraft hardware and are not replayed from a flight telemetry archive.

The use of synthetic telemetry is intentional. It permits deterministic training behaviour, ensures that the procedures can be repeated, and allows values in different panels to respond consistently to the same operator action. For example, payload power influences both EPS electrical telemetry and TCS thermal telemetry; memory dump state influences C&DH queues and progress; and imaging state influences payload and C&DH data products.

4.12.2 Deterministic Variation versus Physical Simulation

Many secondary telemetry values use bounded sinusoidal variation around a nominal centre. This prevents the interface from appearing artificially static and lets operators practise reading continuously changing measurements. Such variation should be interpreted as telemetry texture, not as a solution of the governing physical differential equations. State-coupled parameters carry more operational meaning because they change in response to procedure actions, mission mode or anomaly state.

The trainee should therefore distinguish three classes of displayed information: scenario truth that controls the procedure; state-coupled engineering telemetry derived from that truth; and supplemental telemetry included for engineering realism but not used to determine procedure progression. A parameter's presence on a panel does not imply that it is part of the active scenario decision chain.

4.12.3 Status Colours and Limits

Green, yellow and red status classes in the simulator provide an immediate monitoring aid, but the underlying thresholds are simulator-development values. In a flight control system, limits are defined, verified, configuration-controlled and often dependent on mode, validity and persistence. A single static threshold can be inadequate when a parameter has different acceptable ranges during warm-up, operation, safe mode or calibration.

For training, the operator should first read the value, unit and subsystem context and then use the displayed status and procedure criterion. The colour is not a substitute for engineering interpretation. This becomes particularly important for parameters such as elevation, where a physically valid number below the operational mask may still be unacceptable for the next procedure step.

4.12.4 What the Model Can and Cannot Demonstrate

The current model can demonstrate spacecraft subsystem vocabulary, telemetry interpretation, functional coupling, operational sequencing, loss and recovery of observability, command discipline, use of ground-station geometry, payload-resource constraints and coordinated response to a contingency. It can also serve as a software framework into which higher-fidelity physical or recorded telemetry models can be introduced later.

The current model cannot establish flight performance, flight limits, reliability, quantitative link margin, orbit-prediction accuracy, thermal qualification, battery life, pointing accuracy, command protocol compliance or mission safety. Those conclusions require mission-specific design data, validated models, calibrated hardware behaviour, configuration-controlled operational databases and formal verification. Later sections of this manual use the engineering concepts defined here to explain the two implemented training scenarios without crossing that boundary.

5 Mission Scenario Operations

This section defines the operational use of the two implemented MCS26_V26_01 training scenarios. The descriptions below are based on the current English application baseline and are intended to be sufficient for a complete multi-console run without requiring the operator to infer missing actions. The procedure table displayed on the SOM console is treated as the controlling operator sequence. The simulator's state-transition logic then determines whether a requested action is accepted, rejected, in progress or complete.

5.1 General Rules Applicable to Both Scenarios

5.1.1 Role Authority and Console Discipline

Four operator positions are used in the recommended classroom configuration: SOM, SPACON, SOE1 and SOE2. The SOM owns procedure progression and is the only role that initiates the simulation through the procedure interface. SOE1 and SOE2 inspect engineering telemetry in the subsystem panels and report the requested values to the SOM. SPACON executes simulator telecommands after the SOM has formally released the corresponding procedure action.

The operator roles are intentionally separated. A SOM shall not replace an SOE report by silently reading the parameter from another console, and SPACON shall not execute an operational command simply because the command is visible in the command catalogue. The educational objective is to preserve the distinction between observation, decision and commanding that was established in Section 4.

Table 5-1 - Mandatory role sequence

Procedure type	Required sequence	Do not proceed when
Telemetry/reporting step	1. SOM verbally requests the value from the assigned SOE. 2. SOE opens the specified panel and reads the exact parameter(s). 3. SOE reports value, unit and status. 4. SOM compares the report with the displayed criterion. 5. SOM presses the procedure action button only after the report has been assessed.	The parameter is NO TELEMETRY / NO DATA, the wrong panel or code was read, or the reported condition does not satisfy the required criterion.
SPACON command step	1. SOM presses the procedure action button, thereby releasing the command request. 2. SPACON searches/selects the stated code. 3. SPACON confirms the command name. 4. Press ARM. 5. Press GO. 6. Verify the TC History result. 7. Wait for the procedure status to become DONE before advancing.	The SOM has not released the command, the wrong code is selected, the command is not armed, the link condition blocks the command, or the resulting TC History entry begins with FAILED.
SOM decision step	1. Review the latest SOE report and the criterion. 2. Press the SOM decision/confirmation button. 3. Verify that the step status changes from CURRENT.	The criterion is not met or the simulator returns a FAILED result.

5.1.2 Procedure-State Indications

Procedure rows progress through PENDING, CURRENT and DONE. A command step can additionally show SPACON REQUIRED after the SOM has released a command but before SPACON has completed it, and IN PROGRESS while a time-dependent state transition is running. A row marked CURRENT identifies the only procedure step that should be actively executed. An operator shall not attempt to 'prepare' a later command by executing it early; the command logic rejects out-of-sequence execution.

The Last Result field provides the most recent simulator response. FAILED messages are operationally significant and shall be read before repeating an action. Repeatedly pressing buttons without resolving the stated precondition is not an accepted recovery technique.

5.1.3 SPACON SELECT-ARM-GO Execution

SPACON commands are executed through a three-stage control sequence. SELECT identifies the command. ARM establishes the command as ready for execution. GO invokes the command logic and

records the result in TC History. If no command is selected, ARM fails. If GO is pressed without ARM, execution fails. After GO, the selected command is cleared and the arm state is reset; the next command must therefore be selected and armed again.

The command catalogue also contains read-only telemetry parameters. Selecting a telemetry-only entry returns a monitoring-reference success message but does not change simulator state. Operational steps in this manual always identify the required command code and command name so that a parameter-only entry is not mistaken for the commanded action.

Table 5-2 - Operational commands used by Scenarios 1 and 2

Code	SPACON command name	Used in	Primary simulator effect
GSE001	Filter Signal	Scenario 1 Step 5; Scenario 2 Step 22	Sets the active GS receiver path to the filtered/GOOD training state after required checks.
MEM221	Dump Memory	Both scenarios Step 9	Starts payload mass-memory reduction and clears previously stored image state.
BAT330	Battery Equalization Transfer	Scenario 2 Step 13	Redistributes the three simulated battery SOC values toward 34.7%.
PSM001	Enter Power Saving Mode	Scenario 2 Step 16	After transition, sets low-power mode and payload power level to 8 W.
EPT014	Reduce Payload Power	Scenario 1 Step 14; Scenario 2 Step 24	Starts payload load reduction, activates thermal mitigation and clears camera readiness.
PWR740	Increase Payload Power	Scenario 1 Step 17; Scenario 2 Step 27	Raises payload power toward 180 W and begins scenario-specific thermal rise.
CAM000	Configure Camera	Scenario 1 Step 18; Scenario 2 Step 28	Sets the camera configuration to READY when power/EPS gates are satisfied.
IMG901	Take Image	Scenario 1 Step 20; Scenario 2 Step 31	Captures the time-dependent simulated image product and validity state.
STB901	Spacecraft Standby Mode	Scenario 1 Step 21; Scenario 2 Step 32	Places the simulated spacecraft/payload in the low-power end state and starts the end sequence.

5.1.4 Link-Quality Command Gating

Most operational commands require the currently active ground link to be in the simulator GOOD state. This check prevents the nominal procedure from commanding through a deliberately degraded link. The contingency logic makes four exceptions so that recovery actions can still be executed while the signal is weak: Battery Equalization Transfer, Enter Power Saving Mode, Reduce Payload Power and Filter Signal. Once Scenario 2 has transitioned to the filtered GS2 path, subsequent command gating uses the GS2 signal quality.

CAUTION. If a command returns FAILED - SIGNAL NOT GOOD or FAILED - GS2 SIGNAL NOT GOOD, do not bypass the condition. Re-establish the link state required by the current procedure step.

5.1.5 Mission Time, Fixed Windows and Event-Driven Steps

Mission time is a shared scenario variable formatted as T+MM:SS. Some transitions occur at fixed mission times, while others occur only after an operator action. This distinction is critical. GS1 AOS is fixed at T+01:00. The Scenario 1 Frankfurt imaging window is fixed at T+15:00 to T+16:00 and its GS1 LOS logic is fixed at T+17:00. In the recovered branch of Scenario 2, the formal Frankfurt imaging window is T+30:00 to T+30:30. By contrast, the Scenario 2 GS1 loss and the subsequent GS2 acquisition sequence are triggered relative to the time at which the operators reach the relevant contingency steps.

The mission clock continues while operators communicate, wait for a command to complete or diagnose a condition. A procedure can therefore miss a fixed imaging opportunity if the team progresses too slowly. Conversely, payload power shall not be restored excessively early merely because the procedure step has become available: the thermal model continues to evolve with mission time.

5.1.6 Interpretation of Approximate UI Durations and Role Text

Several procedure rows display approximate execution times. These values are training cues, not hard software timers. The current state-transition algorithms can complete sooner or later depending on the starting state. The operator shall therefore use the step status and telemetry result as the acceptance criterion rather than a stopwatch. In particular, memory dump, battery equalisation, payload-power transition and thermal recovery complete when their state criteria are reached.

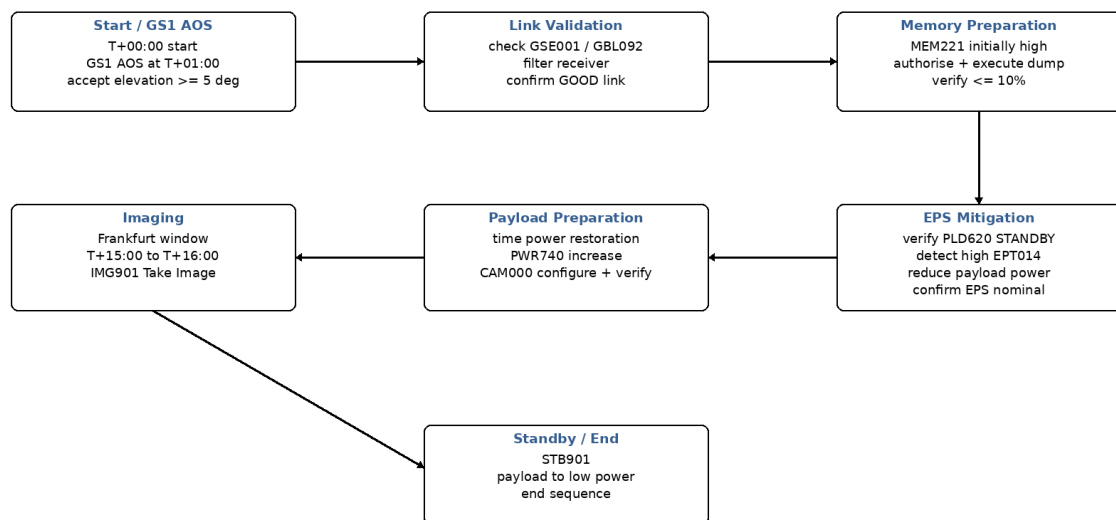
Where a transient Last Result string contains a role label that differs from the role printed in the SOM procedure table, the procedure-table assignment is controlling for console operation. This manual follows the procedure-table role allocation. The Last Result field remains useful for the engineering value and state result but is not used to reassign console responsibility.

5.2 Scenario 1 - Nominal Ground Pass and Frankfurt Airport Imaging

5.2.1 Mission Objective and Training Intent

Scenario 1 is the nominal mission-operations sequence. The team acquires GS1, validates the downlink, prepares mass memory, identifies a non-nominal EPS thermal condition, performs load reduction and thermal recovery, restores payload power, configures the camera, acquires the Frankfurt Airport image inside the fixed imaging window and finally places the spacecraft in standby. The scenario is nominal in the sense that no loss of spacecraft telemetry or external contingency procedure is introduced; it still contains an expected EPS mitigation task so that the operators must demonstrate cross-subsystem coordination.

Scenario 1 - Nominal Ground Pass and Imaging



Fixed mission-time constraints: imaging window T+15:00 to T+16:00; GS1 LOS logic at T+17:00.

Operational rule: completion status and telemetry criteria control progression; approximate UI execution times are not hard timers.

Figure 5-1 - Scenario 1 operational flow.

5.2.2 Initial Conditions and Fixed Mission Events

Table 5-3 - Scenario 1 baseline conditions

Item	Baseline / event	Operational significance
Simulation status	IDLE before SOM start	Mission timer does not advance until SOM starts the scenario.
Initial mass memory	MEM221 = 87%	Memory dump is required because occupancy exceeds the Step 8 threshold.
Payload instrument	PLD620 = STANDBY	Payload is not yet at imaging power.

Item	Baseline / event	Operational significance
Initial payload power state	PWR740 model starts at 20 W	Load is reduced during EPS mitigation, then later raised to 180 W.
Initial EPS temperature	EPT014 model starts at 84.5 deg C and can rise before mitigation	The baseline reaches a condition requiring thermal mitigation.
GS1 AOS	T+01:00	Procedure Step 1 completes automatically at AOS.
Elevation acceptance mask	5.0 deg	Step 3 cannot complete below this value.
Frankfurt imaging window	T+15:00 to T+16:00	IMG901 shall be executed inside this window.
GS1 LOS logic	T+17:00	After LOS the nominal GS1 link is no longer usable for commanding.

5.2.3 Phase A - GS1 Acquisition and Downlink Validation (Steps 1-6)

Step 1 - Start simulation and acquire GS1

Assigned role	SOM
Entry condition	All four clients are connected, Scenario 1 has been selected locally on every client, and the host services are stable.
Required action	Press Start Simulation on the SOM procedure table. Observe the mission clock and GS1 AOS countdown. No further procedure action is required before AOS.
Expected indication	Before T+01:00, GS1 geometry is unavailable. At T+01:00 the AOS state becomes active and GS1 elevation begins from the horizon.
Completion / hold criterion	Proceed only after Step 1 becomes DONE. Do not treat the first non-null elevation as acceptance of the 5 deg operational mask; that is Step 3.

Step 2 - Obtain GS1 elevation

Assigned role	SOM -> SOE1
Entry condition	Step 1 is DONE.
Required action	SOM asks SOE1 to open Ground Station and report GEL005 Antenna Elevation. SOE1 reports the current numerical value in degrees. SOM then records the step with Request GS elevation status.
Expected indication	GEL005 increases from the AOS geometry as the pass develops. Other GS geometry such as azimuth and slant range also becomes available.
Completion / hold criterion	A value must have been reported. Step 2 does not itself require GEL005 >= 5 deg.

Step 3 - Confirm acquisition geometry

Assigned role	SOM
Entry condition	SOE1 has reported GEL005.
Required action	Compare GEL005 with the 5.0 deg elevation mask. Press Confirm acquisition geometry only when GEL005 is at least 5.0 deg.
Expected indication	At >= 5 deg, the ground-station carrier-lock logic is eligible for LOCK and TM-history generation can begin.
Completion / hold criterion	If the button is pressed below 5 deg, the simulator returns FAILED - ELEVATION BELOW 5 deg. Wait for elevation to rise and repeat the confirmation.

Step 4 - Assess initial GS1 downlink quality

Assigned role	SOM -> SOE2
Entry condition	Step 3 is DONE and GS1 elevation is accepted.
Required action	SOM asks SOE2 to read GSE001 Signal Quality and GBL092 Beacon Level / link-quality state on the Ground Station panel. SOE2 reports both. SOM records Verify downlink signal quality.
Expected indication	In the baseline unfiltered state above 5 deg, GBL092 is MEDIUM and GSE001 is a mid-level numerical percentage; therefore receiver filtering is expected to be required.
Completion / hold criterion	If the reported signal were already GOOD, the application can mark filtering unnecessary. In the baseline, continue to Step 5.

Step 5 - Filter the GS1 signal

Assigned role	SOM -> SPACON
Entry condition	Step 4 is complete and the initial signal is not GOOD.
Required action	SOM presses configure receiver chain. SPACON searches/selects GSE001 - Filter Signal, presses ARM, then GO, and checks TC History.
Expected indication	A successful command returns SUCCESS - SIGNAL FILTERED / SOM VERIFICATION REQUIRED. The step status changes to DONE.
Completion / hold criterion	If SPACON receives a failure indicating missing SOM request, unchecked signal quality or unconfirmed elevation, return to the stated prerequisite. The displayed '~10 s' is not an enforced delay in the current implementation.

Step 6 - Verify TM link after filtering

Assigned role	SOM -> SOE1
Entry condition	GSE001 Filter Signal completed successfully.
Required action	SOM asks SOE1 to re-read the Ground Station link indicators. SOE1 confirms the post-filter state. SOM presses Confirm TM lock.
Expected indication	GBL092 should be GOOD and GSE001 should show the higher filtered signal-quality range. Carrier and frame-related indicators should be consistent with a usable link.
Completion / hold criterion	The simulator accepts the step only when the active signal-quality state is GOOD.

5.2.4 Phase B - Payload Memory and Standby Verification (Steps 7-11)

Step 7 - Report mass-memory utilisation before dump

Assigned role	SOM -> SOE2
Entry condition	The GS1 link is verified GOOD.
Required action	SOM asks SOE2 to open C&DH and report MEM221 Payload Memory Used. SOE2 reports the percentage and SOM records Request MMU status.
Expected indication	The baseline value is 87%, significantly above the nominal <= 10% reference shown in the procedure.
Completion / hold criterion	Do not interpret the high value as a software fault. It is the intended precondition for the memory-management exercise.

Step 8 - Authorise payload memory dump

Assigned role	SOM
Entry condition	MEM221 has been reported.
Required action	Apply the procedure criterion. If MEM221 <= 50%, a dump is not required; if MEM221 > 50%, authorise the dump by pressing Authorize payload memory dump.
Expected indication	With the baseline MEM221 = 87%, the result is SOM AUTHORIZED - MEM221 > 50% / SPACON MAY DUMP MEMORY.
Completion / hold criterion	The current baseline requires the authorisation path because MEM221 is > 50%. A premature authorisation attempt at <= 50% is rejected.

Step 9 - Execute payload memory dump

Assigned role	SOM -> SPACON
Entry condition	Step 8 is authorised.
Required action	SOM presses Execute MMU dump. SPACON selects MEM221 - Dump Memory, ARM, GO. Keep the C&DH panel visible on one SOE console while the dump progresses.
Expected indication	Memory usage decreases from 87% in discrete updates until it reaches 5%. DMP801/DMP802 and related C&DH telemetry reflect dump state. Any previously stored captured-image state is cleared when the dump starts.
Completion / hold criterion	Wait until the procedure status becomes DONE and Last Result states MEMORY DUMP COMPLETE. The current algorithm normally requires about 14 simulation seconds from the baseline 87% state; the UI's '~15 s' is an approximation.

Step 10 - Verify memory after dump

Assigned role	SOM -> SOE1
Entry condition	Memory dump is complete.
Required action	SOM asks SOE1 to report MEM221 from C&DH. SOE1 reports the post-dump value. SOM presses Ask / Verify Memory.
Expected indication	MEM221 should be <= 10%; the baseline algorithm completes at 5%.
Completion / hold criterion	Do not proceed if MEM221 remains above 10%. This indicates that the dump has not reached the expected completion state.

Step 11 - Verify payload instrument standby

Assigned role	SOM -> SOE2
Entry condition	Post-dump memory state is acceptable.
Required action	SOM asks SOE2 to open Payload and report PLD620 Instrument Mode. Press Verify payload STANDBY mode after the SOE report.
Expected indication	PLD620 should read STANDBY because the payload has not yet been raised to imaging power.
Completion / hold criterion	Proceed only after the reported state is consistent with STANDBY.

5.2.5 Phase C - EPS Assessment and Thermal Mitigation (Steps 12-16)

Step 12 - Assess EPS thermal and power state

Assigned role	SOM -> SOE1
Entry condition	Payload is verified in STANDBY.
Required action	SOM asks SOE1 to open EPS and report EPT014 EPS Main Electronics temperature, DCC208 DC/DC Converter temperature and NET118 Net Power Margin.
Expected indication	The procedure reference is EPT014 < 75.0 deg C, NET118 approximately 1160 W, and DCC208 within the displayed nominal range. In the baseline run EPT014 is intentionally elevated and mitigation is required.
Completion / hold criterion	Press Request EPS telemetry status only after the values are reported. The next step is the mitigation decision.

Step 13 - Request thermal mitigation

Assigned role	SOM
Entry condition	EPS telemetry from Step 12 has been assessed.
Required action	If EPT014 is high, press Request thermal Mitigation. The displayed procedure identifies EPT014 > 85.0 deg C as an immediate-mitigation condition.
Expected indication	The SOM request clears any stale payload-power/camera/image state and releases the subsequent load-reduction command.
Completion / hold criterion	If EPS is already nominal the result can state that mitigation is not required; for the intended baseline, continue with load reduction.

Step 14 - Reduce payload power

Assigned role	SOM -> SPACON
Entry condition	Step 13 has released mitigation.
Required action	SOM presses Execute load shedding. SPACON selects EPT014 - Reduce Payload Power, ARM, GO.
Expected indication	The simulator starts payload-power reduction and activates thermal cooling. From the Scenario 1 baseline payload-power state, PWR740 falls toward the 10 W safe level; the camera readiness state is cleared.
Completion / hold criterion	Wait for DONE / POWER REDUCTION COMPLETE. The algorithm uses a state-dependent reduction rate and can complete in less than the '~15 s' procedure cue.

Step 15 - Report post-mitigation EPS state

Assigned role	SOM -> SOE2
Entry condition	Payload-power reduction is complete.
Required action	SOM asks SOE2 to report EPT014 and NET118 from EPS, then presses Request post-mitigation EPS telemetry.
Expected indication	NET118 is approximately 1130 W in the mitigation state. EPT014 decreases while thermal cooling is active.
Completion / hold criterion	This step records the report; it does not by itself declare EPS nominal. Continue to Step 16 only after observing the required temperature range.

Step 16 - Confirm EPS nominal

Assigned role	SOM
Entry condition	Post-mitigation telemetry has been reported.
Required action	Wait until EPT014 is within 70.0 to 75.0 deg C and the power margin remains acceptable. Press Confirm EPS parameters nominal.
Expected indication	The actual acceptance guard requires EPT014 between 70 and 75 deg C, NET118 >= 900 W and the scenario battery reference >= 75%. In the normal mitigation state NET118 is approximately 1130 W.
Completion / hold criterion	If the simulator returns FAILED - EPS NOT NOMINAL / WAIT OR MITIGATE, continue monitoring; do not force progression.

5.2.6 Phase D - Payload Restoration, Imaging and Standby (Steps 17-21)

CAUTION. Scenario 1 is time critical from this point. After payload power is raised, the EPS temperature rises at approximately 0.03 deg C per simulation second. If EPT014 exceeds 75 deg C after SOM nominal confirmation, the application automatically invalidates the downstream payload/camera/image readiness and returns the procedure to the mitigation sequence. Do not restore payload power unnecessarily early.

For the baseline timing, a practical operating target is to hold at Step 17 until approximately T+14:00 or later. This leaves enough time to raise payload power and configure/verify the camera before the T+15:00 imaging-window opening, while minimising the interval during which EPT014 rises.

Step 17 - Restore payload power for imaging

Assigned role	SOM -> SPACON
Entry condition	EPS is confirmed nominal and mission time is sufficiently close to the T+15:00 imaging window.
Required action	SOM presses Restore payload power for imaging. SPACON selects PWR740 - Increase Payload Power, ARM, GO.
Expected indication	PWR740 increases from the 10 W safe state toward 180 W in 17 W steps per simulation second. Completion normally takes about 10 simulation seconds. NET118 returns toward approximately 1160 W.
Completion / hold criterion	Wait for PAYLOAD POWER INCREASE COMPLETE. Continue to monitor EPT014; if it exceeds 75 deg C, the simulator reopens mitigation and the payload sequence must be repeated.

Step 18 - Configure the camera

Assigned role	SOM -> SPACON
Entry condition	Payload power is fully raised and EPS nominal confirmation is still valid.
Required action	SOM presses Configure payload imaging mode. SPACON selects CAM000 - Configure Camera, ARM, GO.
Expected indication	CAM000 changes from STANDBY to READY. The command is rejected if payload power is not raised, SOM EPS confirmation is absent, or EPS is no longer nominal.
Completion / hold criterion	A successful CAM000 command advances directly to the verification step. The displayed caution about thermal rise remains active operationally.

Step 19 - Verify camera readiness

Assigned role	SOM -> SOE1
Entry condition	CAM000 configuration command succeeded.
Required action	SOM asks SOE1 to open Payload and verify CAM000. SOE1 reports the camera state. SOM presses Verify Camera.
Expected indication	CAM000 shall read READY. Other payload readiness telemetry should remain consistent with an active, configured payload.
Completion / hold criterion	Do not proceed to imaging if CAM000 is STANDBY or if the procedure has automatically returned to EPS mitigation.

Step 20 - Acquire Frankfurt Airport image

Assigned role	SOM -> SPACON
Entry condition	Camera is verified READY, EPS remains acceptable, GS1 signal is GOOD, and mission time is within T+15:00 to T+16:00.
Required action	At the imaging window, SOM presses Execute Frankfurt image acquisition. SPACON selects IMG901 - Take Image, ARM, GO.
Expected indication	A command executed between T+15:00 and T+16:00 produces imageValidity = VALID TARGET IMAGE and a SUCCESS - TARGET CAPTURED command result. The Captured Image panel is populated with image, capture time and location metadata.
Completion / hold criterion	Do not intentionally execute before T+15:00 or after T+16:00. Earlier/later image states exist for training diagnostics but do not satisfy the formal procedure objective.

Operational note. The top-bar imaging indication is green during the first half of the one-minute window and red during the second half. The red indication represents time urgency; both halves of the formal T+15:00 to T+16:00 window are mapped to VALID TARGET IMAGE by the Scenario 1 image-selection logic.

Step 21 - Enter spacecraft standby

Assigned role	SOM -> SPACON
Entry condition	Image capture has completed successfully.
Required action	SOM presses Command spacecraft to standby. SPACON selects STB901 - Spacecraft Standby Mode, ARM, GO.
Expected indication	The spacecraft standby state becomes active, payload power is set to 8 W, payloadPowerRaised is cleared and camera configuration returns to inactive. The end-of-simulation sequence starts.
Completion / hold criterion	Standby is rejected until imageTaken is true. After success, no additional operator command is required for Scenario 1.

5.2.7 Scenario 1 Completion State

A correctly completed Scenario 1 ends with a valid Frankfurt target image recorded, spacecraft standby active, payload power returned to a low level, and the end sequence running. The procedure progression internally moves beyond the final commanded row to the Mission Complete / Review state. If the image window was missed or the GS1 link has reached LOS before image acquisition, the preferred training action is to terminate/reset and repeat the scenario rather than to treat a late image as nominal completion.

5.3 Scenario 2 - Communications and Thermal Contingency

5.3.1 Mission Objective and Scenario Logic

Scenario 2 begins with the same GS1 acquisition, link-validation and memory-preparation logic as Scenario 1, but the initial EPS battery state contains a deliberate critical imbalance. The normal pre-contingency procedure is therefore interrupted at Step 12 before its displayed calibration/test-imaging sequence can be completed. The SOM must recognise the battery emergency, contact the simulated GNC authority, import a contingency procedure, recover battery and power state, manage a controlled

GS1 link loss, acquire GS2, recover spacecraft telemetry, mitigate the elevated EPS temperature, restore the payload and execute a later Frankfurt Airport image.

Scenario 2 - Communications and Thermal Contingency

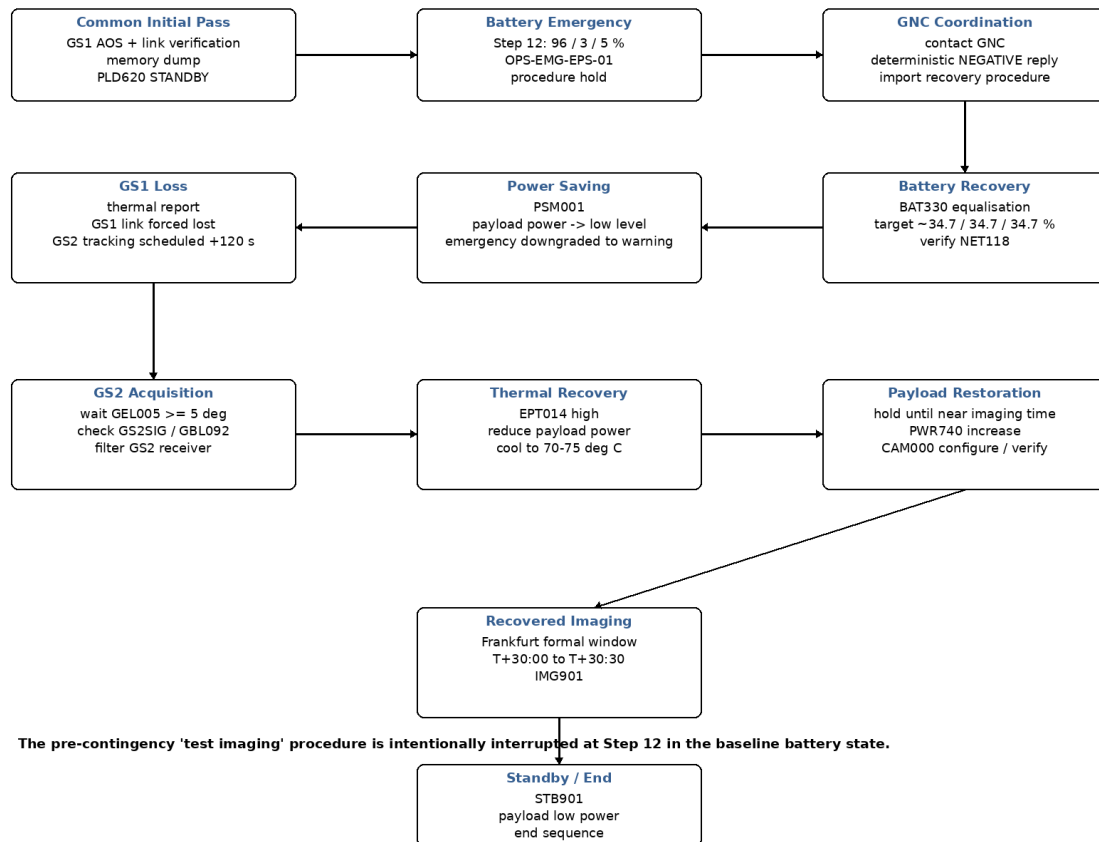


Figure 5-2 - Scenario 2 operational flow and contingency branches.

5.3.2 Initial Conditions and Time-Dependent Events

Table 5-4 - Scenario 2 baseline conditions

Item	Baseline / event	Operational significance
Battery A - BCH096	96%	High donor battery in the training equalisation model.
Battery B - BCH097	3%	Below the <10% critical threshold.
Battery C - BCH098	5%	Below the <10% critical threshold.
Emergency trigger	Any battery <20%; critical display when any battery <10%	Baseline Step 12 enters OPS-EMG-EPS-01.
Initial procedure title	Payload Calibration / Test Imaging	The baseline cannot reach its imaging Step 20 because the emergency holds at Step 12.
GS1 AOS	T+01:00	Common initial acquisition sequence.
GS1 loss	Event driven at recovery Step 18	Not tied to a fixed absolute mission time.
GS2 tracking start	120 s after Step 18 forces GS1 loss	Implemented timer; the live countdown is authoritative.
GS2 TM lock	GS2 elevation >= 5 deg while GS1 is lost	Spacecraft telemetry becomes observable again.
Recovered Frankfurt window	T+30:00 to T+30:30	Formal Scenario 2 target-imaging window.
Top-bar recovered LOS reference	T+31:00	Scenario end/time reference; standby should be completed before this point.

5.3.3 Phase A - Common Initial Pass (Steps 1-11)

Steps 1 through 11 are operated using the same command logic and acceptance thresholds described for Scenario 1, with the role allocation shown in the Scenario 2 procedure table. Execute the sequence in full; do not skip directly to the battery emergency.

Table 5-5 - Scenario 2 Steps 1-11

Step	Assigned role	Required operation	Acceptance / expected result
1	SOM	Start simulation; wait for GS1 AOS.	AOS at T+01:00; Step becomes DONE.
2	SOM -> SOE1	Report GS1 GEL005.	Numerical elevation reported.
3	SOM	Confirm GS1 acquisition geometry.	GEL005 >= 5.0 deg.
4	SOM -> SOE2	Report GSE001 and GBL092.	Baseline unfiltered link requires filtering.
5	SOM -> SPACON	GSE001 - Filter Signal; ARM; GO.	GS1 signal filtered successfully.
6	SOM -> SOE1	Verify post-filter signal.	Active signal quality GOOD.
7	SOM -> SOE2	Report C&DH MEM221 before dump.	Baseline about 87%; dump required.
8	SOM	Authorise memory dump.	MEM221 > 50%.
9	SOM -> SPACON	MEM221 - Dump Memory; ARM; GO.	Memory reaches 5%; dump complete.
10	SOM -> SOE1	Report/verify MEM221 after dump.	MEM221 <= 10%.
11	SOM -> SOE2	Report Payload PLD620 Instrument Mode.	PLD620 = STANDBY.

5.3.4 Phase B - Battery Emergency Detection and Procedure Hold (Step 12)

Step 12 - Evaluate battery state of charge

Assigned role	SOM -> SOE1
Entry condition	Steps 1-11 are complete and spacecraft telemetry is available through GS1.
Required action	SOM asks SOE1 to open EPS and report BCH096, BCH097 and BCH098. SOE1 reports all three SOC values. SOM presses Evaluate battery state-of-charge.
Expected indication	The baseline values are approximately BCH096 = 96%, BCH097 = 3% and BCH098 = 5%. Because at least one value is below 20%, the simulator declares an emergency; because B and C are below 10%, the banner is in the critical EMERGENCY SITUATION state.
Completion / hold criterion	The normal procedure is held at Step 12. Do not attempt the visible pre-contingency Steps 13-20. Recovery requires the emergency communication workflow and import of the new procedure.

Operational note. Before contingency import, the SOM procedure title is 'Payload Calibration / Test Imaging' and rows for a nominal continuation may remain visible. In the baseline battery state those rows are intentionally unreachable because current procedure state is held at Step 12. The emergency path is the only valid continuation.

5.3.5 Phase C - Emergency Communication with GNC

The emergency overlay identifies event OPS-EMG-EPS-01 / EPS BATTERY DISCHARGE CONTINGENCY. The required workflow is deterministic and must be completed by the SOM.

Table 5-6 - Emergency communication workflow

Action	SOM operation	Expected result
Open emergency contact list	Press OPEN CONTACT LIST.	External subsystem contacts are displayed.
Select authority	Select GNC - GNC Console / Attitude Dynamics.	The prepared message view opens; GNC is the priority contact that advances the scenario.
Review prepared message	Wait for the typewriter message to finish.	Message requests approval for an attitude change intended to improve solar-array charging attitude.

Action	SOM operation	Expected result
Send message	Press SEND MESSAGE.	Status changes to WAITING FOR RESPONSE.
Wait for GNC	No additional action for approximately 10 s.	GNC response is deterministic NEGATIVE; attitude change is not approved.
Accept recommendation	Read the GNC response modal.	The simulator recommends importing the contingency procedure and continuing with battery recovery.

No physical attitude manoeuvre is executed during this interaction. The GNC response is a scenario decision point, not an AOCS simulation result. The software does not model an engineering reason for the rejection; the negative response is fixed to drive the intended recovery branch.

5.3.6 Phase D - Importing the Contingency Procedure

After the GNC response, the SOM presses IMPORT NEW PROCEDURE. The interface displays DOWNLOADING / IMPORTING and, after approximately 1.8 s in the current implementation, replaces the pre-contingency rows with the Scenario 2 recovery procedure beginning at Step 13. The import is a local simulator state change; it does not retrieve a procedure from an external network.

At import, the simulator retains the evaluated battery condition, resets the recovery-step states and sets EPT014 to 90.2 deg C as the initial thermal condition for the contingency continuation. The next valid procedure action is Battery Equalization Transfer.

5.3.7 Phase E - Battery Recovery and Low-Power Configuration (Steps 13-17)

Step 13 - Execute battery equalisation transfer

Assigned role	SOM -> SPACON
Entry condition	Contingency procedure imported; Step 13 is CURRENT.
Required action	SOM presses Battery load redistribution. SPACON selects BAT330 - Battery Equalization Transfer, ARM, GO.
Expected indication	Battery A decreases while Batteries B and C increase until all three reach 34.7%. The command state is IN PROGRESS during redistribution.
Completion / hold criterion	Wait for BAT330 COMPLETE / DONE. From the 96/3/5% baseline, the current update rates require roughly 36 simulation seconds, despite the '~20 s' text shown in the procedure. Status completion, not elapsed time, is authoritative.

Step 14 - Verify recovered battery values

Assigned role	SOM -> SOE2
Entry condition	BAT330 is complete.
Required action	SOM asks SOE2 to report BCH096, BCH097 and BCH098 again. Press Verify battery recovery status after the report.
Expected indication	All three values should be approximately 34.7% at completion. A slow 0.01 percentage-point-per-second discharge begins after equalisation.
Completion / hold criterion	If any battery remains below the emergency threshold, do not continue. The baseline equalisation completes above 20% but below the >50% nominal definition, so the expected status is WARNING rather than fully nominal.

Step 15 - Verify power margin before low-power mode

Assigned role	SOM -> SOE1
Entry condition	Battery recovery has been reported.
Required action	SOM asks SOE1 to report NET118 and, for situational awareness, PWR740. Press Ask Power.
Expected indication	NET118 should be ≥ 900 W; the baseline value before power-saving mode is approximately 1160 W.
Completion / hold criterion	Proceed to power-saving mode only when the power-margin criterion is met.

Step 16 - Enter power-saving mode

Assigned role	SOM -> SPACON
Entry condition	Step 15 power margin is acceptable.
Required action	SOM presses Enter power-saving configuration. SPACON selects PSM001 - Enter Power Saving Mode, ARM, GO.
Expected indication	The simulator enters a timed transition. After 10 simulation seconds, Power Saving Mode becomes active and PWR740 is set to 8 W. NET118 becomes approximately 1140 W.
Completion / hold criterion	Wait for DONE / POWER SAVING MODE ACTIVE. The procedure's '~15 s' is an approximate cue; the implemented transition is 10 s.

Step 17 - Verify low-power recovery and initiate GS1 degradation

Assigned role	SOM -> SOE2
Entry condition	Power Saving Mode is active.
Required action	SOM asks SOE2 to report NET118 and BCH096/BCH097/BCH098, then presses Confirm low-power configuration active.
Expected indication	The acceptance guard requires Power Saving Mode active, NET118 $\geq$ 900 W and no battery below 20%. On success, the emergency is downgraded to WARNING and the simulator begins controlled GS1 degradation from the current pass geometry.
Completion / hold criterion	After successful confirmation, expect a weak-signal warning. The next step intentionally progresses into thermal assessment and complete GS1 loss.

5.3.8 Phase F - Thermal Assessment and GS1 Loss (Steps 18-19)

Step 18 - Report thermal state and trigger GS1 loss

Assigned role	SOM -> SOE1
Entry condition	Step 17 has confirmed low-power recovery and GS1 degradation is active.
Required action	SOM asks SOE1 to report EPT014 and NET118 from EPS. Press Request thermal telemetry.
Expected indication	The recovery branch presents an elevated EPS condition; when required, the action enforces EPT014 = 90.2 deg C before reporting. The same action ends the weak-signal phase, sets GS1 connection inactive and schedules GS2 tracking 120 s later.
Completion / hold criterion	After this action, spacecraft subsystem telemetry becomes unavailable until GS2 reaches the telemetry-lock geometry. The operator shall recognise this as loss of observability, not as proof of onboard shutdown.

Step 19 - Confirm GS1 loss of signal

Assigned role	SOM -> SOE2
Entry condition	GS1 connection has been set inactive by Step 18.
Required action	SOM asks SOE2 to verify GS1 signal state using GSE001 / related link indicators, then presses Confirm GS1 loss of signal.
Expected indication	GS1 spacecraft-link parameters show loss/no signal and the subsystem TM histories are cleared. The top bar shows GS1 LINK LOST / GS2 AOS countdown.
Completion / hold criterion	The implemented GS2 tracking delay is 120 s from the Step 18 loss event. One legacy Last Result string states '60 seconds'; disregard that text and use the live countdown and GS2 elevation as the controlling indications.

5.3.9 Phase G - GS2 Acquisition and Telemetry Recovery (Steps 20-22)

GS2 tracking begins 120 s after the Step 18 GS1-loss event. The synthetic GS2 elevation then rises from the horizon according to the Scenario 2 pass profile. Spacecraft telemetry lock becomes true automatically when GS1 is lost and GS2 elevation reaches at least 5.0 deg. With the current geometry this occurs roughly 19 simulation seconds after GS2 tracking begins. The exact operator action is nevertheless controlled by the displayed GS2 GEL005 value.

Step 20 - Confirm GS2 elevation

Assigned role	SOM -> SOE1
Entry condition	GS1 is lost and the GS2 pass has begun.
Required action	SOE1 monitors the GS2 column of the Ground Station panel. Wait until GS2 GEL005 is at least 5.0 deg. Only then can SOM press Wait for GS2 acquisition window.
Expected indication	At >= 5 deg the GS2 telemetry-lock condition is satisfied and spacecraft-subsystem telemetry becomes visible again.
Completion / hold criterion	The button is disabled below 5 deg. Do not use GS1 geometry after GS2 tracking begins; the application intentionally removes GS1 spacecraft geometry from display.

Step 21 - Assess unfiltered GS2 downlink quality

Assigned role	SOM -> SOE1
Entry condition	GS2 elevation has been confirmed.
Required action	SOM asks SOE1 to report GS2SIG and GBL092 from the GS2 Ground Station telemetry. Press Verify GS2 downlink acquisition.
Expected indication	Before filtering, the baseline GS2 quality is MEDIUM once above the mask; therefore filtering is required. The report status explicitly indicates FILTER REQUIRED BY SPACON.
Completion / hold criterion	Proceed to Step 22 with the SOM filter request; do not treat elevation alone as proof of a GOOD data link.

Step 22 - Filter the GS2 signal

Assigned role	SOM -> SPACON
Entry condition	GS2 elevation confirmed and GS2 signal quality checked.
Required action	SOM presses Configure GS2 receiver chain. SPACON selects GSE001 - Filter Signal, ARM, GO. Although GSE001 is the command code, the command logic applies to GS2 because the current Scenario 2 procedure step is 22.
Expected indication	The result is SUCCESS - GS2 SIGNAL FILTERED. GS2 signal quality becomes GOOD and subsequent operational commands use GS2 as the active link.
Completion / hold criterion	There is no separate post-filter verification row in the current procedure. Successful completion of Step 22 is the transition to thermal recovery.

5.3.10 Phase H - EPS Thermal Recovery after GS2 Reacquisition (Steps 23-26)

Step 23 - Request thermal mitigation

Assigned role	SOM
Entry condition	GS2 signal has been filtered successfully and spacecraft telemetry is available.
Required action	Compare the recovered EPT014 value with the nominal 70-75 deg C criterion. Because the baseline condition is high, press Request thermal Mitigation.
Expected indication	The mitigation request resets downstream payload/camera/image readiness and releases the EPT014 reduction command.
Completion / hold criterion	Do not restore payload power until the mitigation sequence through Step 26 is complete.

Step 24 - Reduce payload power / establish safe thermal state

Assigned role	SOM -> SPACON
Entry condition	Thermal mitigation requested.
Required action	SOM presses Perform payload load shedding. SPACON selects EPT014 - Reduce Payload Power, ARM, GO.
Expected indication	Power Saving Mode has already placed PWR740 at 8 W. The reduction command therefore recognises that payload power is already at or below the 10 W safe level, normalises the safe level to 10 W and marks reduction complete while thermal cooling remains active.
Completion / hold criterion	A near-immediate SUCCESS - PAYLOAD POWER ALREADY AT SAFE LEVEL is normal in the baseline Scenario 2 branch; it is not evidence that the command was skipped.

Step 25 - Monitor post-mitigation temperature

Assigned role	SOM -> SOE2
Entry condition	Step 24 is complete and thermal cooling is active.
Required action	SOM asks SOE2 to report EPT014 from EPS and monitor the cooling trend. Press Request post-mitigation thermal telemetry after a value has been reported.
Expected indication	EPT014 decreases at approximately 0.28 deg C per simulation second while above 72.5 deg C, then settles in a narrow band near the lower portion of the 70-75 deg C nominal range.
Completion / hold criterion	Do not confirm EPS nominal merely because the temperature has started to fall; wait until the Step 26 guard is satisfied.

Step 26 - Confirm EPS nominal after recovery

Assigned role	SOM
Entry condition	Post-mitigation telemetry has been reported.
Required action	Observe EPT014 until it is between 70 and 75 deg C and stable enough to support the next payload activity. Press Confirm EPS and thermal parameters within limits.
Expected indication	The software acceptance guard requires EPT014 within 70-75 deg C, NET118 >= 900 W and the internal scenario battery reference >= 75%. With cooling active, the baseline settles around 72.2-72.5 deg C.
Completion / hold criterion	If confirmation fails, continue monitoring. Do not repeatedly press the confirmation button while EPT014 is above 75 deg C.

5.3.11 Phase I - Payload Restoration and Camera Readiness (Steps 27-30)

CAUTION. After PWR740 is restored, Scenario 2 EPT014 rises at 0.005 deg C per simulation second in the current thermal model. If EPT014 later exceeds 75 deg C while the SOM nominal-confirmation flag is active, the application invalidates payload restoration and returns the procedure to the mitigation sequence. Accordingly, do not execute Step 27 immediately after Step 26 if the mission is still far from T+30:00.

A robust training sequence is to hold at Step 27 until approximately T+29:00, then perform Steps 27-30 without unnecessary delay. This provides adequate time for the ~10 s payload-power transition and camera verification while keeping thermal exposure short before the formal T+30:00 imaging-window opening.

Step 27 - Restore payload imaging power

Assigned role	SOM -> SPACON
Entry condition	EPS is confirmed nominal and mission time is sufficiently close to the T+30:00 imaging window.
Required action	SOM presses Restore payload power for imaging window. SPACON selects PWR740 - Increase Payload Power, ARM, GO.
Expected indication	PWR740 rises from the 10 W safe level to 180 W at 17 W per simulation second; completion requires about 10 simulation seconds. Scenario 2 thermal rise then continues at 0.005 deg C/s.
Completion / hold criterion	Wait for DONE. If EPT014 exceeds 75 deg C and the simulator returns the procedure to mitigation, repeat Steps 23-27 only after restoring nominal thermal state.

Step 28 - Configure imaging payload

Assigned role	SOM -> SPACON
Entry condition	PWR740 increase is complete, EPS nominal confirmation is still valid and GS2 signal is GOOD.
Required action	SOM presses Configure imaging payload. SPACON selects CAM000 - Configure Camera, ARM, GO.
Expected indication	CAM000 becomes READY. The command is rejected if payload power is not raised, EPS confirmation is missing or EPS is no longer nominal.
Completion / hold criterion	Proceed to SOE verification only on successful TC History result.

Step 29 - Verify camera and payload active state

Assigned role	SOM -> SOE1
Entry condition	Camera configuration succeeded.
Required action	SOM asks SOE1 to read PLD620 Instrument Mode and CAM201 Camera Head Temperature. Press Verify Camera after the report.
Expected indication	PLD620 should be ACTIVE and CAM201 should remain within its nominal displayed range. The simulator additionally requires payload power raised and EPS nominal.
Completion / hold criterion	If the camera cannot be verified, inspect EPS state first; thermal invalidation can automatically clear the camera configuration.

Step 30 - Report post-restoration thermal trend

Assigned role	SOM -> SOE2
Entry condition	Camera is verified READY.
Required action	SOM asks SOE2 to report EPT014 and DCC208 after payload-power restoration. Press Report post-restoration thermal trend.
Expected indication	A slow rise is expected. The baseline Scenario 2 thermal-rate constant is 0.005 deg C/s even though the procedure text rounds the training cue to approximately +0.01 deg C/s.
Completion / hold criterion	This is a reporting step, not a second nominal confirmation. Continue to monitor EPT014 while waiting for the imaging window.

5.3.12 Phase J - Recovered Frankfurt Imaging and Standby (Steps 31-32)

Step 31 - Acquire Frankfurt Airport image on the recovered mission timeline

Assigned role	SOM -> SPACON
Entry condition	Camera is verified, post-restoration thermal trend has been reported, GS2 signal remains GOOD, and mission time is inside the formal T+30:00 to T+30:30 window.
Required action	At or after T+30:00, SOM presses Execute image acquisition during imaging window. SPACON selects IMG901 - Take Image, ARM, GO.
Expected indication	During T+30:00 to T+30:30 the image-selection logic returns SUCCESS - TARGET CAPTURED with imageValidity = VALID TARGET IMAGE. The dedicated Captured Image panel is populated.
Completion / hold criterion	For procedure compliance, treat T+30:30 as the end of the authorised Scenario 2 imaging window. Do not intentionally wait beyond it.

Operational note. The image-selection helper currently continues to classify the T+30:30 to T+31:00 ground-track segment as VALID TARGET IMAGE, while the formal SOM procedure criterion is T+30:00 to T+30:30. This manual adopts the stricter procedure-table interval as the operational rule so that there is no ambiguity in training.

Step 32 - Enter spacecraft standby and terminate the scenario

Assigned role	SOM -> SPACON
Entry condition	IMG901 has completed and imageTaken is true.
Required action	SOM presses Configure spacecraft standby. SPACON selects STB901 - Spacecraft Standby Mode, ARM, GO.
Expected indication	Spacecraft standby becomes active, PWR740 is set to 8 W, payloadPowerRaised is cleared, CAM000 returns inactive and the end sequence begins.
Completion / hold criterion	Standby is the final commanded procedure action. No additional recovery command is required after successful STB901 execution.

5.3.13 Scenario 2 Completion State

A correctly completed Scenario 2 ends with battery values recovered from the initial critical imbalance, the emergency downgraded during recovery, GS1 unavailable, GS2 established as the active telemetry path, EPS returned to a nominal thermal condition, payload power restored only for the required imaging period, a valid Frankfurt target image acquired during the formal recovered imaging window, and spacecraft standby active. The scenario demonstrates restoration of operational capability under degraded observability rather than a physically validated spacecraft emergency response.

5.4 Time-Critical Decision Aids

Table 5-7 - Fixed and event-driven time references

Reference	Scenario 1	Scenario 2	Operator use
Simulation start	T+00:00	T+00:00	SOM starts once all clients are ready.
GS1 AOS	T+01:00	T+01:00	Begin GS1 geometry procedure after automatic AOS.
Initial/test imaging label	Frankfurt T+15:00 to T+16:00	Test T+15:00 to T+16:00 before import	Scenario 2 baseline emergency prevents reaching its pre-import test image.
GS1 loss	T+17:00 nominal LOS	Triggered by Step 18 after recovery import	Scenario 2 loss is operator-event driven.
GS2 tracking start	N/A	Step 18 time + 120 s	Use live countdown.
GS2 TM availability	N/A	When GS2 GEL005 >= 5 deg	Usually about 19 s after GS2 tracking begins in the current profile.
Recovered imaging	N/A	Formal T+30:00 to T+30:30	Hold payload restoration until sufficiently near this window.
Recovered LOS/end reference	N/A	T+31:00 top-bar reference	Complete imaging and standby before this reference.
Next AOS indication	Current LOS + 30 min	Recovered LOS reference + 30 min	Informational; not used by implemented procedures.

5.5 Command Rejection and Immediate Operator Response

Table 5-8 - Common command/procedure rejection messages

Message / condition	Meaning	Required response
FAILED - WRONG PROCEDURE STEP	The action is not valid for the current procedure state.	Return to the SOM procedure table; execute the CURRENT row only.
FAILED - COMMAND NOT ARMED	SPACON pressed GO without a valid ARM state.	Re-select if necessary, press ARM, then GO.
FAILED - ... NOT REQUESTED BY SOM	SPACON attempted an operational command before the SOM released it.	SOM must press the current procedure action first.
FAILED - SIGNAL NOT GOOD	Nominal active link is degraded.	Complete the link-verification/filtering procedure before retrying.
FAILED - GS2 SIGNAL NOT GOOD	Scenario 2 is using GS2 but its link is not GOOD.	Complete GS2 signal filtering and verify current procedure state.
FAILED - ELEVATION BELOW 5 deg	The ground-station operational mask is not met.	Wait for the displayed GEL005 to reach ≥ 5 deg.
FAILED - EPS NOT NOMINAL / WAIT OR MITIGATE	EPS acceptance guard is not met.	Continue cooling/monitoring; do not proceed to payload operations.
FAILED - PAYLOAD POWER NOT RAISED	Camera configuration requested before imaging power is available.	Complete PWR740 increase first.
FAILED - CAMERA NOT VERIFIED BY SOM	Image command attempted without completed camera verification.	Complete the applicable camera-verification step.
FAILED - IMAGE CAPTURE NOT COMPLETE	Standby commanded before imageTaken is true.	Complete the required IMG901 acquisition first.

5.6 Operator Completion Checks

5.6.1 Scenario 1 Completion Check

Before accepting Scenario 1 as complete, verify that the following conditions have occurred in order: GS1 AOS and elevation acceptance; filtered GOOD link; MEM221 reduced to $\leq 10\%$; PLD620 initially confirmed STANDBY; EPS thermal mitigation completed; EPT014 confirmed in the nominal range; PWR740 restored close to the imaging window; CAM000 verified READY; IMG901 captured a VALID TARGET IMAGE between T+15:00 and T+16:00; and STB901 entered spacecraft standby.

5.6.2 Scenario 2 Completion Check

Before accepting Scenario 2 as complete, verify that the following conditions have occurred in order: common GS1 acquisition and memory preparation; Step 12 detection of the 96/3/5% battery emergency; GNC contact and negative response; contingency procedure import; BAT330 completion near 34.7% on all three batteries; PSM001 active; battery emergency downgraded to WARNING; event-driven GS1 loss; GS2 elevation accepted at ≥ 5 deg; GS2 signal filtered GOOD; EPS cooled to and confirmed within 70-75 deg C; PWR740 restored only near the recovered imaging window; camera verified; post-power thermal trend reported; IMG901 completed inside T+30:00 to T+30:30; and STB901 entered spacecraft standby.

If any required item is missing, the exercise should be treated as incomplete rather than reconstructed after the fact. Use the reset procedure described in Section 3.11 to establish a clean state before a formal repeat.

6 Troubleshooting, Fault Isolation and Recovery

This section provides the reference troubleshooting procedure for MCS26_V26_01. Fault isolation is performed from the infrastructure outward: software baseline, host processes, TCP reachability, WebSocket synchronisation, browser state, scenario state and finally procedure logic. This order is important because several faults produce similar visual symptoms. For example, a browser can display the user interface correctly even when the shared-state server is unreachable; similarly, NO TELEMETRY can be a correct mission condition rather than a software failure.

6.1 Troubleshooting Philosophy

6.1.1 Separate Infrastructure Faults from Simulated Mission Conditions

The first diagnostic decision is whether the observed condition belongs to the real software/network infrastructure or to the simulated mission. An infrastructure fault prevents the application from operating as designed. Examples are an inactive host process, an occupied TCP port, a blocked firewall rule, a failed WebSocket connection, a stale frontend, or an incorrect role URL. A simulated mission condition is an intended state inside the scenario, such as pre-AOS NO TELEMETRY, a degraded GS1 signal, an EPS thermal warning, or Scenario 2 telemetry loss before GS2 reacquisition.

Do not reset the simulator simply because a value is red, a command is rejected, or spacecraft telemetry disappears. First verify whether the procedure predicts that condition. Premature reset removes diagnostic evidence and can conceal a valid training event.

6.1.2 Use the Least-Disruptive Diagnostic Action First

Troubleshooting should progress from non-destructive observation to increasingly disruptive recovery. Begin by reading the displayed status and Last Result, checking the browser URL, examining the host terminals and testing network reachability. A browser hard reload, service restart, dependency reinstallation or repository replacement should be performed only when the preceding level demonstrates that it is necessary.

Table 6-1 - Troubleshooting escalation levels

Level	Action	Effect on current exercise	Use when
A - Observe	Read procedure status, TC History, host terminal output, browser Console/Network and current URLs.	No state loss.	The system is still responsive and the failure mechanism is unknown.
B - Local client recovery	Correct role URL, re-open a panel, or reload one non-SOM operator client.	Usually recoverable if the SOM timer and host server remain active.	One operator console is stale or disconnected while the rest remain healthy.
C - Browser cache recovery	Hard reload or Empty Cache and Hard Reload.	Local client is reinitialised; active exercise may be disrupted.	The workstation appears to display an old frontend after a software update.
D - Host service restart	Stop and restart WebSocket and Vite services.	Shared in-memory state is cleared; exercise must restart.	Synchronisation state is inconsistent, SOM timer was lost, or a clean baseline is required.
E - Software baseline repair	Git verification/update, npm install/npm ci, build test, cache verification.	Host unavailable until repair completes.	Startup/build/dependency faults are present.
F - Re-clone / rebuild host	Create a fresh repository working tree and reinstall dependencies.	Full local installation replacement.	The local repository or dependency tree cannot be trusted or recovered efficiently.

6.1.3 Preserve Diagnostic Evidence Before a Full Reset

When a reproducible defect is suspected, record the software revision, mission state and infrastructure state before restarting the host. The minimum useful evidence is the Git commit, Node.js and npm versions, host IPv4 address, listening TCP ports, browser role URL, selected scenario, mission time,

Last Result, relevant TC History entry, and the browser/server error message. A screenshot is useful only when it includes the information required to identify the state.

```
git log -1 --online
git status
node --version
npm --version
ipconfig
Get-NetTCPConnection -LocalPort 5173 -State Listen
Get-NetTCPConnection -LocalPort 3001 -State Listen
```

6.2 Rapid Fault-Isolation Matrix

Table 6-2 - First diagnostic action by symptom

Observed symptom	Most likely fault class	First diagnostic action	Immediate interpretation
Host script does not start either service.	Runtime / path / dependency	Run <code>node --version</code> , <code>npm --version</code> and open a terminal in the project root.	If <code>node/npm</code> are unavailable, this is not a browser or network problem.
Frontend opens on host but not on operator PC.	Host IP / firewall / LAN	Test-NetConnection HOST-IP -Port 5173 from the operator PC.	Failure means HTTP frontend reachability is not established.
Frontend opens on all PCs but mission time/procedure do not synchronise.	MCS WebSocket path	Test-NetConnection HOST-IP -Port 3001 and inspect browser Console.	Port 5173 can work while the shared-state path on 3001 is unavailable.
Vite terminal reports 5174 or another port.	Port 5173 occupied	Inspect <code>Get-NetTCPConnection -LocalPort 5173 -State Listen</code> .	Current role scripts still point to 5173; baseline startup is not valid.
WebSocket terminal reports EADDRINUSE on 3001.	Port 3001 occupied	Identify <code>OwningProcess</code> for TCP 3001.	Another server instance or process already owns the required port.
One console shows an older UI or older telemetry layout.	Browser cache / stale client	Compare Git revision on host, then hard reload the affected browser.	Do not change application code until stale resources are excluded.
SOM console accidentally opens as SOE.	Invalid/missing role query	Inspect the complete browser URL.	The application defaults to SOE for an absent or invalid role.
All spacecraft subsystem panels show NO TELEMETRY before AOS.	Expected mission condition	Check mission time and GS acquisition state.	This is correct before a valid spacecraft telemetry path exists.
All spacecraft panels show NO TELEMETRY after confirmed GS2 lock.	State/sync fault or wrong scenario	Verify same scenario locally, GS2 GEL005 >= 5 deg, port 3001 and browser Console.	Post-lock persistent blackout is not expected.
SPACON command returns FAILED.	Procedure/command precondition	Read the complete failure string and current SOM procedure row.	The message identifies the missing gate; do not repeat GO blindly.
Mission time freezes while status remains RUNNING.	SOM timer/client loss	Check whether the SOM page was refreshed, closed or crashed.	Current mission timer is generated by the SOM browser; clean restart is normally required.
Intro screen never leaves the video stage.	Media asset / browser playback	Open DevTools Network and check <code>/videos/welcome-intro.mp4</code> .	The application advances on the video's ended event; failed media can block scenario entry.
Valid image metadata appears but image itself is blank/broken.	Image asset path / cache	Check Network request for <code>/photo/<image-name>.jpg</code> .	Payload state can be valid while the static image file fails to load.

6.3 Host Startup and Runtime Failures

6.3.1 start-host-dev.bat Does Not Start Correctly

The supplied host script changes directory to the project root, opens a new command window for npm run start:server, and runs npm run start:frontend in the original window. If one or both windows close immediately, reproduce the failing command manually from the project root so that the error remains visible.

```
cd <path-to-MCS26_V26_01>
npm run start:server
```

Open a second terminal and run:

```
cd <path-to-MCS26_V26_01>
npm run start:frontend
```

This separates a batch-script/path problem from a Node.js/Vite problem. If both manual commands work, the project runtime is healthy and the remaining fault is local to script invocation or working-directory handling.

6.3.2 'node' or 'npm' Is Not Recognised

This condition means that the Windows shell cannot resolve the Node.js/npm executable through its current environment. Verify the commands directly. If Node.js was installed while the terminal was already open, close that terminal and open a new one before changing system settings.

```
node --version
npm --version
```

Do not attempt npm cache cleaning, repository replacement or firewall changes when the executable itself is not available. Restore the runtime prerequisite first.

6.3.3 Missing Package or Module Errors

Errors such as 'Cannot find package', 'Cannot find module', or missing Vite/Vue/Socket.IO packages normally indicate that node_modules is absent, incomplete or inconsistent with the current source baseline. From the project root run:

```
npm install
npm run build
```

If a clean lock-file-based installation is required, stop the simulator and use npm ci. npm ci removes the existing node_modules directory and recreates it from the lock file; it also refuses to continue when package.json and package-lock.json are inconsistent [20].

```
npm ci
npm run build
```

6.3.4 npm Cache Problems

Do not use npm cache clean --force as a routine first response. npm documents its cache as content-addressed and self-healing and provides npm cache verify to check integrity and remove unneeded cache data [48]. Use cache verification only after ordinary dependency installation has failed in a manner consistent with package-fetch or cache integrity problems.

```
npm cache verify
```

If npm install requires the public registry, Internet access must be available at maintenance time or an approved local/internal package source must provide the required packages. This requirement does not apply to normal simulator execution after the dependencies are installed.

6.3.5 Build Failure

A failed npm run build indicates that the source/dependency baseline is not internally acceptable for training even if the development server can display a partial page. Record the first meaningful

TypeScript, Vue or bundling error rather than the final summary line. Confirm the Git revision and working-tree state, then reinstall dependencies if appropriate.

```
git status
git log -1 --oneline
npm ci
npm run build
```

CAUTION. Do not resolve a build failure by editing random TypeScript types, deleting source code, or downgrading packages on the operational host. If the clean repository revision does not build with the documented runtime, treat it as a software-baseline defect.

6.3.6 Port 3001 Already in Use

server.js binds the shared-state service explicitly to TCP port 3001. If another process already owns the address, Node.js reports EADDRINUSE. Node.js defines EADDRINUSE as failure to bind because another server is already using the local address or port [44].

```
Get-NetTCPConnection -LocalPort 3001 -State Listen | Select-Object
LocalAddress,LocalPort,State,OwningProcess
```

Use the reported OwningProcess value to identify the process:

```
Get-Process -Id <PID>
```

If the process is an older MCS26 WebSocket server, close its terminal cleanly with Ctrl+C and restart the reference host script. If it belongs to another application, do not terminate it without confirming that doing so is permitted. MCS26 currently assumes port 3001 and the client code also connects explicitly to that port; changing only the server port is not a valid repair.

6.3.7 Frontend Starts on 5174 or Another Port

The Vite development server normally uses port 5173 but, when the port is occupied and strict-port mode is not enabled, Vite automatically tries the next available port [43]. The current MCS26 launcher URLs and documentation are fixed to 5173. Therefore a terminal message showing http://...:5174/ is a fault condition for the baseline deployment, not an alternative normal configuration.

```
Get-NetTCPConnection -LocalPort 5173 -State Listen | Select-Object
LocalAddress,LocalPort,State,OwningProcess
```

Identify and stop the conflicting process only after its ownership is known, then restart MCS26 and verify that Vite reports port 5173. Vite also documents --strictPort as a method of exiting instead of silently selecting another port [43]; the current repository does not enable that option by default.

6.3.8 Vite Dependency-Optimisation Cache Suspected

When the host source and node_modules are confirmed correct but the development server continues to serve an inconsistent dependency bundle, Vite provides a --force option that instructs the optimiser to ignore its existing cache and re-bundle dependencies [43]. Stop the frontend and run:

```
npm run start:frontend -- --force
```

Use this only for a host-side Vite optimisation problem. It is different from the operator browser cache described in Section 6.6.

6.4 Host IP, LAN and Firewall Problems

6.4.1 Selecting the Correct IPv4 Address

The browser and the MCS WebSocket client derive all remote communication from the host address entered in the role URL. The helper script show-host-ip.bat can display multiple IPv4 addresses, including physical Ethernet/Wi-Fi, VPN and virtual-machine interfaces. Select the address of the adapter that is actually connected to the operator-network segment.

```
ipconfig
```

If the host changes from Wi-Fi to Ethernet, reconnects to a different access point, or receives a new DHCP lease, its IPv4 address can change. Re-run show-host-ip.bat and update all operator URLs. A previously bookmarked address shall not be assumed to remain valid.

6.4.2 Test TCP Services, Not Only Ping

A successful ping is not sufficient proof that MCS26 can operate, and a failed ping does not necessarily prove that TCP is blocked because ICMP can be filtered independently. Test the two application ports from the affected operator workstation using the PowerShell TCP diagnostic described in Section 3 [21].

Test-NetConnection	HOST-IP	-Port	5173
Test-NetConnection	HOST-IP	-Port	3001

Both commands shall report TcpTestSucceeded = True. Port 5173 proves reachability of the frontend service; port 3001 proves reachability of the MCS26 shared-state service. Diagnose them independently.

6.4.3 Frontend Reachable Locally but Not from Other Computers

The project starts Vite with --host 0.0.0.0, which instructs Vite to listen on all addresses, including LAN interfaces [43]. If http://localhost:5173 works on the host but HOST-IP:5173 fails from an operator computer, first verify the selected host IP and Test-NetConnection result. The fault is then normally outside the Vue application: firewall policy, adapter selection, routing, network isolation or the host service itself.

6.4.4 Windows Firewall

The host requires inbound TCP access to 5173 and 3001 from the authorised training network. Microsoft documents inbound port and program/service rules for Windows Firewall and recommends limiting scope and ports rather than disabling the firewall globally [45]. Where local policy permits configuration, rules should be restricted to the appropriate network profile and, where practical, the training subnet.

For a managed DLR/TU workstation, firewall policy can be controlled by Group Policy. If the local user cannot create the required rule, the correct action is to involve the responsible administrator rather than disabling security controls or moving the simulator to an uncontrolled network.

6.4.5 Wireless Client Isolation or Segmented Networks

Some wireless or institutional network configurations intentionally prevent peer-to-peer connections between clients even though every workstation has Internet or gateway connectivity. The symptom is usually that the operator PC can reach external resources but Test-NetConnection to the host fails. Confirm that the host and operators are on the approved classroom LAN/VLAN and that client-to-client traffic is permitted. This condition cannot be corrected inside MCS26.

6.5 WebSocket Synchronisation Faults

6.5.1 Architecture Relevant to Troubleshooting

MCS26 uses two different browser-server relationships. Port 5173 serves the frontend. Port 3001 runs the Socket.IO shared-state server. The application constructs the MCS synchronisation URL from the hostname of the loaded page and port 3001. Therefore a client opened at http://192.168.1.20:5173/?role=SOM attempts to connect its MCS state channel to http://192.168.1.20:3001.

Both the client and server are configured to use the WebSocket transport explicitly. The standard Socket.IO library can support reconnection and can normally fall back to HTTP long-polling [49], but the current MCS26 configuration restricts transport to WebSocket. The classroom network and firewall must therefore permit that WebSocket connection to TCP 3001.

6.5.2 Expected Browser Console Messages

Open browser developer tools with F12 and select Console. A correctly loaded client produces application messages similar to:

```
APP      VUE      IS      RUNNING
OPERATOR ROLE:  SOM      #    or    SOE    /    SPACON
Connected      to      WebSocket      server:      <socket-id>
State received from server: <shared-state-object>
```

During state-changing actions or the SOM mission timer, the application also logs SYNC NOW SENT. A connection failure produces a WebSocket connection error message through the client's connect_error handler. Vite can also use WebSocket internally for development hot-module replacement; do not confuse a Vite/HMR message with the MCS26 Socket.IO connection on port 3001.

6.5.3 Expected WebSocket Server Messages

The server terminal provides an independent confirmation of client connectivity. A healthy server starts with:

```
WebSocket server running on port 3001
```

Each operator browser connection then creates a line similar to:

```
Client      connected:      <socket-id>
Client disconnected: <socket-id>
```

If the browser shows the frontend but the server terminal never reports a client connection, isolate port 3001 before investigating procedure logic. If the terminal reports ENGINE CONNECTION ERROR, record the error code, message and context before restarting.

6.5.4 Frontend Loads but Consoles Do Not Share Mission State

This symptom is characteristic of a working 5173 path and failed 3001 path. Confirm all of the following in order: (1) the WebSocket server terminal is still open; (2) port 3001 is listening on the host; (3) Test-NetConnection from the affected operator workstation succeeds; (4) the browser Console reports Connected to WebSocket server; and (5) the WebSocket server logs the corresponding client connection.

```
Get-NetTCPConnection      -LocalPort      3001      -State      Listen
Test-NetConnection HOST-IP -Port 3001
```

Do not change the procedure, telemetry code or scenario timing until this transport path is confirmed.

6.5.5 Shared State Persists When Browsers Are Closed

The server stores the current shared state in an in-memory object. Closing operator browser tabs does not clear that object. A new client connection receives the last shared server state immediately. This is useful for reconnecting an operator console, but it also means that closing every browser is not a clean reset of the exercise.

To guarantee an empty shared-state baseline, stop the WebSocket server and start it again as defined in Section 3.11. A server process restart recreates the shared-state object from empty initial state.

6.5.6 SOM Browser Refresh or Loss During an Active Run

CAUTION. The mission timer is generated by a setInterval running in the SOM browser after Start Simulation. The server stores and redistributes mission time but does not independently advance it. Therefore the SOM browser shall not be refreshed, closed or replaced during a running exercise.

If the SOM page is lost while simulationStatus remains RUNNING on the server, connected consoles can retain the last shared state but the mission time can stop advancing because the timer source has disappeared. A new SOM client can receive the RUNNING state, but Start Simulation does not create a second timer when simulationStatus is already RUNNING. The controlled recovery is to terminate the exercise, restart the host services, hard-refresh the clients and begin from a clean baseline.

6.5.7 Non-SOM Client Refresh During an Active Run

A non-SOM client can normally be recovered without stopping the mission provided that the SOM browser and host server remain healthy. Re-open the correct role URL, select the same scenario locally, complete the intro sequence, and wait for the next shared-state update before acting. Confirm that mission time and the current procedure consequences match the other consoles before resuming the role.

CAUTION. Do not issue an immediate SPACON command from a freshly reloaded client until its state has demonstrably converged with the running session. A local screen can transiently show default values while the shared state is being re-applied.

6.6 Browser Cache and Stale Frontend Recovery

6.6.1 Typical Cache Symptoms

Browser caching is one of the most common causes of apparent inconsistency after the project source has been updated. Typical symptoms include one operator workstation showing an older panel layout, an old telemetry list, a command name that differs from the host revision, an old JavaScript behaviour, or a static image/video that does not match the current repository while other consoles are correct.

6.6.2 Standard Hard Reload

After a host software update, first complete the clean host restart. Then hard-reload every operator browser before starting the next exercise. On Chromium-based browsers, Ctrl+Shift+R performs a hard reload that bypasses cached resources. The existing laboratory workflow may also use Ctrl+F5, which commonly forces a reload on Windows browsers. Chrome documents that a Hard Reload bypasses cache and that an Empty Cache and Hard Reload is available while DevTools is open [42].

6.6.3 Empty Cache and Hard Reload

If a normal hard reload does not remove the stale behaviour, open DevTools, keep it open, long-press the browser Reload control and choose Empty Cache and Hard Reload. Chrome states that this workflow clears the cache and then retrieves resources from the network rather than reusing the local cache [42].

An alternative diagnostic method is DevTools -> Network -> Disable cache, followed by reload. This setting is intended for development/testing and is active while DevTools remains open [42].

6.6.4 Distinguish Browser Cache from Server State

A cache problem affects the frontend files executed by one browser. A server-state problem affects the mission state distributed through port 3001. Hard reload does not clear the WebSocket server's shared state, and restarting the WebSocket server does not necessarily clear a browser's cached frontend. When both are suspect after a software update, use the full sequence: stop both host services, start them again, then hard-refresh every operator browser before selecting a scenario.

6.6.5 Do Not Hard-Reload the SOM During an Active Exercise

The cache-recovery procedure belongs to pre-exercise setup or controlled restart. During an active run, hard-reloading the SOM removes the local mission timer as described in Section 6.5.6. If a stale SOM frontend is discovered after Start Simulation, terminate the run and establish a clean baseline rather than attempting an in-place refresh.

6.7 Role URL and Local Scenario-State Problems

6.7.1 Wrong Operator Role

The application reads the role from the URL query parameter at page load. Valid values are SOM, SOE and SPACON. Any missing or unrecognised value defaults to SOE. Therefore a SOM or SPACON workstation opened with a bare URL can appear functional while having the wrong authority.

Table 6-3 - Correct role URLs

Console	Required URL pattern	Diagnostic consequence of wrong URL
SOM	http://HOST-IP:5173/?role=SOM	Start/decision authority is not available if the page defaulted to SOE.
SPACON	http://HOST-IP:5173/?role=SPACON	SELECT/ARM/GO authority is not available if the page defaulted to SOE.
SOE1	http://HOST-IP:5173/?role=SOE	Correct; SOE1 and SOE2 intentionally share the same technical role.
SOE2	http://HOST-IP:5173/?role=SOE	Correct; operational assignment distinguishes SOE1 from SOE2.

Confirm the Console log OPERATOR ROLE line after any role-related fault. The role is determined at page load; editing the URL text without reloading does not change the active role.

6.7.2 Scenario Selection Is Local

selectedScenario is intentionally not included in the shared-state object. Each browser selects Scenario 1 or Scenario 2 locally. Consequently, all participants must select the same scenario. A mixed session can still be connected to the same WebSocket state while interpreting that state through different local scenario logic, producing misleading procedure steps and telemetry behaviour.

If one workstation shows a different scenario title, imaging-window label, GS2 logic or emergency flow, return that workstation to scenario selection and choose the correct scenario. If the exercise is already running, recover only a non-SOM client as described in Section 6.5.7; otherwise perform a clean restart.

6.7.3 Panel Selection Is Local and Is Not a Synchronisation Test

activePanel is local to each browser. It is correct for SOM to view the procedure while SOE1 views EPS and SOE2 views Payload. Different visible panels do not indicate desynchronisation. Use mission time, procedure effects, TC History and state-dependent telemetry to judge shared-state consistency.

6.8 Telemetry Availability: Expected States versus Faults

6.8.1 Before Simulation Start

Before simulationStatus becomes RUNNING, Ground Station telemetry is intentionally in the unavailable baseline and spacecraft-subsystem telemetry is also unavailable. This is not a communication fault. Start the scenario from SOM before diagnosing pre-start NO TELEMETRY.

6.8.2 RUNNING but Before GS1 AOS

During the interval from T+00:00 to GS1 AOS at T+01:00, local ground-station housekeeping can be displayed because the ground equipment itself is part of the running simulator, while spacecraft telemetry remains unavailable because no valid space-to-ground telemetry path exists. Geometry-dependent values can remain NO DATA and RF-dependent values can show NO SIGNAL/NO LOCK.

6.8.3 After AOS but Below the Elevation Mask

AOS begins the pass geometry, but the operational 5 deg elevation mask has not yet been satisfied. Elevation, azimuth and range can be present while the receiver chain is still acquiring. Do not classify this as telemetry corruption. Wait for the procedure to confirm GEL005 >= 5 deg.

6.8.4 Scenario 2 GS1 Loss

After the Scenario 2 contingency sets GS1 connection inactive, spacecraft-subsystem panels intentionally show NO TELEMETRY. Ground-station equipment telemetry can still exist because the ground station has not ceased to function; the spacecraft RF path has been lost. The training objective is to distinguish local ground equipment health from spacecraft observability.

6.8.5 Scenario 2 GS2 Acquisition

GS2 tracking begins only after the event-driven delay defined in the contingency. Before GS2 tracking starts, GS2 spacecraft geometry and RF data are intentionally unavailable. After tracking starts, geometry appears, but spacecraft-subsystem telemetry remains unavailable until GS2 GEL005 reaches

at least 5 deg. Once the lock condition is true, the spacecraft telemetry path returns even though the GS2 signal can still be MEDIUM before receiver filtering.

6.8.6 Persistent NO TELEMETRY after a Valid Link

If spacecraft-subsystem panels remain NO TELEMETRY after the applicable link has been established, verify the condition systematically: correct scenario selected locally; current GS elevation and link state; WebSocket port 3001; browser Console connection message; and shared mission time. If these are correct but all subsystem composables remain unavailable, record the state as a software defect rather than changing individual telemetry parameters.

Table 6-4 - Interpretation of common telemetry placeholders

Display	Normal meaning	When to troubleshoot
NO TELEMETRY	The ground has no valid path to the spacecraft subsystem, or the simulator is not running.	If it persists after the expected telemetry-lock condition.
NO SIGNAL	An RF-dependent ground measurement has no usable spacecraft signal.	If displayed while the active link should be GOOD and other consoles disagree.
NO LOCK	Carrier/frame function has not acquired or has lost lock.	If elevation/link conditions satisfy the procedure but lock never follows.
NO DATA	The particular geometry/measurement is not currently defined.	If the underlying pass/tracking state should already provide the measurement.
STANDBY	Equipment is intentionally inactive or awaiting operational configuration.	Only if the current procedure requires ACTIVE/READY and the command completed successfully.

6.9 Procedure and Telecommand Failures

6.9.1 Always Read the Complete Failure String

The command logic returns explicit FAILED messages for most invalid actions. These messages are the first fault-isolation source. The TC History preserves SPACON GO results, while SOM Last Result preserves procedure-action outcomes. A rejected command normally means a precondition is false; it does not mean the WebSocket or browser has failed.

Table 6-5 - Procedure/command failure diagnosis

Failure message or symptom	Cause	Recovery
FAILED - NO COMMAND SELECTED	ARM or GO was attempted without a selected catalogue entry.	Search/select the required code first.
FAILED - COMMAND NOT ARMED	GO was pressed without an active ARM state.	Press ARM, verify READY FOR GO, then press GO.
FAILED - WRONG PROCEDURE STEP	The current state does not authorise the requested action.	Return to the SOM procedure table and execute only the CURRENT row.
FAILED - ... NOT REQUESTED BY SOM	SPACON acted before the SOM procedure release.	SOM presses the current command-request button; SPACON then SELECT-ARM-GO.
FAILED - SIGNAL QUALITY NOT CHECKED	Filtering was requested before the required SOE/SOM signal check.	Complete the signal-reporting step first.
FAILED - ELEVATION NOT CONFIRMED / BELOW 5 deg	The operational acquisition gate is not met.	Wait for GEL005 $\geq$ 5 deg and complete the SOM confirmation.
FAILED - SIGNAL NOT GOOD / GS2 SIGNAL NOT GOOD	Active link does not satisfy command gating.	Complete the relevant receiver filtering/link-verification sequence.
FAILED - MEMORY DUMP NOT AUTHORIZED	MEM221 command released without the prior SOM decision.	Complete the authorisation step.
FAILED - EPS NOT NOMINAL / WAIT OR MITIGATE	Temperature/power gate is not satisfied.	Continue thermal mitigation and monitor until the nominal guard passes.
FAILED - PAYLOAD POWER NOT RAISED	CAM000 attempted before PWR740 restoration.	Complete PWR740 increase and verify the procedure state.
FAILED - CAMERA NOT CONFIGURED / NOT VERIFIED BY SOM	IMG901 preconditions are incomplete.	Complete camera configuration and the SOM verification row.

Failure message or symptom	Cause	Recovery
FAILED - IMAGE CAPTURE NOT COMPLETE	STB901 attempted before imageTaken is true.	Complete the required image acquisition.

6.9.2 Repeated Failure after the Correct Precondition Is Met

If the displayed condition now satisfies the requirement but the same failure persists, verify that the console has received the latest shared state. Compare mission time with the SOM console, inspect port 3001 connectivity and check the browser Console. A stale client can display one value while its command logic still operates on a different local state until synchronisation converges.

6.10 Scenario 1 Specific Troubleshooting

6.10.1 Memory Dump Appears Slow

The baseline memory dump reduces MEM221 in discrete simulation-time updates until the completion target is reached. Judge completion from MEM221, DMP telemetry and the procedure status, not from the approximate duration printed in the table. If mission time itself is not advancing, troubleshoot the SOM timer rather than the memory model.

6.10.2 EPS Mitigation Does Not Complete

After EPT014 - Reduce Payload Power, confirm that the command succeeded and that thermalCoolingActive is represented by a falling EPS temperature. The SOM nominal confirmation is accepted only in the defined 70-75 deg C range with acceptable power margin. A temperature above the range is a wait condition; a temperature that never changes while mission time advances indicates a state-transition defect.

6.10.3 Procedure Returns from Payload Preparation to EPS Mitigation

This is expected if payload power was restored too early and EPT014 later exceeds 75 deg C. The application automatically invalidates downstream payload/camera/image readiness and reopens the thermal-mitigation sequence. Repeat the mitigation, then delay PWR740 restoration until closer to the T+15:00 imaging window.

6.10.4 Frankfurt Image Is Wrong or Invalid

The image product is mission-time dependent. Verify the exact T+ time at IMG901 execution. Scenario 1 formal acquisition is T+15:00 to T+16:00. If the command was executed outside that interval, a different ground-track image or validity classification is an expected simulation result, not an image-panel defect.

6.10.5 Imaging Window Missed

If the team reaches image acquisition after the formal window or after the GS1 link is no longer usable, do not redefine the result as nominal. Record the training outcome, perform the clean reset from Section 3.11 and repeat the scenario. The fixed window is part of the mission-operations training objective.

6.11 Scenario 2 Specific Troubleshooting

6.11.1 Battery Emergency Does Not Appear

At the baseline Step 12, BCH096/BCH097/BCH098 should be approximately 96/3/5%. Confirm that Scenario 2 is selected locally and that Step 12 is the CURRENT row. If the batteries show unrelated values, check whether the client joined an old shared server state from a previous run; perform a clean host restart if a new exercise was intended.

6.11.2 Emergency Workflow Appears Stuck

The valid path is OPEN CONTACT LIST -> select GNC -> allow the prepared message to complete -> SEND MESSAGE -> wait for the deterministic response -> IMPORT NEW PROCEDURE. Selecting Flight Director, EPS, Thermal or Ground Segment does not advance the recovery path. After SEND MESSAGE, the response timer is approximately 10 s. After IMPORT NEW PROCEDURE, the import delay is approximately 1.8 s.

6.11.3 GS1 Is Lost but GS2 Does Not Appear Immediately

This is expected. GS2 tracking is scheduled 120 simulation seconds after the Step 18 GS1-loss event. The delay is relative to the operator-driven contingency event, not to a universal absolute mission time. Use the live GS2 AOS countdown and mission time. If the countdown itself stops, troubleshoot the SOM timer.

6.11.4 GS2 Geometry Is Present but Spacecraft Panels Still Show NO TELEMETRY

This is expected while GS2 elevation is below 5 deg. The GS2 telemetry-lock condition requires GS1 lost, GS2 tracking active and GS2 elevation ≥ 5 deg. Continue monitoring GEL005. If elevation exceeds 5 deg and spacecraft panels remain unavailable, verify port 3001 synchronisation and local Scenario 2 selection.

6.11.5 GS2 Signal Remains MEDIUM

GS2 telemetry lock and GOOD signal quality are separate states. After the elevation lock, Step 21 verifies the unfiltered GS2 link and Step 22 releases GSE001 Filter Signal. The command applies to GS2 because the Scenario 2 procedure is at the GS2 filtering step. If GSE001 was executed at the wrong procedure step, the expected GS2 filtered flag is not established.

6.11.6 Payload Restoration Is Invalidated before T+30:00

After the recovered PWR740 increase, EPT014 rises slowly. If payload power is restored too early, the temperature can again exceed the nominal limit before the formal imaging window and the application returns to mitigation. This is a mission-planning/timing consequence. Re-cool EPS and restore payload power closer to T+30:00.

6.11.7 Image Taken after T+30:30

The formal Scenario 2 procedure criterion is T+30:00 to T+30:30. The underlying image-selection helper continues to label the following short segment as VALID TARGET IMAGE, but the manual intentionally adopts the stricter procedure criterion. Treat an acquisition after T+30:30 as a procedure-timing deviation.

6.12 Audio, Video and Image Asset Problems

6.12.1 Browser Autoplay Restrictions

Modern browsers can block programmatic playback of audible media until the user has interacted with the page. MDN documents that media with audible audio is generally subject to autoplay blocking, while muted media is normally not blocked under the same rule [47]. MCS26 anticipates this by attempting to unlock intro audio through user interaction and by catching rejected play() calls.

If the introductory background audio is silent but the interface otherwise works, first click/interact with the scenario-selection page and select a scenario. Audio silence alone is not evidence of a WebSocket or simulation-state fault.

6.12.2 Welcome Video Does Not Start or Finish

Scenario entry depends on the welcome video reaching its ended event. The current source loads /videos/welcome-intro.mp4 after scenario selection. If that file is missing, returns an HTTP error, is unsupported by the browser or cannot be decoded, the application can remain in the intro-video phase. Open DevTools -> Network, reload and inspect the request for the MP4 file.

A successful request should return the media resource from the host. A 404 indicates an asset/package problem; a media decode error indicates browser/codecs handling. After repairing the asset or browser condition, reload before starting the exercise.

6.12.3 End Video Does Not Play

The end sequence loads /videos/end.mp4 as a muted autoplay video. Muted video is less restricted by browser autoplay policy [47]. If the simulation fades to black and does not return to scenario selection, check the end-video request and browser Console for 'End video play failed'. A full host restart is not

required to diagnose the media request, but a clean reset is recommended before the next formal exercise.

6.12.4 Captured Image Is Missing

Captured image files are loaded from `/photo/<capturedImageName>.jpg`. If image metadata and `imageValidity` are present but the picture is broken, the simulator state has already selected an image name; the next diagnostic target is the static file request. Inspect DevTools Network for the exact JPG path. A 404 or blocked request is an asset/deployment problem rather than a payload command failure.

6.13 Browser Developer Tools as a Diagnostic Instrument

6.13.1 Console

Use Console for application startup role identification, WebSocket connection status, state-sync messages, media playback failures and uncaught JavaScript exceptions. Record the first error in time rather than only later cascaded errors.

6.13.2 Network

Use Network to distinguish HTTP frontend/media failures from WebSocket failures. Chrome DevTools can disable cache, clear cache, and perform an Empty Cache and Hard Reload [42]. Filter by WS when diagnosing WebSocket traffic and by Media/Img when diagnosing videos, audio or captured-image assets.

The MCS state connection is expected to target port 3001. Requests on port 5173 are frontend resources and Vite development traffic. A healthy page-load request does not prove that the port-3001 channel is healthy.

6.13.3 Do Not Use DevTools to Modify State during Training

CAUTION. Developer tools are a diagnostic instrument, not an operator control interface. Do not edit Vue state, execute JavaScript to force procedure variables, alter network responses or modify DOM values during a formal training run. Such actions invalidate the scenario state and make subsequent telemetry/procedure results uninterpretable.

6.14 Source-Control and Version-Mismatch Problems

6.14.1 Confirm the Host Revision

When one installation behaves differently from the documented manual, confirm the host revision before assuming a runtime defect.

```
git          branch          --show-current
git          log             -1          --oneline
git status
```

A clean main branch at the intended revision is the preferred operational baseline. If local changes are present, document them or move them out of the training baseline before further comparison.

6.14.2 Software Updated but Browser Still Shows Old Behaviour

Use a complete update sequence rather than only `git pull`: stop the services, update the repository, run `npm install` or `npm ci` as appropriate, run `npm run build`, restart both services, and hard-refresh all operator browsers. This sequence removes the most common ambiguity between source revision, dependencies, running process and cached frontend.

6.14.3 Git Pull Refuses to Fast-Forward

A refusal from `git pull --ff-only` indicates that the local branch cannot be updated as a simple fast-forward. Do not resolve this on the operational host by creating an unreviewed merge. Inspect local commits and modifications, preserve development work, and restore the training copy to the approved repository workflow.

6.15 Known Architectural Limitations Relevant to Troubleshooting

Table 6-6 - Current implementation characteristics that can resemble faults

Characteristic	Current implementation	Operational implication
Shared-state persistence	The WebSocket server stores state only in RAM and keeps it until the server process stops.	Closing browsers is not a clean reset; restarting the server clears the session.
Mission time source	The SOM browser increments missionSeconds once per second and broadcasts it.	Refreshing/closing SOM during RUNNING can freeze the mission clock.
Scenario selection	selectedScenario is local and not part of the shared-state object.	Every client must choose the same scenario, including after a client reload.
Panel selection	activePanel is local.	Different visible panels across consoles are normal.
Role parsing	Only SOM, SOE and SPACON are valid; anything else defaults to SOE.	A malformed SOM/SPACON URL can silently produce an SOE console.
Transport	Socket.IO is explicitly restricted to WebSocket on both client and server.	Port 3001/WebSocket must work; do not rely on polling fallback.
Authentication / encryption	The current classroom baseline uses local HTTP/WebSocket without user authentication.	Operate only on the trusted isolated/controlled LAN defined in Section 2.
Frontend service	Vite development server is used for classroom hosting.	If 5173 is occupied, Vite can select another port unless corrected; role scripts do not follow that change.
Media progression	Welcome/end transitions depend on video playback/ended events.	Missing/failed media can prevent automatic navigation.
Telemetry model	Many values are synthetic or state-coupled rather than flight data.	A changing value is not proof of a physical model; troubleshoot logic only when behaviour contradicts the defined scenario.

6.16 Controlled Recovery Procedures

6.16.1 Recovery of One Non-SOM Operator Console

Use this procedure only when SOM mission time is still advancing and the other consoles remain synchronised.

Table 6-7 - Single-client recovery

Step	Action	Acceptance criterion
1	Record the affected role, mission time and visible error.	Diagnostic evidence preserved.
2	Confirm host TCP 5173 and 3001 reachability from the affected workstation.	Both TcpTestSucceeded values are True.
3	Close the affected browser tab and re-open the exact role URL.	Correct role displayed/logged.
4	Select the same scenario locally.	Scenario title/window agrees with other consoles.
5	Wait for state synchronisation before any operator action.	Mission time and shared procedure effects converge with SOM.
6	Resume the role only after consistency is confirmed.	No stale command/telemetry decision is made.

6.16.2 Full Clean Restart

Use this procedure after SOM loss, mission-time freeze, ambiguous shared state, software update, or when a new formal exercise must start from a known baseline.

Table 6-8 - Full clean restart

Step	Action	Acceptance criterion
1	Stop operator actions and record the fault state.	No further commands issued.
2	Close or stop the WebSocket server with Ctrl+C.	Port 3001 listener disappears.
3	Stop the Vite frontend with Ctrl+C.	Port 5173 MCS listener disappears.

Step	Action	Acceptance criterion
4	If software changed, confirm Git revision, dependencies and npm run build.	Known clean baseline.
5	Run scripts/start-host-dev.bat.	3001 and 5173 services start on the documented ports.
6	Run scripts/show-host-ip.bat and confirm the active adapter address.	Current HOST-IP recorded.
7	Test ports from an operator workstation.	5173 and 3001 reachable.
8	Hard-refresh/re-open all role browsers.	Current frontend loaded.
9	Select the same scenario on all clients.	Consistent local scenario context.
10	SOM starts the simulation only after all consoles report ready.	Mission time advances and shared state is coherent.

6.16.3 Rebuilding a Suspect Host Installation

If the local repository, dependency tree and runtime state have accumulated uncontrolled modifications, a clean re-clone is preferable to repeated ad-hoc repairs. Preserve any required logs or local development work first. Create a fresh directory, clone the approved repository, install dependencies, build, start the services and repeat the installation acceptance test from Section 3. This recovery should be treated as configuration restoration, not as an operator-level action during a training session.

6.17 Troubleshooting Acceptance Criteria

A fault is considered recovered only when the system has returned to a defined operational condition, not merely when an error dialog disappears. For infrastructure recovery, the host shall run the intended Git revision; TCP ports 5173 and 3001 shall be listening; an operator workstation shall reach both ports; each browser shall load the correct role and scenario; the MCS WebSocket connection shall be established; and shared mission time/state shall be coherent across consoles. For procedure recovery, the CURRENT row, reported telemetry and command preconditions shall agree with the scenario definition in Section 5. Reproducible unresolved conditions shall be classified as software defects; undocumented workarounds shall not be used in formal training.

7Appendix A : Research Roadmap

MCS26_V26_01 is intentionally a training simulator rather than a flight-representative spacecraft simulator or digital twin. Its current architecture nevertheless provides a useful foundation for a substantially more rigorous research platform because the operator roles, procedure logic, subsystem views, telecommand workflow and distributed multi-console interface already exist. Future work can therefore concentrate on replacing or augmenting the present synthetic telemetry source and scenario state models without discarding the mission-operations interface.

The development path described in this section is divided into two principal research tracks. The first track uses recorded operational telemetry as an external data source. The second replaces selected state-coupled training equations with physical or semi-physical spacecraft subsystem models. A third, hybrid track combines recorded data with validated simulation models. These tracks serve different research questions and shall not be treated as interchangeable.

7.1 Development Boundary and Research Objectives

7.1.1 Current Baseline

The current baseline generates spacecraft and ground telemetry inside the Vue application from mission time and shared scenario state. Dedicated telemetry composables provide EPS, AOCS, TCS, Payload, C&DH and Ground Station data, while the main application orchestrates the procedure and scenario events. This arrangement is effective for deterministic classroom training because the entire scenario is self-contained and can operate on a local network without an external simulator or telemetry archive.

The same architecture limits physical fidelity. Most numerical telemetry is generated from bounded relationships selected for training consistency. The ground-pass geometry is a scenario function rather than propagated ephemeris. Thermal behaviour is not derived from a validated thermal network, battery behaviour is not an electrochemical model, AOCS values are not generated by integrating rigid-body dynamics, and the RF parameters are not the result of a full end-to-end link budget. These limitations are acceptable for the educational objective stated in Sections 0 and 4 but define the starting point for research development.

7.1.2 Research Objectives beyond the Training Baseline

A research-grade extension should pursue measurable engineering objectives rather than simply increasing the number of telemetry rows. Relevant objectives include: replaying long-duration operational telemetry without altering its statistical characteristics; reproducing subsystem behaviour from explicit differential or state-space models; making operator commands causal inputs to those models; injecting repeatable faults; correlating simulator output with trusted reference data; quantifying uncertainty and model error; and evaluating anomaly-detection or forecasting algorithms under mission-operations conditions.

Once development moves into these areas, the project is no longer primarily an educational simulator. It becomes a research platform for spacecraft operations, simulation, data engineering and automated monitoring. Documentation, configuration control, model validation and experiment reproducibility should therefore increase accordingly.

Table 7-1 - Principal future-development tracks

Track	Primary data source	Interaction model	Research value	Principal limitation
A - Recorded telemetry replay	Historical spacecraft telemetry, e.g. ESA-AD	Operator mainly observes/replays; historical telecommands/events can be displayed	Real distributions, gaps, rare events and anomalies; ideal for monitoring and anomaly-detection research	Recorded data cannot respond physically to new operator commands.

Track	Primary data source	Interaction model	Research value	Principal limitation
B - Physics-based simulation	Mathematical subsystem and environment models	Operator commands directly change simulated spacecraft state	Counterfactual operations, fault injection, control studies and causal scenario development	Requires model design, parameters, verification and validation.
C - Hybrid replay + model	Recorded telemetry used to initialise/calibrate models, then model propagation	Historical context followed by simulated intervention	Combines realism of flight data with interactive what-if experiments	Counterfactual validity depends on the calibrated model; cannot be claimed from replay data alone.

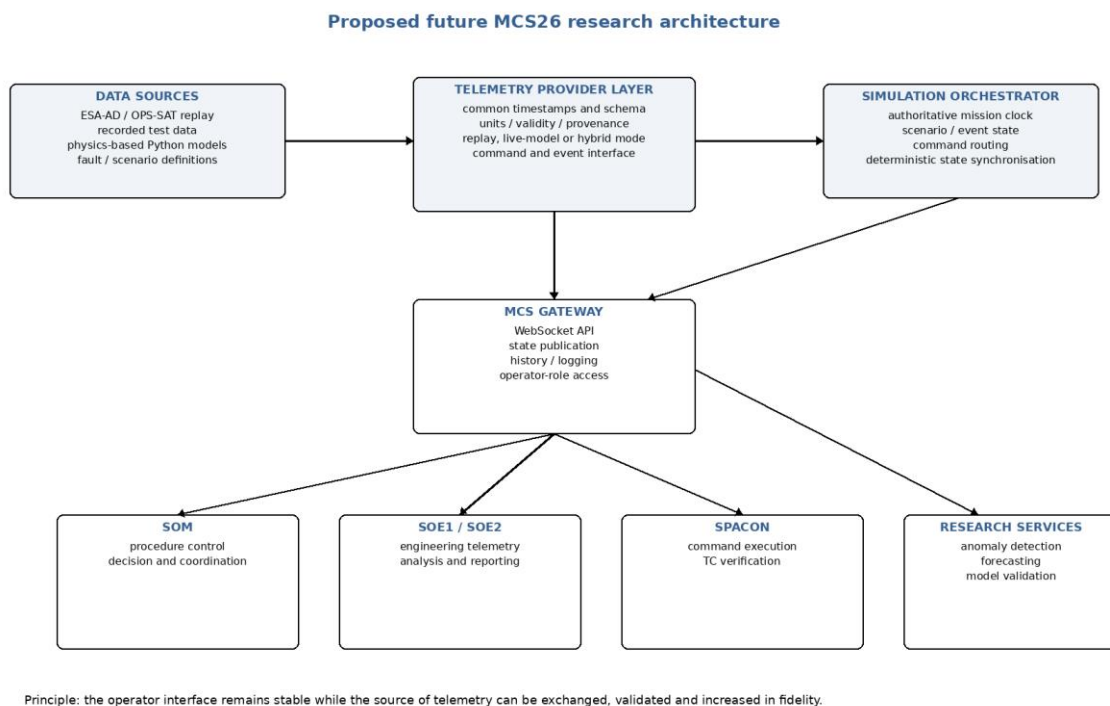


Figure 7-1 - Proposed future MCS26 architecture with an interchangeable telemetry-provider layer.

7.2 Architecture Required before Increasing Model Fidelity

7.2.1 Move Authoritative Simulation Time to the Host

The first architectural change should be independent of telemetry fidelity: mission time should become an authoritative host-side service rather than a timer generated by the SOM browser. The current browser-side timer is adequate for training but makes the session dependent on the continued existence of a particular client. A research simulator should maintain simulation time, event scheduling and pause/run state on the host so that operator clients are observers/controllers of the same deterministic time base.

The host clock should support at least real-time execution, paused execution, accelerated replay and deterministic single-step execution. Recorded telemetry replay additionally requires seeking to a defined epoch and replaying at a configurable time scale. Physics-based models require a well-defined integration step or scheduler contract. All published telemetry, commands and events should reference this same simulation time.

7.2.2 Introduce a Telemetry-Provider Interface

The operator interface should no longer need to know whether a value originated from a Vue training equation, a historical dataset, a Python model or a live laboratory source. A telemetry-provider layer should normalise those sources into a common contract. This preserves the existing panels while allowing the underlying source to be changed by configuration.

Table 7-2 - Minimum telemetry-provider data contract

Field	Purpose	Requirement
timestamp	Common temporal reference for the sample.	Monotonic within a source; convertible to simulation time.
parameter_id	Stable machine-readable channel identifier.	Unique within the configured data source.
display_name	Human-readable engineering name when known.	May remain anonymised for ESA-AD.
subsystem	Logical subsystem or channel group.	Must be source-derived or explicitly mapped; never guessed.
data_type	Numerical, categorical, boolean, counter, event, etc.	Required for rendering and analytics.
value_raw	Original source value.	Preserve before conversion or filtering.
value_engineering	Converted engineering value where conversion is known.	Optional when units/conversion are anonymised.
unit	Engineering unit.	Unknown shall be represented as unknown, not inferred.
validity	Valid, invalid, gap, stale, quality flag.	Separate data validity from anomaly status.
status	Nominal/warning/alarm or research classification.	Derived by source rules; must retain provenance.
source	Synthetic, ESA-AD Mission1, Python EPS model, etc.	Mandatory for traceability.
event_labels	Anomaly, rare nominal event, communication gap, invalid segment, fault ID.	Optional but version-controlled.

7.2.3 Separate Data Generation from Presentation

The present composables combine data generation and presentation-ready rows. For research use, these functions should be separated into: model/data-source services; an engineering telemetry schema; and presentation adapters. Numerical state should be generated in SI or explicitly defined engineering units and only then formatted for the user interface. Status colours should be produced from limit definitions or event labels rather than embedded ad hoc in display logic.

7.2.4 Server-Side Scenario Orchestration

Scenario truth, fault injection, procedure-release state and command routing should migrate to the same host-side orchestrator as the mission clock. This provides one authoritative state machine for all operator clients and allows future Python models to receive commands independently of which browser issued them. It also enables full event logs, deterministic replay and automated regression tests.

7.3 Track A - Integration of Real ESA Spacecraft Telemetry

7.3.1 Identification of the Relevant ESA Dataset

The public ESA resource that most closely matches the previously identified requirement is the ESA Anomaly Dataset (ESA-AD), released within the European Space Agency Anomaly Detection Benchmark (ESA-ADB). The dataset contains real operational spacecraft telemetry collected from three ESA missions and curated with spacecraft operations engineers from ESOC [50]-[53]. The public mission identities, channel names, some timelines and engineering units are intentionally anonymised to protect sensitive information while retaining the time-series structure [51].

Engineering note. The ESA-AD source contains data from three missions, not three telemetry 'states'. The detailed 2026 dataset paper describes three anonymised spacecraft labelled Mission1, Mission2 and Mission3. The ESA-ADB evaluation framework separately defines several benchmark scenarios/mission phases; these concepts shall not be conflated.

7.3.2 Scale and Engineering Content of ESA-AD

The detailed 2026 publication reports 224 telemetry channels, 821 telecommands/control signals, 3,512,724 telecommand executions and 1,430 annotated events across the three spacecraft [51]. It reports 2,296,122,157 individual data points over an anonymised combined duration of 25.5 years, with 11.6 GB compressed data. The channel set includes numerical and categorical telemetry from six common spacecraft subsystems. The public ESA-ADB materials provide a shorter headline description of approximately 1.5 billion data points and 12 GB; the manual should use the detailed paper when exact per-mission statistics are required [50][51].

Table 7-3 - ESA-AD detailed statistics reported in the 2026 dataset paper [51]

Property	Mission1	Mission2	Mission3	Total
Telemetry channels	76	100	48	224
Numerical / categorical	76 / 0	90 / 10	4 / 44	170 / 54
Subsystems represented	4	5	3	6 unique across dataset
Telecommands	698	123	0	821
Telecommand executions	1,594,722	1,918,002	0	3,512,724
Data points	774,856,895	776,734,364	744,530,898	2,296,122,157
Anonymised duration	14 years	3.5 years	8 years	25.5 years
Compressed size	3.8 GB	4.1 GB	3.7 GB	11.6 GB
Annotated events	200	644	586	1,430
Anomalies	118	31	8	157
Rare nominal events	78	613	25	716
Communication gaps	4	0	397	401
Invalid segments	0	0	156	156

7.3.3 Why ESA-AD Is Valuable for MCS26 Research

ESA-AD provides exactly the type of behaviour that is difficult to reproduce convincingly with manually chosen oscillations: long-duration drift, non-stationarity, irregular sampling, data gaps, mixed numerical and categorical channels, rare nominal events, anomalous events and millions of operational control actions. The dataset was intentionally designed to reflect the practical complexity of telemetry available to spacecraft operations engineers rather than a clean laboratory time series [51].

This makes it suitable for research on operator monitoring, anomaly recognition, alarm design, temporal context, data reduction, forecasting and human-machine interaction. It is also suitable for testing how a browser-based MCS behaves when the telemetry volume is no longer limited to a few hundred deterministic values generated in memory.

7.3.4 Anonymisation Constraints

Anonymisation is a central constraint. ESA-AD deliberately hides information including mission and channel names, timelines and units for some measured values [51]. The authors explicitly note that this prevents physics-informed interpretation and prevents algorithms from using mission-specific semantic knowledge such as matching telecommands to channels by names. Consequently, a future MCS26 integration shall not relabel an anonymous ESA channel as 'battery voltage', 'reaction-wheel speed' or 'payload temperature' unless the released metadata explicitly supports that mapping.

The scientifically correct first integration is therefore a generic 'Real Telemetry / ESA-AD' monitoring mode that preserves source channel identifiers and source-defined grouping. Subsystem-specific MCS panels can be populated only where a defensible mapping exists. Unknown unit means unknown unit. An anonymised timeline must remain a replay-relative timeline rather than being presented as a real mission UTC epoch.

7.3.5 Data Acquisition and Configuration Control

The raw dataset should be treated as an external controlled research asset, not copied into the main MCS26 Git repository. The public Zenodo release consists of separate Mission1, Mission2 and Mission3 archives and publishes checksums and a DOI [52]. A research installation should maintain a data

manifest containing dataset version, DOI, archive checksum, local storage path, preprocessing-software revision and the exact subset used by each experiment.

This prevents an experiment from becoming irreproducible when a dataset version or preprocessing rule changes. The original archive should remain read-only. Derived caches, downsampled visualisation products and train/validation/test material should be generated into separate directories.

7.3.6 Ingestion and Normalisation Pipeline

The ingestion pipeline should be implemented as a backend process rather than loading multi-gigabyte archives directly into the browser. It should parse the ESA-AD channel and event metadata, preserve the original sample timestamps, identify categorical and numerical channels, retain missing-data intervals and attach anomaly/rare-event labels without overwriting the raw values.

A normalisation layer may create a common internal timestamp and channel schema for MCS26, but it should not force all channels onto a single sampling rate unless a specific algorithm requires resampling. One of the research-relevant properties of spacecraft telemetry is precisely that different channels can have different sampling frequencies and gaps [51]. The original irregularity must remain recoverable.

7.3.7 Replay Engine

The replay engine should provide deterministic time-window playback from the host. At a minimum it should support start epoch, stop epoch, pause, seek, 1x real-time, accelerated replay and reproducible re-run. The engine should read only the time window needed for the next update rather than materialising billions of points in memory.

For operator display, the server can publish the most recent sample of each selected channel at the configured display update rate. For plots, the server should provide bounded time windows with decimation or aggregation suitable for visualisation. Analytical processes should continue to access the original resolution. This separation prevents display optimisation from silently changing the data used by research algorithms.

7.3.8 Telecommands in Recorded Data

ESA-AD contains hundreds of telecommand/control-signal identifiers and millions of executions [51]. In a replay scenario these are historical events: they indicate that a command occurred in the recorded mission timeline. They are not executable commands that MCS26 can resend to a real spacecraft or to the historical data.

The correct replay representation is to display them as time-stamped command events and use them as context for telemetry interpretation. If the research question requires the trainee to choose a different command and observe a counterfactual result, the system must transition to a validated simulation model. Historical telemetry alone cannot answer 'what would have happened if a different command had been sent'.

7.3.9 Constructing Training or Research Scenarios from ESA-AD Events

ESA-AD distinguishes anomalies, rare nominal events, communication gaps and invalid data segments [51]. This structure allows new MCS scenarios to be based on authentic temporal patterns without claiming a specific mission identity. A scenario can begin before an annotated event, hide the label from the operator, and ask SOE personnel to identify abnormal behaviour, communicate the affected channels and recommend an operational response.

The ground-truth labels should remain hidden during the operator phase and revealed only during review. This enables quantitative assessment of detection delay, false alarms, missed events and channel localisation. Rare nominal events are particularly valuable because they test whether a trainee or algorithm can avoid classifying every unusual behaviour as an anomaly.

7.3.10 Limitations of ESA-AD for a Commandable Spacecraft Simulator

ESA-AD is a telemetry dataset, not a complete spacecraft engineering model. It cannot provide arbitrary command response, continuous parameter semantics, verified physical units for every channel, full

spacecraft configuration, orbit/ground geometry or component-level causality. The anonymisation further prevents domain-specific physical interpretation [51]. For those reasons it should not replace the physics-model track when the research objective is closed-loop spacecraft control.

Its strongest role in MCS26 is as a real-data replay and anomaly-analysis source, a validation source for data-handling infrastructure, and a statistical reference for how operational telemetry behaves. It is not a flight dynamics or subsystem model.

7.4 OPS-SAT Telemetry as a Complementary Named-Mission Dataset

A complementary ESA-related resource is OPSSAT-AD, an AI-ready anomaly-detection dataset containing telemetry acquired onboard OPS-SAT, an ESA-operated CubeSat [55]. Unlike the anonymised three-mission ESA-AD collection, this dataset is explicitly associated with OPS-SAT. It is useful where the research question benefits from a named mission context or from comparing algorithms across a smaller satellite benchmark.

OPSSAT-AD should not be treated as a replacement for the larger ESA-AD corpus. The two resources address related but different benchmarking needs. A future MCS26 data-source interface should be generic enough to support both, thereby preventing the UI from becoming coupled to one dataset format.

7.5 Track B - Physics-Based Spacecraft Models in Python

7.5.1 Model-Service Architecture

The second development track is to move spacecraft behaviour into explicit subsystem models, for example implemented in Python. Each model should own a defined state vector, inputs, parameters, outputs and update law. The model engine should advance those states from the authoritative host simulation clock and receive telecommands through a controlled command interface. Telemetry should then be generated from the model states rather than generated directly for display.

Python is appropriate for a research prototype because of its numerical and scientific ecosystem, but the architecture should not make the browser dependent on Python-specific objects. The browser should receive a language-neutral telemetry schema through the gateway. This enables later replacement of a Python model by C++, Modelica, MATLAB/Simulink-generated code, an SMP component or hardware-in-the-loop source without redesigning the UI.

Table 7-4 - Common model interface for a future simulation engine

Interface element	Function
initialise(configuration, initial_state)	Loads model parameters, initial conditions and random seed.
step(t, dt, inputs)	Advances the model by a deterministic simulation step.
apply_command(command, arguments)	Validates and applies an operator/automation command.
publish_telemetry()	Returns current engineering state through the common telemetry schema.
inject_fault(fault_id, parameters)	Applies a controlled fault or degraded mode.
checkpoint() / restore()	Supports deterministic restart, branching and experiment replay.
health()	Reports numerical/model health separately from spacecraft health.

7.5.2 EPS and Solar-Array Model

A first physics-based EPS model should compute spacecraft power balance from solar-array generation, conversion efficiency, active loads and battery exchange rather than from fixed margin values. The environment/orbit model supplies Sun illumination and eclipse state. Array output should depend on illumination, incidence angle, temperature and configurable degradation; a simplified maximum-power-point or regulated-bus model then supplies the main bus [23][40].

Loads should be explicit consumers identified by subsystem and mode. Payload imaging, transmitter operation, AOCS actuators, processors and heaters should each contribute to the bus load. This allows NET118-style margin to become a derived engineering quantity rather than a scenario lookup.

7.5.3 Battery Model

A research battery model can begin with coulomb-counting SOC and an equivalent-circuit voltage model. Current sign convention, cell/pack configuration, capacity, charge/discharge efficiency and temperature dependence must be defined explicitly. A higher-fidelity model can include open-circuit-voltage versus SOC curves, internal resistance, RC polarisation branches, cell imbalance and ageing/SOH effects [39].

Battery equalisation should then be based on an implemented electrical architecture. If three independent packs can exchange energy, the transfer path, converter efficiency, current limits and thermal losses must be modelled. If the actual architecture does not permit pack-to-pack energy transfer, the Scenario 2 training abstraction should remain clearly separated from the research model.

7.5.4 Thermal Model

A practical first TCS research model is a lumped-parameter thermal network. Each node represents an equipment unit or structural region with thermal capacitance; conductive links connect nodes and radiative surfaces reject heat to space. Internal dissipation is received directly from the EPS/load model. Solar, albedo and planetary infrared inputs are supplied by the environment model. Heaters become commanded electrical loads with thermostatic or software control [24].

Model fidelity can increase from a few nodes representing EPS, payload, avionics and batteries to a larger network calibrated against a dedicated thermal analysis. The required fidelity should be driven by the research question. A training model does not benefit from hundreds of nodes if the resulting telemetry cannot be validated or interpreted.

7.5.5 AOCS Model

A physics-based AOCS requires rigid-body attitude propagation, quaternion kinematics, actuator dynamics and sensor models. The state should include attitude quaternion, angular velocity and, where appropriate, reaction-wheel momentum. Environmental disturbance torques can be added progressively. Guidance defines nadir, Sun-pointing, target-tracking or safe-mode references; an estimator uses noisy sensor measurements; and a controller produces actuator commands [25][31].

Star trackers, Sun sensors, magnetometers and gyros should include realistic update rates, biases, noise, validity and dropout rather than independent display oscillations. Reaction wheels should include torque and speed limits so that momentum saturation becomes a causal event. Magnetorquer unloading or thruster use can then be modelled as an actual response to stored momentum.

7.5.6 Orbit, Environment and Ground-Station Geometry

The current sine-based ground pass should be replaced by a propagated orbit and station geometry. A research implementation can begin with two-body/Keplerian propagation or SGP4 when a TLE is the data source, and later move to numerical propagation with perturbations as required. Position must be transformed consistently between inertial, Earth-fixed and local topocentric frames [26][30].

Ground-station azimuth, elevation and slant range should then be calculated from station coordinates and spacecraft state. The station model can apply a constant or azimuth-dependent elevation mask; ESA GODOT already provides concepts for ground-station masks, range, Doppler and observation modelling that can serve as an engineering reference [28]. Doppler should be computed from relative range rate and the configured carrier frequency rather than a pass-progress cosine.

7.5.7 Communications Model

A future communications model should calculate received carrier power and margin from spacecraft transmitter power, antenna gains, free-space loss, pointing loss, ground antenna state, receiver noise temperature and configured coding/modulation. The output can then drive carrier lock, Eb/N0, BER/FER and data-rate availability coherently. Commands and telemetry can be blocked by the same link-state model rather than by a separate boolean.

The communications model should still distinguish physical RF reception from frame/data validity. Acquisition can be represented as a state machine: antenna tracking, carrier acquisition, symbol/bit

synchronisation, decoding and frame synchronisation. This provides a physically defensible basis for future AOS/LOS and degraded-link scenarios.

7.5.8 Payload Model

A payload model should be built around the mission product rather than around an arbitrary list of telemetry values. For an optical-imaging instrument, inputs include spacecraft pointing, ground target geometry, detector/optics temperature, electrical power, timing and available storage. The model then determines whether acquisition is permitted and produces an image-data volume and quality metrics.

A research version does not initially need to render physically accurate Earth imagery. It is more important that exposure timing, boresight/ground intersection, pointing error, SNR assumptions, detector state and storage volume are internally coherent. Image files can be attached later to the generated metadata or produced by a separate scene-generation model.

7.5.9 C&DH and Data-System Model

C&DH should become the explicit information backbone of the simulated spacecraft. Commands should enter a queue, pass validation, be time-tagged/executed and produce acknowledgement states. Subsystems should publish telemetry at configured rates. A packetisation layer can group values into housekeeping/event services, while a mass-memory model stores payload files and recorded telemetry. Downlink removes data according to the available communications rate.

This approach would make memory occupancy, queue depth, telemetry rate and dump duration derived quantities rather than independent indicators. It also enables realistic failure cases such as packet backlog, file-system errors, corrupted data products, delayed command execution or time-synchronisation faults.

7.5.10 Fault and FDIR Models

Research scenarios should distinguish the injected physical/software fault from the telemetry symptoms and from the FDIR response. For example, a reaction-wheel friction increase is a fault; increased wheel current and temperature are effects; an onboard wheel-isolation command is an FDIR response. Encoding only the final red telemetry value removes the diagnostic chain that anomaly-response research requires.

Faults should be reproducible and parameterised by start time, duration, magnitude and recovery behaviour. The simulator should support both known faults used for training and hidden faults used for operator or algorithm evaluation.

Table 7-5 - Suggested state/input/output definition for physics-based subsystem models

Model	Core state	Principal inputs	Principal outputs
Orbit/environment	Position, velocity, Sun/eclipse geometry	Initial orbit, time, force/environment configuration	ECI/ECEF state, latitude/longitude/altitude, illumination, station geometry
EPS	Bus state, battery SOC/equivalent-circuit states	Solar input, loads, switch commands, battery temperature	Voltages, currents, generated power, SOC, power margin, trip states
TCS	Thermal-node temperatures, heater states	Dissipation from EPS, external fluxes, heater commands	Equipment temperatures, radiator flow, margins, trends
AOCS	Quaternion, angular rate, wheel speeds, estimator states	Guidance target, disturbance torque, sensor measurements	Attitude, rates, errors, actuator states, pointing validity
COMMS	Acquisition/link state, queues	Geometry, antenna pointing, transmitter mode, data rate	RSSI, C/N0, Eb/N0, BER/FER, carrier/frame lock, available rate
Payload	Instrument mode, detector/optics state, acquisition/file state	Power, TCS, pointing, time, storage	Readiness, exposure state, data volume, image quality/metadata
C&DH	Command queues, memory/files, time, software modes	Operator commands, subsystem TM, link availability	Command verification, TM packets, memory occupancy, event logs

7.6 Simulation-Model Portability and Standards-Based Integration

7.6.1 ECSS Simulation Modelling Platform

If the project develops beyond a Python-only research prototype, model interfaces should follow a portable, component-based philosophy. ECSS-E-ST-40-07C Rev.1, Simulation modelling platform - Level 1, was issued in August 2025 specifically to complement ECSS software engineering for simulation software and to enable reuse of simulation models between environments, organisations and missions [56]. The standard is the current ECSS evolution of the Simulation Model Portability work historically led by ESA [57].

The principal architectural lesson is directly applicable to MCS26 even if the first implementation does not claim formal SMP compliance: model physics should be separated from simulator infrastructure, model interfaces should be explicit, data types and fields should be declared, and common services such as time, events and logging should be supplied by the simulation environment rather than reimplemented independently in every subsystem model.

7.6.2 Model Reuse across Development Stages

ESA's Simulation Model Portability concept was motivated by the need to reuse models across engineering, test, operations preparation and training simulators [57]. This is particularly relevant to the proposed MCS26 evolution. A battery, thermal or AOCS model should not be written only for one screen or one scenario. It should expose a stable interface so the same model can be used in offline analysis, automated tests, a multi-console simulator and later hardware- or software-in-the-loop facilities.

7.7 Track C - Hybrid Recorded-Data and Physics-Based Simulation

7.7.1 Purpose of a Hybrid Mode

The most powerful long-term architecture can combine historical telemetry and executable models. A replay window can provide an authentic pre-event context. At a selected branch point, the platform can initialise a model state from the available data and allow the operator to apply commands that were not present in the historical record. This creates a controlled what-if experiment.

7.7.2 State Estimation before Branching

A replay-to-model transition is not achieved by copying the last displayed value from each channel. Internal model states that are not directly measured - such as battery polarisation states, thermal-node temperatures, estimator covariance or reaction-wheel disturbance states - must be inferred or initialised. A state-estimation or calibration step is therefore required before the model branch can be considered physically meaningful.

7.7.3 Counterfactual Limitations

After branching, the output is simulated and must no longer be described as recorded ESA telemetry. Historical data provides the observed trajectory for the command sequence that actually occurred; it does not validate a different counterfactual response. The uncertainty of the model should increase with time after the branch unless additional observations are assimilated.

Engineering note. A hybrid mode should visibly identify the data provenance at all times: RECORDED, MODELLED, ASSIMILATED or DERIVED. Mixing recorded and simulated values without provenance would undermine the scientific value of the platform.

7.8 Verification, Validation and Model Credibility

7.8.1 Verification versus Validation

Future research models require a stronger assurance process than the present training equations. In ECSS terminology, verification demonstrates with objective evidence that a product conforms to its specified requirements, whereas validation addresses suitability for the intended operational use [60][61]. Simulation-model validation additionally asks whether the model represents the simulated item at the required fidelity. ECSS simulation guidance explicitly calls for a documented model-validation programme and correlation against suitable reference results [62].

7.8.2 Verification Activities

Verification should be performed at model, interface and integrated-system level. Unit tests confirm equations, bounds and state transitions. Interface tests confirm time, unit and command semantics. Integration tests confirm that cross-subsystem couplings conserve or account for the relevant quantities and that command execution produces the intended causal response. Regression tests protect previously validated behaviour when models are modified.

Table 7-6 - Example verification and validation matrix for future MCS26 models

Area	Verification evidence	Validation/reference evidence
EPS	Power-balance equations, unit tests, switch logic, numerical bounds	Manufacturer curves, electrical test data, trusted power-system model or mission telemetry with known semantics
Battery	SOC integration, current sign, energy conservation, limit logic	Cell/pack cycling data, OCV-SOC curves, thermal tests
TCS	Thermal matrix assembly, energy accounting, solver stability	Thermal-vacuum data, validated thermal analysis, equipment temperature test data
AOCS	Quaternion normalisation, frame transforms, actuator saturation, deterministic propagation	Reference dynamics tool, hardware/sensor data, manoeuvre telemetry
Orbit/GS	Coordinate transforms, event detection, range/Doppler formulae	Reference ephemeris, ESA GODOT or independent propagator comparison
COMMS	Link-budget arithmetic, state-machine transitions	RF subsystem specifications, link tests or trusted mission link budget
Payload	Readiness logic, data-volume accounting, boresight geometry	Instrument performance model, calibration/test data
C&DH	Queue ordering, memory accounting, command verification, packet timing	Flight-software/SVF behaviour, reference packet streams or integration tests

7.8.3 Cross-Subsystem Invariants

Integrated models should be checked against engineering invariants rather than only against plotted telemetry. Examples include quaternion norm near unity, energy balance between generated power/load/storage, non-negative memory capacity, conservation of file size during queue transfer, bounded reaction-wheel momentum, consistent station range and Doppler sign, and thermal energy changes compatible with applied heat flows. Such invariants detect model-integration errors that can look superficially plausible on individual panels.

7.8.4 Validation Data and the Role of ESA-AD

ESA-AD can validate MCS26's data-ingestion, replay, anomaly-handling and machine-learning pipeline because those activities operate directly on the released telemetry. It cannot, by itself, validate an EPS, TCS or AOCS physical model where channel semantics and units are anonymised. Physical-model validation requires reference data whose engineering meaning, configuration and boundary conditions are known.

7.8.5 Uncertainty and Fidelity Statement

Each model should carry an explicit fidelity statement: what physics is represented, what is approximated, what is omitted, the valid operating domain, source of parameters and expected uncertainty. Increasing the number of equations without quantifying these limits does not increase scientific credibility.

7.9 Data Engineering for Large-Scale Telemetry

7.9.1 Storage Strategy

A multi-billion-point telemetry source requires a storage architecture different from the current in-memory training model. Raw data should remain in its source files; analysis-friendly derived stores can use columnar/chunked formats and time partitioning. The browser should request bounded windows from a backend service rather than loading complete mission histories.

7.9.2 Indexing and Query Model

The backend should index by mission/source, channel, time interval, event label and subsystem/group. Operator panels usually need the latest value of tens or hundreds of channels, whereas anomaly analysis may need long windows from a selected subset. These are different query patterns and should be served separately.

7.9.3 Decimation for Visualisation

Plotting every raw sample is neither necessary nor possible at mission scale. Visualisation services may decimate or aggregate samples to the pixel/time resolution of the graph, but the reduction algorithm and its parameters should be known. Extremes and short anomalies can disappear under naive averaging, so research displays should use methods that preserve relevant minima/maxima or event boundaries.

7.9.4 Provenance and Reproducibility

Every displayed or analysed value should be traceable to a source dataset/model version and transformation chain. Research results should record dataset version, channel subset, preprocessing, time interval, model configuration and software commit. Without that provenance, comparisons between anomaly algorithms or operator experiments cannot be reproduced.

7.10 Anomaly Detection and AI-Assisted Operations Research

7.10.1 ESA-ADB as a Benchmark rather than only a Dataset

ESA-ADB includes more than the telemetry files. It provides a reproducible evaluation pipeline, operationally motivated requirements and metrics developed with spacecraft operations engineers, and reference experiments for time-series anomaly-detection algorithms [50][51]. ESA/ESOC identifies anomaly detection as part of its Artificial Intelligence for Automation roadmap [54]. A future MCS26 research mode could therefore integrate algorithm outputs into the operator environment while retaining ESA-ADB as the independent benchmark basis.

7.10.2 Advisory, Not Autonomous Commanding

The first integration should be advisory. An algorithm can publish an anomaly score, affected-channel ranking, confidence and event onset estimate to an SOE research panel. The human operator remains responsible for interpretation and procedure decisions. Automated command generation should not be added until the monitoring problem, false-alarm behaviour and operational authority model have been separately validated.

7.10.3 Relevant Research Questions

Examples include detection delay versus false alarms; recognition of rare nominal events; channel localisation; anomaly classification; forecasting of telemetry beyond the current time; operator response with and without AI assistance; explainability of multivariate detections; robustness to communication gaps; and computational feasibility under real-time replay. These questions are directly supported by the characteristics and annotations of ESA-AD [50][51].

7.10.4 Avoiding Data Leakage

When using annotated datasets, the operator/algorithm under test must not receive future labels or post-event data during an online-monitoring experiment. Training, validation and test intervals must be kept separate according to the experiment design. ESA-ADB itself uses mission-specific chronological splits for benchmark evaluation [51]. MCS26 should reproduce those rules when claiming comparable results.

7.11 From Simulator to Digital Twin: Terminology and Required Evidence

7.11.1 Why the Current System Is Not a Digital Twin

MCS26_V26_01 is not synchronised with an identified physical spacecraft, does not assimilate timely measurements from such an asset, and its subsystem equations have not been calibrated and validated

against a specific flight vehicle. Calling the present system a digital twin would therefore overstate its engineering status.

7.11.2 Conditions for a Future Digital-Twin Claim

NASA describes digital twins as real-time virtual replicas and emphasises the integration of observations, models and forecasting for monitoring and prediction [58]. ESA's digital-engineering direction similarly emphasises structured digital representations and digital continuity across the lifecycle [59]. For MCS26, a defensible spacecraft digital-twin research path would require at least an identified physical asset, configuration-controlled model of that asset, continuous or timely asset data, model calibration, validated state synchronisation, uncertainty characterisation, predictive capability and traceable lifecycle data.

Even after those capabilities are implemented, the intended use matters. A model validated for operator training is not automatically validated for fault prognosis, commanding or safety decisions. Verification and validation must be tied to the claimed use [60][61].

7.11.3 Recommended Terminology during Intermediate Development

Table 7-7 - Recommended terminology as fidelity increases

Development state	Recommended description	Terms to avoid without further evidence
Current MCS26	Educational / training spacecraft-operations simulator	Digital twin, flight twin, operational MCS
ESA-AD integration	Real-telemetry replay and anomaly-analysis environment	Digital twin; commandable flight simulation
Python subsystem models	Physics-based or semi-physical spacecraft simulator	Flight-representative unless validated to a specific design
Calibrated integrated model	High-fidelity / validated simulator for the stated domain	Digital twin if no physical-asset synchronisation exists
Asset-linked, continuously assimilated, validated model	Digital-twin research platform, subject to defined validation scope	Flight-qualified or safety-authoritative unless separately certified/qualified

7.12 Proposed Staged Development Roadmap

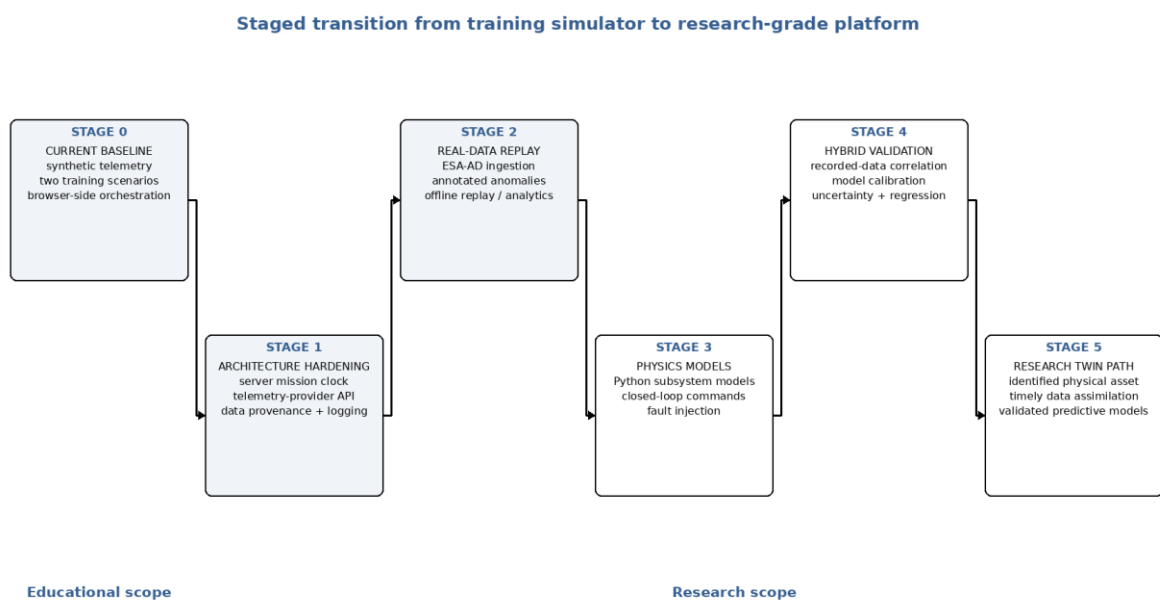


Figure 7-2 - Staged transition from the current educational simulator to a possible research-grade asset-linked platform.

7.12.1 Stage 0 - Preserve the Current Training Baseline

The existing simulator should remain a reproducible baseline while research development occurs on separate branches. Current Scenario 1 and Scenario 2 are valuable acceptance tests for UI, role coordination and procedure behaviour. Future architectural changes should demonstrate that these baseline exercises still operate correctly unless a deliberate requirements change has been approved.

7.12.2 Stage 1 - Architecture Hardening

Move mission time, procedure state and event scheduling to the host; define the telemetry-provider schema; add structured logging and session IDs; add deterministic checkpoint/restore; and establish automated tests for current scenario behaviour. This stage should occur before large external datasets or physical models are introduced.

7.12.3 Stage 2 - Real-Telemetry Replay

Implement the ESA-AD adapter, dataset manifest, server-side time-window reader and replay controls. Add a generic real-telemetry panel that preserves anonymous channel identities. Integrate event annotations into post-run review and anomaly-detection research. Add OPS-SAT support as a second adapter to prove that the interface is dataset-independent.

7.12.4 Stage 3 - Independent Physics-Based Subsystem Models

Develop and validate subsystem models independently before full integration. EPS/battery and thermal models are suitable early candidates because the present scenarios already contain strong power-thermal coupling. Orbit/ground geometry is another high-value model because it can replace the current scripted pass and produce physically derived elevation, range and Doppler. AOCS and payload models can then consume the same environment state.

7.12.5 Stage 4 - Integrated Closed-Loop Simulator

Connect the models through the common simulation scheduler and command interface. At this stage SPACON commands should change physical model state; SOM procedures should evaluate derived telemetry; faults should propagate across subsystem interfaces; and C&DH/COMMS should determine what the ground can actually observe.

7.12.6 Stage 5 - Correlation, Calibration and Hybrid Experiments

Use reference test data or non-anonymised engineering data to calibrate model parameters and quantify error. Introduce replay-to-model branching only where initial-state estimation is sufficient. Add Monte Carlo variation and uncertainty propagation so that procedures are evaluated against a range of credible conditions rather than one deterministic trajectory.

7.12.7 Stage 6 - Asset-Linked Digital-Twin Research, If Data Access Exists

Only if access to a real spacecraft or representative engineering asset becomes available should the project consider continuous state assimilation and digital-twin terminology. The asset interface should be read-only during early research. Any path toward real commanding would require a separate security, qualification, operational authority and verification programme far beyond the scope of the current project.

Table 7-8 - Roadmap deliverables and acceptance gates

Stage	Principal deliverable	Acceptance gate before progression
0	Stable current training baseline	Scenarios 1 and 2 reproduce documented procedure outcomes.
1	Host-authoritative simulation kernel + provider API	Deterministic replay/checkpoint; all clients use one time/state source.
2	ESA-AD and secondary recorded-data adapters	Reproducible time-window replay with provenance and event labels; no semantic relabelling without evidence.
3	Validated independent subsystem models	Model-level requirements, tests, parameter sources and validation reports available.
4	Integrated closed-loop simulator	Cross-subsystem invariants and command-response regression tests pass.

Stage	Principal deliverable	Acceptance gate before progression
5	Calibrated/hybrid research configuration	Model error and uncertainty quantified against appropriate reference data.
6	Asset-linked research twin candidate	Identified asset, timely observations, configuration control and validation for a defined intended use.

7.13 Recommended Research Priorities for MCS26

The highest-value near-term research sequence is not to replace every telemetry row simultaneously. It is to establish a robust simulation/data architecture and then choose a small number of models or datasets that can be validated well. A recommended order is: host-authoritative time and state; telemetry-provider interface; ESA-AD replay; server-side history/logging; physical orbit/ground geometry; EPS/battery model; thermal network; commandable payload/C&DH; and finally closed-loop AOCS and communications.

This order creates useful research results at each stage. The ESA-AD integration enables anomaly-detection and operator-monitoring experiments without waiting for a full spacecraft model. Orbit/ground geometry immediately increases the physical credibility of AOS/LOS and Doppler. EPS and TCS then provide a causal basis for the central contingency logic already present in Scenario 2. AOCS and payload can subsequently use the same time, power, thermal and environmental state.

7.14 Final Development Perspective

The present MCS26_V26_01 should be preserved as an educational operations simulator whose strength is the integration of role-based procedures, distributed consoles and cross-subsystem mission logic. Its future potential lies in making the telemetry source and spacecraft behaviour progressively replaceable without losing that operations layer.

Two extensions are technically credible and complementary. Real ESA telemetry can transform the system into a research environment for monitoring, anomaly detection and long-duration telemetry analysis. Physics-based subsystem models can transform it into a commandable spacecraft simulator in which telemetry is the consequence of explicit states and equations. A hybrid configuration can connect those paths, but only with controlled provenance, state estimation and model validation.

The transition from training simulator to research platform should therefore be driven by evidence rather than terminology. More data does not by itself create a digital twin, and more equations do not by themselves create fidelity. The required progression is controlled architecture, traceable data, explicit interfaces, validated models, quantified uncertainty and reproducible experiments. Only after those elements exist - and only if the platform is connected to and validated against an identified physical asset - would digital-twin terminology become technically defensible.

8 References

- [1] European Cooperation for Space Standardization (ECSS), ECSS-E-ST-70C, “Ground systems and operations,” 31 July 2008. [Online]. Available: <https://ecss.nl/standard/ecss-e-st-70c-ground-systems-and-operations/>
- [2] European Cooperation for Space Standardization (ECSS), ECSS-E-ST-10C Rev.1, “System engineering general requirements,” 15 February 2017, Annex P, Product User Manual (PUM or UM). [Online]. Available: <https://ecss.nl/standard/ecss-e-st-10c-rev-1-system-engineering-general-requirements-15-february-2017/>
- [3] European Cooperation for Space Standardization (ECSS), ECSS-E-ST-40C Rev.1, “Software,” 30 April 2025. [Online]. Available: <https://ecss.nl/standard/ecss-e-st-40c-rev-1-software-30-april-2025/>
- [4] German Aerospace Center (DLR), “DLR_School_Lab TU Darmstadt” and “Kontrollzentrum,” DLR_School_Lab TU Darmstadt. Accessed: 26 August 2026. [Online]. Available: https://www.dlr.de/de/schoollab/standorte/tu_darmstadt ; https://www.dlr.de/de/schoollab/standorte/tu_darmstadt/experimente/kontrollzentrum
- [5] European Space Agency (ESA), “Ladybird Guide to Spacecraft Operations Training Course,” ESA Academy. Accessed: 26 August 2026. [Online]. Available: https://www.esa.int/Education/ESA_Academy/Apply_for_the_Ladybird_Guide_to_Spacecraft_Operations_Training_Course
- [6] Telespazio VEGA Deutschland, “Learn to fly a satellite like a pro in the new DLR School Lab of the TU Darmstadt,” 21 September 2017. [Online]. Available: https://www.telespazio.de/en/c/portal/update_language?languageId=de_DE&p_l_id=13817547&redirect=%2Fen%2Fnews-and-stories-detail%2F-%2Fdetail%2Fdlr-school-lab-tu-darmstadt
- [7] Telespazio Germany, “Training Software Infrastructure – VEGA Emulation Technology (VET4).” Accessed: 26 August 2026. [Online]. Available: <https://www.telespazio.de/en/business/aviation/training-solutions/training-software-infrastructure>
- [8] Telespazio Germany, “Spacecraft Simulators & Simulation Infrastructure.” Accessed: 26 August 2026. [Online]. Available: <https://www.telespazio.de/en/business/space/simulations>
- [9] Telespazio VEGA Deutschland, “Telespazio embarks on Mission to Mercury,” 17 October 2018. [Online]. Available: <https://www.telespazio.de/en/news-and-stories-detail/-/detail/mission-to-mercury>
- [10] European Space Agency / European Space Operations Centre (ESA/ESOC), BR-265, ground-segment integration, satellite simulators and simulations programme, 2007. [Online]. Available: <https://www.esa.int/esapub/br/br265/br265.pdf>
- [11] OpenJS Foundation, “Node.js Releases.” Accessed: 26 August 2026. [Online]. Available: <https://nodejs.org/en/about/previous-releases>
- [12] Vite, “Vite 8.0 is out! – Node.js Support,” 2026. Accessed: 26 August 2026. [Online]. Available: <https://vite.dev/blog/announcing-vite8>
- [13] Vite, “Getting Started – Browser Support” and “Building for Production – Browser Compatibility.” Accessed: 26 August 2026. [Online]. Available: <https://vite.dev/guide/> ; <https://vite.dev/guide/build>
- [14] Git for Windows Project, “Git for Windows.” Accessed: 26 August 2026. [Online]. Available: <https://gitforwindows.org/>
- [15] npm, “package-lock.json.” Accessed: 26 August 2026. [Online]. Available: <https://docs.npmjs.com/cli/v11/configuring-npm/package-lock-json/>
- [16] Vite, “Server Options.” Accessed: 26 August 2026. [Online]. Available: <https://vite.dev/config/server-options>
- [17] Microsoft, “Configure Firewall Rules with Group Policy,” Microsoft Learn. Accessed: 26 August 2026. [Online]. Available: <https://learn.microsoft.com/en-us/windows/security/threat-protection/windows-firewall/best-practices-configuring>
- [18] Git Project, “git-clone Documentation.” Accessed: 26 August 2026. [Online]. Available: <https://git-scm.com/docs/git-clone>
- [19] Git Project, “git-pull Documentation.” Accessed: 26 August 2026. [Online]. Available: <https://git-scm.com/docs/git-pull>
- [20] npm, “npm ci – Clean install a project.” Accessed: 26 August 2026. [Online]. Available: <https://docs.npmjs.com/cli/v11/commands/npm-ci/>
- [21] Microsoft, “Test-NetConnection (NetTCP/IP),” Microsoft Learn. Accessed: 26 August 2026. [Online]. Available: <https://learn.microsoft.com/en-us/powershell/module/nettcpip/test-netconnection>

- [22] National Aeronautics and Space Administration (NASA), "Basics of Space Flight, Chapter 11: Onboard Systems." Accessed: 26 August 2026. [Online]. Available: <https://science.nasa.gov/learn/basics-of-space-flight/chapter11-1/> ; <https://science.nasa.gov/learn/basics-of-space-flight/chapter11-2/> ; <https://science.nasa.gov/learn/basics-of-space-flight/chapter11-3/> ; <https://science.nasa.gov/learn/basics-of-space-flight/chapter11-4/>
- [23] European Cooperation for Space Standardization (ECSS), ECSS-E-ST-20-20C, "Electrical design and interface requirements for power supply," 15 April 2016. [Online]. Available: <https://ecss.nl/standard/ecss-e-st-20-20c-space-engineering-electrical-design-and-interface-requirements-for-power-supply-15-april-2016-2/>
- [24] European Cooperation for Space Standardization (ECSS), ECSS-E-ST-31C, "Thermal control," 15 November 2008. [Online]. Available: <https://ecss.nl/standard/ecss-e-st-31c-thermal-control/>
- [25] European Cooperation for Space Standardization (ECSS), ECSS-E-ST-60-30C, "Satellite attitude and orbit control system (AOCS) requirements," 30 August 2013. [Online]. Available: <https://ecss.nl/standard/ecss-e-st-60-30c-satellite-attitude-and-orbit-control-system-aocs-requirements/>
- [26] European Cooperation for Space Standardization (ECSS), ECSS-E-ST-10-09C, "Reference coordinate system," 31 July 2008. [Online]. Available: <https://ecss.nl/standard/ecss-e-st-10-09c-reference-coordinate-system/>
- [27] National Aeronautics and Space Administration (NASA), "Basics of Space Flight, Chapter 10: Telecom." Accessed: 26 August 2026. [Online]. Available: <https://science.nasa.gov/learn/basics-of-space-flight/chapter10-1/>
- [28] European Space Agency (ESA), Flight Dynamics, GODOT 1.20.0 documentation, ground-station definitions and azimuth-dependent elevation masks, 2026. [Online]. Available: https://godot.io.esa.int/docs/api_reference/generated/generated/godot.core.station.html ; https://godot.io.esa.int/docs/api_reference/generated/generated/generated/godot.core.station.Mask.html
- [29] P. Fortescue, G. Swinerd and J. Stark, eds., Spacecraft Systems Engineering, 4th ed. Chichester, UK: Wiley, 2011.
- [30] D. A. Vallado, Fundamentals of Astrodynamics and Applications, 4th ed. Hawthorne, CA, USA: Microcosm Press, 2013.
- [31] National Aeronautics and Space Administration (NASA), Small Spacecraft Systems Virtual Institute, "State-of-the-Art of Small Spacecraft Technology: Guidance, Navigation, and Control," updated 7 May 2026. [Online]. Available: <https://www.nasa.gov/smallsat-institute/sst-soa/guidance-navigation-and-control/>
- [32] European Cooperation for Space Standardization (ECSS), ECSS-E-ST-70-41C, "Telemetry and telecommand packet utilization," 15 April 2016. [Online]. Available: <https://ecss.nl/standard/ecss-e-st-70-41c-space-engineering-telemetry-and-telecommand-packet-utilization-15-april-2016/>
- [33] European Cooperation for Space Standardization (ECSS), ECSS-E-ST-70-31C, "Ground systems and operations – Monitoring and control data definition," 31 July 2008. [Online]. Available: <https://ecss.nl/standard/ecss-e-st-70-31c-ground-systems-and-operations-monitoring-and-control-data-definition/>
- [34] National Aeronautics and Space Administration (NASA), "Basics of Space Flight, Chapter 12: Science Instruments." Accessed: 26 August 2026. [Online]. Available: <https://science.nasa.gov/learn/basics-of-space-flight/chapter12-1/>
- [35] European Space Agency (ESA), "Sentinel-2 Instrument – Multispectral Imager." Accessed: 26 August 2026. [Online]. Available: https://www.esa.int/Applications/Observing_the_Earth/Copernicus/Sentinel-2/Instrument
- [36] National Aeronautics and Space Administration (NASA), State of the Art Small Spacecraft Technology, 2018, Section 8, "Command and Data Handling." [Online]. Available: https://www.nasa.gov/wp-content/uploads/2017/03/soa2018_final_doc.pdf
- [37] European Cooperation for Space Standardization (ECSS), ECSS-E-ST-50-12C Rev.1, "SpaceWire – Links, nodes, routers and networks," 15 May 2019. [Online]. Available: <https://ecss.nl/wp-content/uploads/2019/05/ECSS-E-ST-50-12C-Rev.1%2815May2019%29.pdf>
- [38] National Aeronautics and Space Administration (NASA), "Basics of Space Flight, Chapter 8: Experiments – Science and Engineering Data." Accessed: 26 August 2026. [Online]. Available: <https://science.nasa.gov/learn/basics-of-space-flight/chapter8-1/>
- [39] European Cooperation for Space Standardization (ECSS), ECSS-E-HB-20-02A, "Li-ion battery testing handbook," 1 October 2015. [Online]. Available: <https://ecss.nl/hbstms/ecss-e-hb-20-02a-li-ion-battery-testing-handbook-1-october-2015/>
- [40] European Cooperation for Space Standardization (ECSS), ECSS-E-ST-20-08C Rev.2, "Photovoltaic assemblies and components," 20 April 2023. [Online]. Available: <https://ecss.nl/standards/active-standards/>
- [41] NASA Goddard Space Flight Center / University of California, Berkeley, "HESSI Berkeley Ground Station," station-mask and acquisition-visibility information. Accessed: 26 August 2026. [Online]. Available: https://hesperia.gsfc.nasa.gov/rhessi3/ssl_site/ground_systems/station.html

- [42] Google, "Chrome DevTools: Open DevTools" and "Network features reference." Accessed: 26 August 2026. [Online]. Available: <https://developer.chrome.com/docs/devtools/open> ; <https://developer.chrome.com/docs/devtools/network>
- [43] Vite, "Server Options" and "Command Line Interface." Accessed: 26 August 2026. [Online]. Available: <https://vite.dev/config/server-options> ; <https://vite.dev/guide/cli>
- [44] OpenJS Foundation, "Node.js Errors – Common system errors," Node.js documentation. Accessed: 26 August 2026. [Online]. Available: <https://nodejs.org/api/errors.html>
- [45] Microsoft, "Configure Windows Firewall Rules," Microsoft Learn, updated 2026. Accessed: 26 August 2026. [Online]. Available: <https://learn.microsoft.com/en-us/windows/security/operating-system-security/network-security/windows-firewall/configure>
- [46] Microsoft, "Get-NetTCPConnection (NetTCPIP)," Microsoft Learn. Accessed: 26 August 2026. [Online]. Available: <https://learn.microsoft.com/en-us/powershell/module/nettcpip/get-nettcpconnection>
- [47] Mozilla Developer Network (MDN), "Autoplay guide for media and Web Audio APIs." Accessed: 26 August 2026. [Online]. Available: <https://developer.mozilla.org/en-US/docs/Web/Media/Guides/Autoplay>
- [48] npm, "npm cache." Accessed: 26 August 2026. [Online]. Available: <https://docs.npmjs.com/cli/cache/>
- [49] Socket.IO, "Socket.IO documentation and product overview." Accessed: 26 August 2026. [Online]. Available: <https://socket.io/>
- [50] European Space Agency Anomaly Detection Benchmark (ESA-ADB), "Materials." Accessed: 26 August 2026. [Online]. Available: <https://ai4gs.space-codex.org/esa-adb/materials/>
- [51] K. Kotowski, C. Haskamp, J. Andrzejewski, B. Ruszczak, J. Nalepa, D. Lakey, P. Collins, J. Martínez-Heras and G. De Canio, "European Space Agency Dataset and Benchmark for Real-World Anomaly Detection in Spacecraft Time Series," Data-centric Machine Learning Research, vol. 3, paper 23, 2026. [Online]. Available: <https://data.mlr.press/volumes/03.html> ; <https://openreview.net/forum?id=bbpRMoatVO>
- [52] European Space Agency (ESA), "ESA Anomaly Dataset," Zenodo, DOI: 10.5281/zenodo.12528695; version record DOI: 10.5281/zenodo.12528696, first public release June 2024. [Online]. Available: <https://doi.org/10.5281/zenodo.12528695> ; <https://zenodo.org/records/12528696>
- [53] European Space Operations Centre (ESOC), "ESA releases the building block of an open database of satellite anomalies," 2024. [Online]. Available: <https://esoc.esa.int/esa-releases-building-block-open-database-satellite-anomalies>
- [54] European Space Operations Centre (ESOC), "Artificial Intelligence for Automation (A2I) Roadmap." Accessed: 26 August 2026. [Online]. Available: <https://esoc.esa.int/a2i-roadmap-0>
- [55] B. Ruszczak et al., "OPSSAT-AD – anomaly detection dataset for satellite telemetry," Zenodo, version 2, 24 June 2024. [Online]. Available: <https://zenodo.org/records/15108715>
- [56] European Cooperation for Space Standardization (ECSS), ECSS-E-ST-40-07C Rev.1, "Simulation modelling platform – Level 1," 5 August 2025. [Online]. Available: <https://ecss.nl/standard/ecss-e-st-40-07c-rev-1-simulation-modelling-platform-level-1-5-august-2025/>
- [57] European Space Agency (ESA), "Simulation Model Portability (SMP)," modelling and simulation background and principles. Accessed: 26 August 2026. [Online]. Available: https://www.esa.int/TEC/Modelling_and_simulation/TEC2DCNWTPE_2.html
- [58] National Aeronautics and Space Administration (NASA), "Why does the world (and NASA) need digital twins?," 2025. [Online]. Available: <https://science.nasa.gov/biological-physical/why-does-the-world-and-nasa-need-digital-twins/>
- [59] European Space Agency (ESA), "Digital modelling to build space missions faster," 2023. [Online]. Available: https://www.esa.int/Enabling_Support/Space_Engineering_Technology/Digital_modelling_to_build_space_missions_faster
- [60] European Cooperation for Space Standardization (ECSS), ECSS-E-ST-10-02C Rev.1, "Verification," 1 February 2018. [Online]. Available: <https://ecss.nl/standard/ecss-e-st-10-02c-rev-1-verification-1-february-2018/>
- [61] European Cooperation for Space Standardization (ECSS), ECSS-S-ST-00-01C glossary entries, "Verification" and "Validation." Accessed: 26 August 2026. [Online]. Available: https://ecss.nl/item/?glossary_id=2690 ; https://ecss.nl/item/?glossary_id=2689
- [62] European Cooperation for Space Standardization (ECSS), ECSS-E-TM-40-07 Volume 1A, "Simulation modelling platform – simulation model validation guidance," 25 January 2011. [Online]. Available: <https://ecss.nl/wp-content/uploads/standards/ecss-e/ECSS-E-TM-40-07-Volume1A25January2011.pdf>